Puddle rewrite catalog

Generated on 2026-10-22 from the working tree at `5d4a6b8a9e8da2262f08025fc51307622beb530a` (`math-fehr/instsimplify-proofs`).

1,703 renderings, covering all **534 discovered production `Pattern OpCode` declarations** under `Veir/`: 472 builder definitions, 3 direct record definitions, and 59 aliases or dispatchers. The renderings also include concrete instances extracted from compiled pass patterns. Generic builders are covered through the displayed instances; compiler-generated auxiliary declarations and unit tests are excluded.

The output comes from the Lean Puddle pretty printer, after simplifying builder state with the rules used by `unfoldPuddleBuilder`. Every rendering retains the complete `where` block, including types, properties, guards, and native computations. This is reading notation, not an input format.

Parameter coverage

- Scalar parameters remain symbolic when they do not determine graph structure. Boolean graph choices and the supported finite opcode/shape selectors are expanded.
- Scalar-literal families cover integer, zero-marker, and poison constructors. For `IS5122`, integer conditions use `0` and `1` to represent the two possible i1 values; selected integer payloads remain symbolic.
- Unbounded GEP families use representative graph sizes: 0–3 dynamic indices, 0–2 zero-index payloads, and 0–2 dynamic indices on each side of a poison index. These samples do not enumerate every arity. The two `Core` definitions are direct-record versions of builder families and are included too.
- Generic pass helpers use their concrete pass instances. Comparison lowering includes its supported 8-, 32-, and 64-bit forms; widening keeps the target width symbolic. Arbitrary combinations of opcode/helper parameters are not enumerated.

Coverage

Area	Builder definitions	Other Pattern declarations	Renderings
InstSimplify	422	17	1,403
RISC-V instruction selection	34	45	146
LLVM to GMIR	5	0	17
Legalization	3	0	19
RISC-V combines	8	0	118
Total	**472**	**62**	**1,703**

Coverage was checked against the elaborated Lean declarations: no discovered production pattern declaration is omitted, and no rendering failed.

Module index

Module	Renderings
Veir.InstSimplify.GEPDynamicSchema	8
Veir.InstSimplify.GEPPoisonSchema	9
Veir.InstSimplify.GEPZeroSchema	6
Veir.InstSimplify.IS0582	20
Veir.InstSimplify.IS0601	2
Veir.InstSimplify.IS0609	2
Veir.InstSimplify.IS0621	2
Veir.InstSimplify.IS0626	4
Veir.InstSimplify.IS0633	1
Veir.InstSimplify.IS0637	1
Veir.InstSimplify.IS0754	2
Veir.InstSimplify.IS0763	1
Veir.InstSimplify.IS0767	1
Veir.InstSimplify.IS0773	1
Veir.InstSimplify.IS0886	1
Veir.InstSimplify.IS0906	2
Veir.InstSimplify.IS0911	2
Veir.InstSimplify.IS0915	2
Veir.InstSimplify.IS0923	4
Veir.InstSimplify.IS0929	1
Veir.InstSimplify.IS1057	4
Veir.InstSimplify.IS1063	4
Veir.InstSimplify.IS1068	4
Veir.InstSimplify.IS1078	4
Veir.InstSimplify.IS1083	4
Veir.InstSimplify.IS1114	16
Veir.InstSimplify.IS1168	2
Veir.InstSimplify.IS1189	2
Veir.InstSimplify.IS1204	2
Veir.InstSimplify.IS1246	1
Veir.InstSimplify.IS1310	3
Veir.InstSimplify.IS1314	3
Veir.InstSimplify.IS1322	6
Veir.InstSimplify.IS1326	3
Veir.InstSimplify.IS1381	2
Veir.InstSimplify.IS1417	2
Veir.InstSimplify.IS1421	1
Veir.InstSimplify.IS1430	1
Veir.InstSimplify.IS1451	1
Veir.InstSimplify.IS1490	2
Veir.InstSimplify.IS1495	1
Veir.InstSimplify.IS1534	8
Veir.InstSimplify.IS1539	8
Veir.InstSimplify.IS1545	16
Veir.InstSimplify.IS1551	16
Veir.InstSimplify.IS1598	8
Veir.InstSimplify.IS1603	8
Veir.InstSimplify.IS1608	4
Ve[illegible]nfo	1[illegible]

InstSimplify

Veir.InstSimplify.GEPDynamicSchema

Source module

Veir.InstSimplify.gepPoisonBaseFamily

Source

Variant 1: `n = 0`

```
llvm: getelementptr(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0) : T0
  P1 = properties(root)
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0
  T0 : Veir.LLVM.PointerType
```

Variant 2: `n = 1`

```
llvm: getelementptr(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X) : T0
  P1 = properties(root)
  X : T1
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Variant 3: `n = 2`

```
llvm: getelementptr(a0, X, Y)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X, Y) : T0
  P1 = properties(root)
  X : T1
  Y : T2
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Variant 4: `n = 3`

```
llvm: getelementptr(a0, X, Y, Z)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X, Y, Z) : T0
  P1 = properties(root)
  X : T1
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 3
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.gepPoisonBaseCore

Source

Variant 1: `n = 0`

```
llvm: getelementptr(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0) : T0
```

```
  P1 = properties(root)
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0
  T0 : Veir.LLVM.PointerType
```

Variant 2: `n = 1`

```
llvm: getelementptr(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X) : T0
  P1 = properties(root)
  X : T1
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Variant 3: `n = 2`

```
llvm: getelementptr(a0, X, Y)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X, Y) : T0
  P1 = properties(root)
  X : T1
  Y : T2
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Variant 4: `n = 3`

```
llvm: getelementptr(a0, X, Y, Z)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X, Y, Z) : T0
  P1 = properties(root)
  X : T1
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 3
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.GEPPoisonSchema

Source module

Veir.InstSimplify.gepPoisonIndexFamily

Source

Variant 1: `before = 0; after = 0`

```
llvm: getelementptr(X, a0)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0 + 1 + 0
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 2: `before = 0; after = 1`

```
llvm: getelementptr(X, a0, Y)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, a0, Y) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0 + 1 + 1
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 3: `before = 0; after = 2`

```
llvm: getelementptr(X, a0, Y, Z)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, a0, Y, Z) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0 + 1 + 2
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 4: `before = 1; after = 0`

```
llvm: getelementptr(X, Y, a0)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 + 1 + 0
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 5: `before = 1; after = 1`

```
llvm: getelementptr(X, Y, a0, Z)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, a0, Z) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 + 1 + 1
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 6: `before = 1; after = 2`

```
llvm: getelementptr(X, Y, a0, Z, X3)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, a0, Z, X3) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  X3 : T4
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 + 1 + 2
  T4 : Veir.IntegerType
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 7: `before = 2; after = 0`

```
llvm: getelementptr(X, Y, Z, a0)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, Z, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2 + 1 + 0
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 8: `before = 2; after = 1`

```
llvm: getelementptr(X, Y, Z, a0, X3)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, Z, a0, X3) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  X3 : T4
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2 + 1 + 1
  T4 : Veir.IntegerType
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 9: `before = 2; after = 2`

```
llvm: getelementptr(X, Y, Z, a0, X3, X4)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, Z, a0, X3, X4) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  X3 : T4
```

```
  X4 : T5
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2 + 1 + 2
  T5 : Veir.IntegerType
  T4 : Veir.IntegerType
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Veir.InstSimplify.GEPZeroSchema

Source module

Veir.InstSimplify.gepAllZeroFamily

Source

Variant 1: `payloads = []; representative length 0, a b : Int`

```
llvm: getelementptr(X)  =>  X
where
  root = getelementptr(X) : T0
  P0 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P0.rawConstantIndices.values).size == 0 &&
    P0.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  T0 : Veir.LLVM.PointerType
```

Variant 2: `payloads = [a]; representative length 1, a b : Int`

```
llvm: getelementptr(X, a)  =>  X
where
  C0 = a : T1
  P0 = properties(C0)
  root = getelementptr(X, C0) : T0
  P1 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 &&
    P1.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: BitVec.ofInt t.bitwidth a == 0
  T0 : Veir.LLVM.PointerType
```

Variant 3: `payloads = [a, b]; representative length 2, a b : Int`

```
llvm: getelementptr(X, a, b)  =>  X
where
  C0 = a : T1
  P0 = properties(C0)
  C1 = b : T2
  P1 = properties(C1)
  root = getelementptr(X, C0, C1) : T0
  P2 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P2.rawConstantIndices.values).size == 1 + 1 &&
    P2.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  require Veir.InstSimplify.intConstantGuard P1 b
  T2 : Veir.IntegerType, where t = T2: BitVec.ofInt t.bitwidth b == 0
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: BitVec.ofInt t.bitwidth a == 0
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.gepAllZeroCore

Source

Variant 1: `payloads = []; representative length 0, a b : Int`

```
llvm: getelementptr(X)  =>  X
where
  root = getelementptr(X) : T0
  P0 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P0.rawConstantIndices.values).size == 0 &&
```

```
    P0.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  T0 : Veir.LLVM.PointerType
```

Variant 2: `payloads = [a]; representative length 1, a b : Int`

```
llvm: getelementptr(X, a)  =>  X
where
  C0 = a : T1
  P0 = properties(C0)
  root = getelementptr(X, C0) : T0
  P1 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 &&
    P1.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: BitVec.ofInt t.bitwidth a == 0
  T0 : Veir.LLVM.PointerType
```

Variant 3: `payloads = [a, b]; representative length 2, a b : Int`

```
llvm: getelementptr(X, a, b)  =>  X
where
  C0 = a : T1
  P0 = properties(C0)
  C1 = b : T2
  P1 = properties(C1)
  root = getelementptr(X, C0, C1) : T0
  P2 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P2.rawConstantIndices.values).size == 1 + 1 &&
    P2.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  require Veir.InstSimplify.intConstantGuard P1 b
  T2 : Veir.IntegerType, where t = T2: BitVec.ofInt t.bitwidth b == 0
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: BitVec.ofInt t.bitwidth a == 0
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS0582

Source module

Veir.InstSimplify.IS0582.pattern_add

Source

```
llvm: (C0 + C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = (C0 + C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  P3 ← Veir.InstSimplify.IS0582.fold_add† T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_sub

Source

```
llvm: (C0 - C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = (C0 - C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  P3 ← Veir.InstSimplify.IS0582.fold_sub† T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_mul

Source

```
llvm: (C0 * C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = (C0 * C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  P3 ← Veir.InstSimplify.IS0582.fold_mul† T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_shl

Source

```
llvm: shl(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = shl(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS0582.constantDefined_shl† T0 P0 P1
  P3 ← Veir.InstSimplify.IS0582.fold_shl† T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_lshr

Source

```
llvm: lshr(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = lshr(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS0582.constantDefined_lshr† T0 P0 P1
  P3 ← Veir.InstSimplify.IS0582.fold_lshr† T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_ashr

Source

```
llvm: ashr(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = ashr(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS0582.constantDefined_ashr† T0 P0 P1
  P3 ← Veir.InstSimplify.IS0582.fold_ashr† T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_and

Source

```
llvm: (C0 & C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = (C0 & C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  P3 ← Veir.InstSimplify.IS0582.fold_and✝ T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_or

Source

```
llvm: (C0 | C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = (C0 | C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  P3 ← Veir.InstSimplify.IS0582.fold_or✝ T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_xor

Source

```
llvm: (C0 ^ C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = (C0 ^ C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  P3 ← Veir.InstSimplify.IS0582.fold_xor✝ T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_udiv

Source

```
llvm: udiv(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = udiv(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS0582.constantDefined_udiv✝ T0 P0 P1
  P3 ← Veir.InstSimplify.IS0582.fold_udiv✝ T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_sdiv

Source

```
llvm: sdiv(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
```

```
  root = sdiv(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS0582.constantDefined_sdiv↑ T0 P0 P1
  P3 ← Veir.InstSimplify.IS0582.fold_sdiv↑ T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_urem

Source

```
llvm: urem(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = urem(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS0582.constantDefined_urem↑ T0 P0 P1
  P3 ← Veir.InstSimplify.IS0582.fold_urem↑ T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_srem

Source

```
llvm: srem(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = srem(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS0582.constantDefined_srem↑ T0 P0 P1
  P3 ← Veir.InstSimplify.IS0582.fold_srem↑ T0 P0 P1
```

Veir.InstSimplify.IS0582.pattern_shl_poison

Source

```
llvm: shl(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = shl(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS0582.constantDefined_shl↑ T0 P0 P1
  P3 = PUnit.unit
```

Veir.InstSimplify.IS0582.pattern_lshr_poison

Source

```
llvm: lshr(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = lshr(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
```

```
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS0582.constantDefined_lshr† T0 P0 P1
  P3 = PUnit.unit
```

Veir.InstSimplify.IS0582.pattern_ashr_poison

Source

```
llvm: ashr(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = ashr(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS0582.constantDefined_ashr† T0 P0 P1
  P3 = PUnit.unit
```

Veir.InstSimplify.IS0582.pattern_udiv_poison

Source

```
llvm: udiv(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = udiv(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS0582.constantDefined_udiv† T0 P0 P1
  P3 = PUnit.unit
```

Veir.InstSimplify.IS0582.pattern_sdiv_poison

Source

```
llvm: sdiv(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = sdiv(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS0582.constantDefined_sdiv† T0 P0 P1
  P3 = PUnit.unit
```

Veir.InstSimplify.IS0582.pattern_urem_poison

Source

```
llvm: urem(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = urem(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
```

```
  require !Veir.InstSimplify.IS0582.constantDefined_urem↑ T0 P0 P1
  P3 = PUnit.unit
```

Veir.InstSimplify.IS0582.pattern_srem_poison

Source

```
llvm: srem(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = srem(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS0582.constantDefined_srem↑ T0 P0 P1
  P3 = PUnit.unit
```

Veir.InstSimplify.IS0601

Source module

Veir.InstSimplify.IS0601.pattern

Source

```
llvm: (X + a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (X + a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0601.pattern_left

Source

```
llvm: (a0 + X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (a0 + X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0609

Source module

Veir.InstSimplify.IS0609.pattern

Source

```
llvm: (X + 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (X + C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS0609.pattern_left

Source

```
llvm: (0 + X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (C0 + X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS0621

Source module

Veir.InstSimplify.IS0621.pattern

Source

Variant 1: `commute = false`

```
llvm: (X + (Y - X))  =>  Y
where
  a0 = (Y - X) : T0
  P0 = properties(a0)
  root = (X + a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 2: `commute = true`

```
llvm: ((Y - X) + X)  =>  Y
where
  a0 = (Y - X) : T0
  P0 = properties(a0)
  root = (a0 + X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0626

Source module

Veir.InstSimplify.IS0626.pattern_00

Source

```
llvm: (X + (X ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (X + a1) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS0626.pattern_01

Source

```
llvm: (X + (C0 ^ X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (X + a1) : T0
  P2 = properties(root)
  X : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS0626.pattern_10

Source

```
llvm: ((X ^ C0) + X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (a1 + X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS0626.pattern_11

Source

```
llvm: ((C0 ^ X) + X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (a1 + X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS0633

Source module

Veir.InstSimplify.IS0633.pattern

Source

Parameters: `n : Int; m : Int`

```
llvm: ((X ^ n) + m)  =>  X
where
  C0 = n : T0
  P0 = properties(C0)
  C1 = m : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  root = (a2 + C1) : T0
  P3 = properties(root)
  X : T0
  require P3.nsw || P3.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require Veir.InstSimplify.bitConstantGuard T0 P1 m
  require match T0.val with
  | Veir.Attribute.integerType it =>
    BitVec.ofInt it.bitwidth n == BitVec.intMin it.bitwidth && BitVec.ofInt it.bitwidth m == BitVec.intMin
it.bitwidth
  | x => false
```

Veir.InstSimplify.IS0637

Source module

Veir.InstSimplify.IS0637.pattern

Source

```
llvm: (X + C0)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = (X + C0) : T0
  P1 = properties(root)
  X : T0
  require P1.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS0754

Source module

Veir.InstSimplify.IS0754.pattern_sub_right

Source

```
llvm: (X - a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (X - a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0754.pattern_sub_left

Source

```
llvm: (a0 - X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (a0 - X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0763

Source module

Veir.InstSimplify.IS0763.pattern

Source

```
llvm: (X - 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (X - C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS0767

Source module

Veir.InstSimplify.IS0767.pattern

Source

```
llvm: (X - X)  =>  mlir.zero()
where
  root = (X - X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
```

```
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Veir.InstSimplify.IS0773

Source module

Veir.InstSimplify.IS0773.pattern

Source

```
llvm: (C0 - X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (C0 - X) : T0
  P1 = properties(root)
  X : T0
  require P1.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS0886

Source module

Veir.InstSimplify.IS0886.pattern

Source

Parameters: `n : Int; k : Nat`

```
llvm: (C0 - (X ^ C0))  =>  X
where
  C0 = n : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (C0 - a1) : T0
  P2 = properties(root)
  X : T0
  require P2.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it => decide (0 < k ∧ k ≤ it.bitwidth) && (BitVec.ofInt it.bitwidth n).toNat == 2
^ k - 1
  | x => false
```

Veir.InstSimplify.IS0906

Source module

Veir.InstSimplify.IS0906.pattern

Source

```
llvm: (X * a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (X * a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0906.pattern_left

Source

```
llvm: (a0 * X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (a0 * X) : T0
```

```
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0911

Source module

Veir.InstSimplify.IS0911.pattern

Source

```
llvm: (X * C0)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (X * C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS0911.pattern_left

Source

```
llvm: (C0 * X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (C0 * X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS0915

Source module

Veir.InstSimplify.IS0915.pattern

Source

```
llvm: (X * 1)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  root = (X * C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 1
```

Veir.InstSimplify.IS0915.pattern_left

Source

```
llvm: (1 * X)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  root = (C0 * X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 1
```

Veir.InstSimplify.IS0923

Source module

Veir.InstSimplify.IS0923.pattern_udiv

Source

```
llvm: (udiv(X, Y) * Y)  =>  X
where
  a0 = udiv(X, Y) : T0
  P0 = properties(a0)
  root = (a0 * Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.exact
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0923.pattern_udiv_left

Source

```
llvm: (Y * udiv(X, Y))  =>  X
where
  a0 = udiv(X, Y) : T0
  P0 = properties(a0)
  root = (Y * a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.exact
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0923.pattern_sdiv

Source

```
llvm: (sdiv(X, Y) * Y)  =>  X
where
  a0 = sdiv(X, Y) : T0
  P0 = properties(a0)
  root = (a0 * Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.exact
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0923.pattern_sdiv_left

Source

```
llvm: (Y * sdiv(X, Y))  =>  X
where
  a0 = sdiv(X, Y) : T0
  P0 = properties(a0)
  root = (Y * a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.exact
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS0929

Source module

Veir.InstSimplify.IS0929.pattern

Source

```
llvm: (X * Y)  =>  mlir.zero()
where
  root = (X * Y) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
  Y : T0
  require P0.nsw
  T0 : Veir.IntegerType
  require match T0.val with
```

```
  | Veir.Attribute.integerType it => it.bitwidth == 1
  | x => false
  P1 = PUnit.unit
```

Veir.InstSimplify.IS1057

Source module

Veir.InstSimplify.IS1057.pattern_sdiv

Source

```
llvm: sdiv(X, a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = sdiv(X, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1057.pattern_udiv

Source

```
llvm: udiv(X, a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = udiv(X, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1057.pattern_srem

Source

```
llvm: srem(X, a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = srem(X, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1057.pattern_urem

Source

```
llvm: urem(X, a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = urem(X, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1063

Source module

Veir.InstSimplify.IS1063.pattern_sdiv

Source

```
llvm: sdiv(X, 0)  =>  mlir.poison()
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = sdiv(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1063.pattern_udiv

Source

```
llvm: udiv(X, 0)  =>  mlir.poison()
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = udiv(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1063.pattern_srem

Source

```
llvm: srem(X, 0)  =>  mlir.poison()
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = srem(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1063.pattern_urem

Source

```
llvm: urem(X, 0)  =>  mlir.poison()
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = urem(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1068

Source module

Veir.InstSimplify.IS1068.pattern_sdiv

Source

```
llvm: sdiv(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = sdiv(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1068.pattern_udiv

Source

```
llvm: udiv(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = udiv(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1068.pattern_srem

Source

```
llvm: srem(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = srem(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1068.pattern_urem

Source

```
llvm: urem(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = urem(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1078

Source module

Veir.InstSimplify.IS1078.pattern_sdiv

Source

```
llvm: sdiv(C0, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = sdiv(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1078.pattern_udiv

Source

```
llvm: udiv(C0, X)  =>  C0
where
```

```
  C0 = 0 : T0
  P0 = properties(C0)
  root = udiv(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1078.pattern_srem

Source

```
llvm: srem(C0, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = srem(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1078.pattern_urem

Source

```
llvm: urem(C0, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = urem(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1083

Source module

Veir.InstSimplify.IS1083.pattern_sdiv

Source

```
llvm: sdiv(X, X)  =>  C1
where
  root = sdiv(X, X) : T0
  P0 = properties(root)
  C1 = mlir.constant() : T0 [created]
  P1 = properties(C1)
  X : T0
  T0 : Veir.IntegerType
  P1 ← match T0.val with
  | Veir.Attribute.integerType it => some { value := Veir.LLVMConstantValue.integer { value := 1, type := it } }
  | x => none
```

Veir.InstSimplify.IS1083.pattern_udiv

Source

```
llvm: udiv(X, X)  =>  C1
where
  root = udiv(X, X) : T0
  P0 = properties(root)
  C1 = mlir.constant() : T0 [created]
  P1 = properties(C1)
  X : T0
  T0 : Veir.IntegerType
  P1 ← match T0.val with
  | Veir.Attribute.integerType it => some { value := Veir.LLVMConstantValue.integer { value := 1, type := it } }
  | x => none
```

Veir.InstSimplify.IS1083.pattern_srem

Source

```
llvm: srem(X, X)  =>  mlir.zero()
where
  root = srem(X, X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Veir.InstSimplify.IS1083.pattern_urem

Source

```
llvm: urem(X, X)  =>  mlir.zero()
where
  root = urem(X, X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Veir.InstSimplify.IS1114

Source module

Veir.InstSimplify.IS1114.pattern_sdiv

Source

```
llvm: sdiv((X * Y), Y)  =>  X
where
  a0 = (X * Y) : T0
  P0 = properties(a0)
  root = sdiv(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.nsw
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_sdiv_commute

Source

```
llvm: sdiv((Y * X), Y)  =>  X
where
  a0 = (Y * X) : T0
  P0 = properties(a0)
  root = sdiv(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.nsw
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_udiv

Source

```
llvm: udiv((X * Y), Y)  =>  X
where
  a0 = (X * Y) : T0
  P0 = properties(a0)
  root = udiv(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.nuw
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_udiv_commute

Source

```
llvm: udiv((Y * X), Y)  =>  X
where
  a0 = (Y * X) : T0
  P0 = properties(a0)
  root = udiv(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.nuw
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_srem

Source

```
llvm: srem((X * Y), Y)  =>  mlir.zero()
where
  a0 = (X * Y) : T0
  P0 = properties(a0)
  root = srem(a0, Y) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T0
  require P0.nsw
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1114.pattern_srem_commute

Source

```
llvm: srem((Y * X), Y)  =>  mlir.zero()
where
  a0 = (Y * X) : T0
  P0 = properties(a0)
  root = srem(a0, Y) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T0
  require P0.nsw
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1114.pattern_urem

Source

```
llvm: urem((X * Y), Y)  =>  mlir.zero()
where
  a0 = (X * Y) : T0
  P0 = properties(a0)
  root = urem(a0, Y) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T0
  require P0.nuw
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1114.pattern_urem_commute

Source

```
llvm: urem((Y * X), Y)  =>  mlir.zero()
where
  a0 = (Y * X) : T0
```

```
  P0 = properties(a0)
  root = urem(a0, Y) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T0
  require P0.nuw
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1114.pattern_quotient_sdiv

Source

```
llvm: sdiv((a0 * Y), Y)  =>  a0
where
  a0 = sdiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (a0 * Y) : T0
  P1 = properties(a1)
  root = sdiv(a1, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_quotient_sdiv_commute

Source

```
llvm: sdiv((Y * a0), Y)  =>  a0
where
  a0 = sdiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y * a0) : T0
  P1 = properties(a1)
  root = sdiv(a1, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_quotient_udiv

Source

```
llvm: udiv((a0 * Y), Y)  =>  a0
where
  a0 = udiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (a0 * Y) : T0
  P1 = properties(a1)
  root = udiv(a1, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_quotient_udiv_commute

Source

```
llvm: udiv((Y * a0), Y)  =>  a0
where
  a0 = udiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y * a0) : T0
  P1 = properties(a1)
  root = udiv(a1, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1114.pattern_quotient_srem

Source

```
llvm: srem((sdiv(X, Y) * Y), Y)  =>  mlir.zero()
where
  a0 = sdiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (a0 * Y) : T0
  P1 = properties(a1)
  root = srem(a1, Y) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P3 = PUnit.unit
```

Veir.InstSimplify.IS1114.pattern_quotient_srem_commute

Source

```
llvm: srem((Y * sdiv(X, Y)), Y)  =>  mlir.zero()
where
  a0 = sdiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y * a0) : T0
  P1 = properties(a1)
  root = srem(a1, Y) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P3 = PUnit.unit
```

Veir.InstSimplify.IS1114.pattern_quotient_urem

Source

```
llvm: urem((udiv(X, Y) * Y), Y)  =>  mlir.zero()
where
  a0 = udiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (a0 * Y) : T0
  P1 = properties(a1)
  root = urem(a1, Y) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P3 = PUnit.unit
```

Veir.InstSimplify.IS1114.pattern_quotient_urem_commute

Source

```
llvm: urem((Y * udiv(X, Y)), Y)  =>  mlir.zero()
where
  a0 = udiv(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y * a0) : T0
  P1 = properties(a1)
  root = urem(a1, Y) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P3 = PUnit.unit
```

Veir.InstSimplify.IS1168

Source module

Veir.InstSimplify.IS1168.pattern_udiv

Source

Parameters: `n : Int`

```
llvm: udiv((X * C0), C0)  =>  X
where
  C0 = n : T0
  P0 = properties(C0)
  a1 = (X * C0) : T0
  P1 = properties(a1)
  root = udiv(a1, C0) : T0
  P2 = properties(root)
  X : T0
  require P2.exact
  require P1.nsw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it =>
    BitVec.ofInt it.bitwidth n != 0 && !decide ((BitVec.ofInt it.bitwidth n).toNat | 2 ^ it.bitwidth)
  | x => false
```

Veir.InstSimplify.IS1168.pattern_sdiv

Source

Parameters: `n : Int`

```
llvm: sdiv((X * C0), C0)  =>  X
where
  C0 = n : T0
  P0 = properties(C0)
  a1 = (X * C0) : T0
  P1 = properties(a1)
  root = sdiv(a1, C0) : T0
  P2 = properties(root)
  X : T0
  require P2.exact
  require P1.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it =>
    BitVec.ofInt it.bitwidth n != 0 && !decide ((BitVec.ofInt it.bitwidth n).toNat | 2 ^ it.bitwidth)
  | x => false
```

Veir.InstSimplify.IS1189

Source module

Veir.InstSimplify.IS1189.pattern_srem

Source

```
llvm: srem(shl(X, Y), X)  =>  mlir.zero()
where
  a0 = shl(X, Y) : T0
  P0 = properties(a0)
  root = srem(a0, X) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T0
  require P0.nsw
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1189.pattern_urem

Source

```
llvm: urem(shl(X, Y), X)  =>  mlir.zero()
where
  a0 = shl(X, Y) : T0
  P0 = properties(a0)
  root = urem(a0, X) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T0
  require P0.nuw
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1204

Source module

Veir.InstSimplify.IS1204.pattern_srem

Source

Parameters: `n : Int; m : Int`

```
llvm: srem((X * n), m)  =>  mlir.zero()
where
  C0 = n : T0
  P0 = properties(C0)
  C1 = m : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  root = srem(a2, C1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  require P2.nsw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require Veir.InstSimplify.bitConstantGuard T0 P1 m
  require match T0.val with
  | Veir.Attribute.integerType it => (BitVec.ofInt it.bitwidth n).srem (BitVec.ofInt it.bitwidth m) == 0
  | x => false
  P4 = PUnit.unit
```

Veir.InstSimplify.IS1204.pattern_urem

Source

Parameters: `n : Int; m : Int`

```
llvm: urem((X * n), m)  =>  mlir.zero()
where
  C0 = n : T0
  P0 = properties(C0)
  C1 = m : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  root = urem(a2, C1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  require P2.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require Veir.InstSimplify.bitConstantGuard T0 P1 m
  require match T0.val with
  | Veir.Attribute.integerType it => (BitVec.ofInt it.bitwidth n).umod (BitVec.ofInt it.bitwidth m) == 0
  | x => false
  P4 = PUnit.unit
```

Veir.InstSimplify.IS1246

Source module

Veir.InstSimplify.IS1246.pattern

Source

```
llvm: srem(Y, sext(X))  =>  mlir.zero()
where
  a0 = sext(X) : T1
  P0 = properties(a0)
  root = srem(Y, a0) : T1
  P1 = properties(root)
  new2 = mlir.zero() : T1 [created]
  P2 = properties(new2)
  X : T0
  Y : T1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val with
  | Veir.Attribute.integerType it => it.bitwidth == 1
  | x => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1310

Source module

Veir.InstSimplify.IS1310.pattern_shl_left

Source

```
llvm: shl(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = shl(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1310.pattern_lshr_left

Source

```
llvm: lshr(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = lshr(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1310.pattern_ashr_left

Source

```
llvm: ashr(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = ashr(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1314

Source module

Veir.InstSimplify.IS1314.pattern_shl_left

Source

```
llvm: shl(C0, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = shl(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1314.pattern_lshr_left

Source

```
llvm: lshr(C0, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = lshr(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1314.pattern_ashr_left

Source

```
llvm: ashr(C0, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = ashr(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1322

Source module

Veir.InstSimplify.IS1322.pattern_shl_right

Source

```
llvm: shl(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = shl(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1322.pattern_lshr_right

Source

```
llvm: lshr(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = lshr(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1322.pattern_ashr_right

Source

```
llvm: ashr(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = ashr(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS1322.pattern_sext_shl

Source

```
llvm: shl(Y, sext(X))  =>  Y
where
  a0 = sext(X) : T1
  P0 = properties(a0)
  root = shl(Y, a0) : T1
  P1 = properties(root)
  X : T0
  Y : T1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val with
  | Veir.Attribute.integerType it => it.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1322.pattern_sext_lshr

Source

```
llvm: lshr(Y, sext(X))  =>  Y
where
  a0 = sext(X) : T1
  P0 = properties(a0)
  root = lshr(Y, a0) : T1
  P1 = properties(root)
  X : T0
  Y : T1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val with
  | Veir.Attribute.integerType it => it.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1322.pattern_sext_ashr

Source

```
llvm: ashr(Y, sext(X))  =>  Y
where
  a0 = sext(X) : T1
  P0 = properties(a0)
  root = ashr(Y, a0) : T1
  P1 = properties(root)
  X : T0
  Y : T1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val with
  | Veir.Attribute.integerType it => it.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1326

Source module

Veir.InstSimplify.IS1326.pattern_shl

Source

Parameters: `n : Int`

```
llvm: shl(X, n)  =>  mlir.poison()
where
  C0 = n : T0
  P0 = properties(C0)
  root = shl(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it => decide (it.bitwidth ≤ (BitVec.ofInt it.bitwidth n).toNat)
  | x => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1326.pattern_lshr

Source

Parameters: `n : Int`

```
llvm: lshr(X, n)  =>  mlir.poison()
where
  C0 = n : T0
  P0 = properties(C0)
  root = lshr(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it => decide (it.bitwidth ≤ (BitVec.ofInt it.bitwidth n).toNat)
  | x => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1326.pattern_ashr

Source

Parameters: `n : Int`

```
llvm: ashr(X, n)  =>  mlir.poison()
where
  C0 = n : T0
  P0 = properties(C0)
  root = ashr(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it => decide (it.bitwidth ≤ (BitVec.ofInt it.bitwidth n).toNat)
  | x => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1381

Source module

Veir.InstSimplify.IS1381.pattern_lshr

Source

```
llvm: lshr(X, X)  =>  mlir.zero()
where
  root = lshr(X, X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
```

```
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Veir.InstSimplify.IS1381.pattern_ashr

Source

```
llvm: ashr(X, X)  =>  mlir.zero()
where
  root = ashr(X, X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Veir.InstSimplify.IS1417

Source module

Veir.InstSimplify.IS1417.pattern_lshr

Source

```
llvm: shl(lshr(X, Y), Y)  =>  X
where
  a0 = lshr(X, Y) : T0
  P0 = properties(a0)
  root = shl(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.exact
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1417.pattern_ashr

Source

```
llvm: shl(ashr(X, Y), Y)  =>  X
where
  a0 = ashr(X, Y) : T0
  P0 = properties(a0)
  root = shl(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.exact
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1421

Source module

Veir.InstSimplify.IS1421.pattern

Source

Parameters: `n : Int`

```
llvm: shl(C0, X)  =>  C0
where
  C0 = n : T0
  P0 = properties(C0)
  root = shl(C0, X) : T0
  P1 = properties(root)
  X : T0
  require P1.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it => (BitVec.ofInt it.bitwidth n).msb
  | x => false
```

Veir.InstSimplify.IS1430

Source module

Veir.InstSimplify.IS1430.pattern

Source

Parameters: `n : Int`

```
llvm: shl(X, n)  =>  mlir.zero()
where
  C0 = n : T0
  P0 = properties(C0)
  root = shl(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  require P1.nsw && P1.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType it =>
    decide (1 < it.bitwidth) && BitVec.ofInt it.bitwidth n == BitVec.ofNat it.bitwidth (it.bitwidth - 1)
  | x => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS1451

Source module

Veir.InstSimplify.IS1451.pattern

Source

```
llvm: lshr(shl(X, Y), Y)  =>  X
where
  a0 = shl(X, Y) : T0
  P0 = properties(a0)
  root = lshr(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.nuw
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1490

Source module

Veir.InstSimplify.IS1490.pattern

Source

```
llvm: ashr(C0, X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = ashr(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS1490.pattern_nested

Source

```
llvm: ashr(shl(C0, X), X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = shl(C0, X) : T0
  P1 = properties(a1)
  root = ashr(a1, X) : T0
```

```
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS1495

Source module

Veir.InstSimplify.IS1495.pattern

Source

```
llvm: ashr(shl(X, Y), Y)  =>  X
where
  a0 = shl(X, Y) : T0
  P0 = properties(a0)
  root = ashr(a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require P0.nsw
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS1534

Source module

Veir.InstSimplify.IS1534.pattern_or

Source

Variant 1: `reversed = false; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) | icmp(X, Y))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `reversed = false; flipped = false; commute = true`

```
llvm: (icmp(X, Y) | icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
```

```
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 3: `reversed = false; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) | icmp(Y, X))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 4: `reversed = false; flipped = true; commute = true`

```
llvm: (icmp(Y, X) | icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
```

```
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 5: `reversed = true; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) | icmp(X, Y))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 6: `reversed = true; flipped = false; commute = true`

```
llvm: (icmp(X, Y) | icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
```

```
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 7: `reversed = true; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) | icmp(Y, X))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else
Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 8: `reversed = true; flipped = true; commute = true`

```
llvm: (icmp(Y, X) | icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else
Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1539

Source module

Veir.InstSimplify.IS1539.pattern_and

Source

Variant 1: `reversed = false; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) & icmp(X, Y))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `reversed = false; flipped = false; commute = true`

```
llvm: (icmp(X, Y) & icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 3: `reversed = false; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) & icmp(Y, X))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
```

```
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 4: `reversed = false; flipped = true; commute = true`

```
llvm: (icmp(Y, X) & icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 5: `reversed = true; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) & icmp(X, Y))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
```

```
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 6: `reversed = true; flipped = false; commute = true`

```
llvm: (icmp(X, Y) & icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 7: `reversed = true; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) & icmp(Y, X))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else
Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
```

```
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 8: `reversed = true; flipped = true; commute = true`

```
llvm: (icmp(Y, X) & icmp((X - Y), 0))  =>  C5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else
Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1545

Source module

Veir.InstSimplify.IS1545.pattern_and

Source

Variant 1: `reversed = false; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) & a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `reversed = false; flipped = false; commute = true`

```
llvm: (a3 & icmp((X - Y), 0))  =>  a3
where
```

```
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `reversed = false; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) & a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `reversed = false; flipped = true; commute = true`

```
llvm: (a3 & icmp((X - Y), 0))  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

```
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 5: `reversed = true; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) & a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 6: `reversed = true; flipped = false; commute = true`

```
llvm: (a3 & icmp((X - Y), 0))  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 7: `reversed = true; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) & a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else
Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 8: `reversed = true; flipped = true; commute = true`

```
llvm: (a3 & icmp((X - Y), 0))  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else
Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1545.pattern_or

Source

Variant 1: `reversed = false; flipped = false; commute = false`

```
llvm: (a2 | icmp(X, Y))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `reversed = false; flipped = false; commute = true`

```
llvm: (icmp(X, Y) | a2)  =>  a2
where
```

```
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `reversed = false; flipped = true; commute = false`

```
llvm: (a2 | icmp(Y, X))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `reversed = false; flipped = true; commute = true`

```
llvm: (icmp(Y, X) | a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

```
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 5: `reversed = true; flipped = false; commute = false`

```
llvm: (a2 | icmp(X, Y))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 6: `reversed = true; flipped = false; commute = true`

```
llvm: (icmp(X, Y) | a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 7: `reversed = true; flipped = true; commute = false`

```
llvm: (a2 | icmp(Y, X))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else
Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 8: `reversed = true; flipped = true; commute = true`

```
llvm: (icmp(Y, X) | a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.ugt else
Veir.Data.LLVM.IntPred.ult)
    else bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1551

Source module

Veir.InstSimplify.IS1551.pattern_and

Source

Variant 1: `reversed = false; flipped = false; commute = false`

```
llvm: (a2 & icmp(X, Y))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `reversed = false; flipped = false; commute = true`

```
llvm: (icmp(X, Y) & a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `reversed = false; flipped = true; commute = false`

```
llvm: (a2 & icmp(Y, X))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `reversed = false; flipped = true; commute = true`

```
llvm: (icmp(Y, X) & a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
```

```
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 5: `reversed = true; flipped = false; commute = false`

```
llvm: (a2 & icmp(X, Y))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 6: `reversed = true; flipped = false; commute = true`

```
llvm: (icmp(X, Y) & a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 7: `reversed = true; flipped = true; commute = false`

```
llvm: (a2 & icmp(Y, X))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
```

```
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else
Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 8: `reversed = true; flipped = true; commute = true`

```
llvm: (icmp(Y, X) & a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 & a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else
Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1551.pattern_or

Source

Variant 1: `reversed = false; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) | a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `reversed = false; flipped = false; commute = true`

```
llvm: (a3 | icmp((X - Y), 0))  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `reversed = false; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) | a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `reversed = false; flipped = true; commute = true`

```
llvm: (a3 | icmp((X - Y), 0))  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then
      Veir.InstSimplify.swapCmpPred (bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif false then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 5: `reversed = true; flipped = false; commute = false`

```
llvm: (icmp((X - Y), 0) | a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 6: `reversed = true; flipped = false; commute = true`

```
llvm: (a3 | icmp((X - Y), 0))  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, Y) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif false then
      Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 7: `reversed = true; flipped = true; commute = false`

```
llvm: (icmp((X - Y), 0) | a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
```

```
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else
Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 8: `reversed = true; flipped = true; commute = true`

```
llvm: (a3 | icmp((X - Y), 0))  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  a2 = icmp(a1, C0) : T1
  P2 = properties(a2)
  a3 = icmp(Y, X) : T1
  P3 = properties(a3)
  root = (a3 | a2) : T1
  P4 = properties(root)
  X : T0
  Y : T0
  require P3.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (bif true then Veir.Data.LLVM.IntPred.uge else
Veir.Data.LLVM.IntPred.ule)
    else bif true then Veir.Data.LLVM.IntPred.uge else Veir.Data.LLVM.IntPred.ule
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1598

Source module

Veir.InstSimplify.IS1598.pattern_and

Source

Variant 1: `flipped = false; commute = false`

```
llvm: (a1 & icmp(Y, 0))  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 & a2) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `flipped = false; commute = true`

```
llvm: (icmp(Y, 0) & a1)  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 & a1) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `flipped = true; commute = false`

```
llvm: (a1 & icmp(Y, 0))  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 & a2) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `flipped = true; commute = true`

```
llvm: (icmp(Y, 0) & a1)  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 & a1) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1598.pattern_or

Source

Variant 1: `flipped = false; commute = false`

```
llvm: (icmp(X, Y) | a2)  =>  a2
where
  C0 = 0 : T0
```

```
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 | a2) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `flipped = false; commute = true`

```
llvm: (a2 | icmp(X, Y))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 | a1) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `flipped = true; commute = false`

```
llvm: (icmp(Y, X) | a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 | a2) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `flipped = true; commute = true`

```
llvm: (a2 | icmp(Y, X))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 | a1) : T1
  P3 = properties(root)
```

```
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1603

Source module

Veir.InstSimplify.IS1603.pattern_and

Source

Variant 1: `flipped = false; commute = false`

```
llvm: (icmp(X, Y) & a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 & a2) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `flipped = false; commute = true`

```
llvm: (a2 & icmp(X, Y))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 & a1) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `flipped = true; commute = false`

```
llvm: (icmp(Y, X) & a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 & a2) : T1
```

```
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `flipped = true; commute = true`

```
llvm: (a2 & icmp(Y, X))  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 & a1) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1603.pattern_or

Source

Variant 1: `flipped = false; commute = false`

```
llvm: (a1 | icmp(Y, 0))  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 | a2) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `flipped = false; commute = true`

```
llvm: (icmp(Y, 0) | a1)  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 | a1) : T1
  P3 = properties(root)
  X : T0
  Y : T0
```

```
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `flipped = true; commute = false`

```
llvm: (a1 | icmp(Y, 0))  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 | a2) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `flipped = true; commute = true`

```
llvm: (icmp(Y, 0) | a1)  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 | a1) : T1
  P3 = properties(root)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS1608

Source module

Veir.InstSimplify.IS1608.pattern_and

Source

Variant 1: `flipped = false; commute = false`

```
llvm: (icmp(X, Y) & icmp(Y, 0))  =>  C4
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 & a2) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
```

```
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `flipped = false; commute = true`

```
llvm: (icmp(Y, 0) & icmp(X, Y))  =>  C4
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 & a1) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 3: `flipped = true; commute = false`

```
llvm: (icmp(Y, X) & icmp(Y, 0))  =>  C4
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 & a2) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 4: `flipped = true; commute = true`

```
llvm: (icmp(Y, 0) & icmp(Y, X))  =>  C4
where
  C0 = 0 : T0
```

```
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 & a1) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ugt else Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1613

Source module

Veir.InstSimplify.IS1613.pattern_or

Source

Variant 1: `flipped = false; commute = false`

```
llvm: (icmp(X, Y) | icmp(Y, 0))  =>  C4
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 | a2) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `flipped = false; commute = true`

```
llvm: (icmp(Y, 0) | icmp(X, Y))  =>  C4
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 | a1) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
```

```
  require P1.predicate == bif false then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 3: `flipped = true; commute = false`

```
llvm: (icmp(Y, X) | icmp(Y, 0))  =>  C4
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a1 | a2) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 4: `flipped = true; commute = true`

```
llvm: (icmp(Y, 0) | icmp(Y, X))  =>  C4
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  a2 = icmp(Y, C0) : T1
  P2 = properties(a2)
  root = (a2 | a1) : T1
  P3 = properties(root)
  C4 = mlir.constant() : T1 [created]
  P4 = properties(C4)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require P1.predicate == bif true then Veir.Data.LLVM.IntPred.ule else Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T1.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P4 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1638

Source module

Veir.InstSimplify.IS1638.pattern

Source

Parameters: `p0 : Veir.Data.LLVM.IntPred; p1 : Veir.Data.LLVM.IntPred; k0 : Int; k1 : Int`

```
llvm: (icmp(X, k0) & icmp(X, k1))  =>  C5
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = icmp(X, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, C1) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  require P3.predicate == p1
  require P2.predicate == p0
  require Veir.InstSimplify.intConstantGuard P1 k1
  require Veir.InstSimplify.intConstantGuard P0 k0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 && Veir.InstSimplify.IS1638.contains t.bitwidth p0 p1 k0 k1
  | x, x_1 => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1643

Source module

Veir.InstSimplify.IS1643.pattern

Source

Parameters: `p0 : Veir.Data.LLVM.IntPred; p1 : Veir.Data.LLVM.IntPred; k0 : Int; k1 : Int`

```
llvm: (icmp(X, k0) | icmp(X, k1))  =>  C5
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = icmp(X, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, C1) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  C5 = mlir.constant() : T1 [created]
  P5 = properties(C5)
  X : T0
  require P3.predicate == p1
  require P2.predicate == p0
  require Veir.InstSimplify.intConstantGuard P1 k1
  require Veir.InstSimplify.intConstantGuard P0 k0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 && Veir.InstSimplify.IS1643.contains t.bitwidth p0 p1 k0 k1
  | x, x_1 => false
  P5 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1651

Source module

Veir.InstSimplify.IS1651.pattern_and

Source

Parameters: `p0 : Veir.Data.LLVM.IntPred; p1 : Veir.Data.LLVM.IntPred; k0 : Int; k1 : Int`

```
llvm: (icmp(X, k0) & a3)  =>  a3
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = icmp(X, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, C1) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
  X : T0
  require P3.predicate == p1
  require P2.predicate == p0
  require Veir.InstSimplify.intConstantGuard P1 k1
  require Veir.InstSimplify.intConstantGuard P0 k0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 && Veir.InstSimplify.IS1651.contains t.bitwidth p0 p1 k0 k1
  | x, x_1 => false
```

Veir.InstSimplify.IS1651.pattern_or

Source

Parameters: `p0 : Veir.Data.LLVM.IntPred; p1 : Veir.Data.LLVM.IntPred; k0 : Int; k1 : Int`

```
llvm: (a2 | icmp(X, k1))  =>  a2
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = icmp(X, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, C1) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  require P3.predicate == p1
  require P2.predicate == p0
  require Veir.InstSimplify.intConstantGuard P1 k1
  require Veir.InstSimplify.intConstantGuard P0 k0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 && Veir.InstSimplify.IS1651.contains t.bitwidth p0 p1 k0 k1
  | x, x_1 => false
```

Veir.InstSimplify.IS1653

Source module

Veir.InstSimplify.IS1653.pattern_and

Source

Parameters: `p0 : Veir.Data.LLVM.IntPred; p1 : Veir.Data.LLVM.IntPred; k0 : Int; k1 : Int`

```
llvm: (a2 & icmp(X, k1))  =>  a2
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = icmp(X, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, C1) : T1
  P3 = properties(a3)
  root = (a2 & a3) : T1
  P4 = properties(root)
```

```
  X : T0
  require P3.predicate == p1
  require P2.predicate == p0
  require Veir.InstSimplify.intConstantGuard P1 k1
  require Veir.InstSimplify.intConstantGuard P0 k0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 && Veir.InstSimplify.IS1653.contains t.bitwidth p0 p1 k0 k1
  | x, x_1 => false
```

Veir.InstSimplify.IS1653.pattern_or

Source

Parameters: `p0 : Veir.Data.LLVM.IntPred; p1 : Veir.Data.LLVM.IntPred; k0 : Int; k1 : Int`

```
llvm: (icmp(X, k0) | a3)  =>  a3
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = icmp(X, C0) : T1
  P2 = properties(a2)
  a3 = icmp(X, C1) : T1
  P3 = properties(a3)
  root = (a2 | a3) : T1
  P4 = properties(root)
  X : T0
  require P3.predicate == p1
  require P2.predicate == p0
  require Veir.InstSimplify.intConstantGuard P1 k1
  require Veir.InstSimplify.intConstantGuard P0 k0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 && Veir.InstSimplify.IS1653.contains t.bitwidth p0 p1 k0 k1
  | x, x_1 => false
```

Veir.InstSimplify.IS1682

Source module

Veir.InstSimplify.IS1682.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) & icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require P3.predicate == Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
```

```
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) & icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require P3.predicate == Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1684

Source module

Veir.InstSimplify.IS1684.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) & icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require P3.predicate == Veir.Data.LLVM.IntPred.slt
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

```
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) & icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require P3.predicate == Veir.Data.LLVM.IntPred.slt
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1688

Source module

Veir.InstSimplify.IS1688.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) & icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
```

```
  require P3.predicate == Veir.Data.LLVM.IntPred.ule
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) & icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require P3.predicate == Veir.Data.LLVM.IntPred.ule
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1690

Source module

Veir.InstSimplify.IS1690.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) & icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
```

```
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require P3.predicate == Veir.Data.LLVM.IntPred.sle
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) & icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require P3.predicate == Veir.Data.LLVM.IntPred.sle
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1696

Source module

Veir.InstSimplify.IS1696.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) & icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
```

```
  root = (a3 & a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require P3.predicate == Veir.Data.LLVM.IntPred.ult
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) & icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require P3.predicate == Veir.Data.LLVM.IntPred.ult
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1699

Source module

Veir.InstSimplify.IS1699.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) & icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
```

```
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require P3.predicate == Veir.Data.LLVM.IntPred.ule
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) & icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require P3.predicate == Veir.Data.LLVM.IntPred.ule
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS1717

Source module

Veir.InstSimplify.IS1717

Source

Variant 1: `c : Int; sw0 = false; sw1 = false; swroot = false`

```
llvm: (icmp(intr.ctpop(X), c) | a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
```

```
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 2: `c : Int; sw0 = false; sw1 = false; swroot = true`

```
llvm: (a4 | icmp(intr.ctpop(X), c))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 3: `c : Int; sw0 = false; sw1 = true; swroot = false`

```
llvm: (icmp(intr.ctpop(X), c) | a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 4: `c : Int; sw0 = false; sw1 = true; swroot = true`

```
llvm: (a4 | icmp(intr.ctpop(X), c))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 5: `c : Int; sw0 = true; sw1 = false; swroot = false`

```
llvm: (icmp(c, intr.ctpop(X)) | a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 6: `c : Int; sw0 = true; sw1 = false; swroot = true`

```
llvm: (a4 | icmp(c, intr.ctpop(X)))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 7: `c : Int; sw0 = true; sw1 = true; swroot = false`

```
llvm: (icmp(c, intr.ctpop(X)) | a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 8: `c : Int; sw0 = true; sw1 = true; swroot = true`

```
llvm: (a4 | icmp(c, intr.ctpop(X)))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Veir.InstSimplify.IS1720

Source module

Veir.InstSimplify.IS1720

Source

Variant 1: `c : Int; sw0 = false; sw1 = false; swroot = false`

```
llvm: (icmp(intr.ctpop(X), c) & a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
```

```
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 2: `c : Int; sw0 = false; sw1 = false; swroot = true`

```
llvm: (a4 & icmp(intr.ctpop(X), c))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 3: `c : Int; sw0 = false; sw1 = true; swroot = false`

```
llvm: (icmp(intr.ctpop(X), c) & a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 4: `c : Int; sw0 = false; sw1 = true; swroot = true`

```
llvm: (a4 & icmp(intr.ctpop(X), c))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
```

```
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a0, C1) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 5: `c : Int; sw0 = true; sw1 = false; swroot = false`

```
llvm: (icmp(c, intr.ctpop(X)) & a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 6: `c : Int; sw0 = true; sw1 = false; swroot = true`

```
llvm: (a4 & icmp(c, intr.ctpop(X)))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(X, C2) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 7: `c : Int; sw0 = true; sw1 = true; swroot = false`

```
llvm: (icmp(c, intr.ctpop(X)) & a4)  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a3 & a4) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Variant 8: `c : Int; sw0 = true; sw1 = true; swroot = true`

```
llvm: (a4 & icmp(c, intr.ctpop(X)))  =>  a4
where
  a0 = intr.ctpop(X) : T0
  P0 = properties(a0)
  C1 = c : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C1, a0) : T1
  P3 = properties(a3)
  a4 = icmp(C2, X) : T1
  P4 = properties(a4)
  root = (a4 & a3) : T1
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P1 c
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
  require match T0.val with
  | Veir.Attribute.integerType t => !BitVec.ofInt t.bitwidth c == 0
  | x => false
```

Veir.InstSimplify.IS1772

Source module

Veir.InstSimplify.IS1772.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) | icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
```

```
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) | icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1774

Source module

Veir.InstSimplify.IS1774.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) | icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
```

```
  root = (a3 | a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.sge
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) | icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.sge
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1778

Source module

Veir.InstSimplify.IS1778.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) | icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
```

```
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.ugt
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) | icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.ugt
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1780

Source module

Veir.InstSimplify.IS1780.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) | icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
```

```
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.sgt
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) | icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require P3.predicate == Veir.Data.LLVM.IntPred.sgt
  require P2.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (0 < (BitVec.ofInt t.bitwidth k0).toInt) &&
        (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat == 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1786

Source module

Veir.InstSimplify.IS1786.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) | icmp(X, C0))  =>  C6
where
```

```
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require P3.predicate == Veir.Data.LLVM.IntPred.uge
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) | icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require P3.predicate == Veir.Data.LLVM.IntPred.uge
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 2)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS1789

Source module

Veir.InstSimplify.IS1789.pattern

Source

Variant 1: `k0 : Int; k1 : Int; commute = false`

```
llvm: (icmp((X + C0), k1) | icmp(X, C0))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a3 | a4) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require P3.predicate == Veir.Data.LLVM.IntPred.ugt
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `k0 : Int; k1 : Int; commute = true`

```
llvm: (icmp(X, C0) | icmp((X + C0), k1))  =>  C6
where
  C0 = k0 : T0
  P0 = properties(C0)
  C1 = k1 : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = icmp(a2, C1) : T1
  P3 = properties(a3)
  a4 = icmp(X, C0) : T1
  P4 = properties(a4)
  root = (a4 | a3) : T1
  P5 = properties(root)
  C6 = mlir.constant() : T1 [created]
  P6 = properties(C6)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require P3.predicate == Veir.Data.LLVM.IntPred.ugt
  require P2.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 k0
  require Veir.InstSimplify.bitConstantGuard T0 P1 k1
  require match T0.val, T1.val with
  | Veir.Attribute.integerType t, Veir.Attribute.integerType b =>
    b.bitwidth == 1 &&
      (decide (BitVec.ofInt t.bitwidth k0 ≠ 0) && (BitVec.ofInt t.bitwidth k1 - BitVec.ofInt t.bitwidth k0).toNat
== 1)
  | x, x_1 => false
  P6 ← match T1.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS2011

Source module

Veir.InstSimplify.IS2011.pattern_and

Source

Variant 1: `c : Int; commute = false`

```
llvm: ((X + c) & ((-c - 1) - X))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = (C1 - X) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `c : Int; commute = true`

```
llvm: (((-c - 1) - X) & (X + c))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = (C1 - X) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS2011.pattern_or

Source

Variant 1: `c : Int; commute = false`

```
llvm: ((X + c) | ((-c - 1) - X))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = (C1 - X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt (-1)
```

```
t) }
  | x => none
```

Variant 2: `c : Int; commute = true`

```
llvm: (((-c - 1) - X) | (X + c))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = (C1 - X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt (-1)
t) }
  | x => none
```

Veir.InstSimplify.IS2011.pattern_xor

Source

Variant 1: `c : Int; commute = false`

```
llvm: ((X + c) ^ ((-c - 1) - X))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = (C1 - X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt (-1)
t) }
  | x => none
```

Variant 2: `c : Int; commute = true`

```
llvm: (((-c - 1) - X) ^ (X + c))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X + C0) : T0
  P2 = properties(a2)
  a3 = (C1 - X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
```

```
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt (-1)
t) }
  | x => none
```

Veir.InstSimplify.IS2024

Source module

Veir.InstSimplify.IS2024.pattern

Source

Variant 1: `sw0 = false; sw1 = false`

```
llvm: ((X ^ -1) & X)  =>  mlir.zero()
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (a1 & X) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  P3 = PUnit.unit
```

Variant 2: `sw0 = false; sw1 = true`

```
llvm: (X & (X ^ -1))  =>  mlir.zero()
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (X & a1) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  P3 = PUnit.unit
```

Variant 3: `sw0 = true; sw1 = false`

```
llvm: ((-1 ^ X) & X)  =>  mlir.zero()
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (a1 & X) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  P3 = PUnit.unit
```

Variant 4: `sw0 = true; sw1 = true`

```
llvm: (X & (-1 ^ X))  =>  mlir.zero()
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (X & a1) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
```

```
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  P3 = PUnit.unit
```

Veir.InstSimplify.IS2028

Source module

Veir.InstSimplify.IS2028.pattern

Source

Variant 1: `sw0 = false; sw1 = false`

```
llvm: ((X | Y) & X)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  root = (a0 & X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 2: `sw0 = false; sw1 = true`

```
llvm: (X & (X | Y))  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  root = (X & a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 3: `sw0 = true; sw1 = false`

```
llvm: ((Y | X) & X)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  root = (a0 & X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 4: `sw0 = true; sw1 = true`

```
llvm: (X & (Y | X))  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  root = (X & a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2034

Source module

Veir.InstSimplify.IS2034.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((X | (Y ^ -1)) & (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X | Y) & (X | (Y ^ -1)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((X | (Y ^ -1)) & (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y | X) & (X | (Y ^ -1)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (((Y ^ -1) | X) & (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X | Y) & ((Y ^ -1) | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (((Y ^ -1) | X) & (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y | X) & ((Y ^ -1) | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((X | (-1 ^ Y)) & (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X | Y) & (X | (-1 ^ Y)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((X | (-1 ^ Y)) & (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y | X) & (X | (-1 ^ Y)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (((-1 ^ Y) | X) & (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X | Y) & ((-1 ^ Y) | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (((-1 ^ Y) | X) & (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y | X) & ((-1 ^ Y) | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 & a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2078

Source module

Veir.InstSimplify.IS2078.pattern

Source

```
llvm: (X & a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (X & a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2086

Source module

Veir.InstSimplify.IS2086.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X & X)  =>  X
where
  root = (X & X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Variant 2: `sw0 = true`

```
llvm: (X & X)  =>  X
where
  root = (X & X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2090

Source module

Veir.InstSimplify.IS2090.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X & 0)  =>  mlir.zero()
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (X & C0) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  P2 = PUnit.unit
```

Variant 2: `sw0 = true`

```
llvm: (0 & X)  =>  mlir.zero()
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (C0 & X) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  P2 = PUnit.unit
```

Veir.InstSimplify.IS2094

Source module

Veir.InstSimplify.IS2094.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X & -1)  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = (X & C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = true`

```
llvm: (-1 & X)  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = (C0 & X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2113

Source module

Veir.InstSimplify.IS2113.pattern

Source

Parameters: `amount : Int; mask : Int`

```
llvm: (a2 & mask)  =>  a2
where
  C0 = amount : T0
  P0 = properties(C0)
  C1 = mask : T0
  P1 = properties(C1)
  a2 = shl(X, C0) : T0
  P2 = properties(a2)
  root = (a2 & C1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 amount
  require Veir.InstSimplify.bitConstantGuard T0 P1 mask
  require match T0.val with
  | Veir.Attribute.integerType t => ~~~BitVec.ofInt t.bitwidth mask >>> (BitVec.ofInt t.bitwidth amount).toNat ==
0
  | x => false
```

Veir.InstSimplify.IS2119

Source module

Veir.InstSimplify.IS2119.pattern

Source

Parameters: `amount : Int; mask : Int`

```
llvm: (a2 & mask)  =>  a2
where
  C0 = amount : T0
  P0 = properties(C0)
  C1 = mask : T0
  P1 = properties(C1)
  a2 = lshr(X, C0) : T0
```

```
  P2 = properties(a2)
  root = (a2 & C1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 amount
  require Veir.InstSimplify.bitConstantGuard T0 P1 mask
  require match T0.val with
  | Veir.Attribute.integerType t => ~~~BitVec.ofInt t.bitwidth mask <<< (BitVec.ofInt t.bitwidth amount).toNat ==
0
  | x => false
```

Veir.InstSimplify.IS2143

Source module

Veir.InstSimplify.IS2143.pattern

Source

Variant 1: `commute = false`

```
llvm: (a0 & sext(X))  =>  a0
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = (a0 & a1) : T1
  P2 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

Variant 2: `commute = true`

```
llvm: (sext(X) & a0)  =>  a0
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = (a1 & a0) : T1
  P2 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2145

Source module

Veir.InstSimplify.IS2145.pattern

Source

Builder families: `Veir.InstSimplify.IS2143.pattern`.

```
llvm: (sext(X) & a0)  =>  a0
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = (a1 & a0) : T1
  P2 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2167

Source module

Veir.InstSimplify.IS2167.pattern

Source

Variant 1: `commute = false`

```
llvm: (X & a1)  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(X, Y, C0) : T0
  P1 = properties(a1)
  root = (X & a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `commute = true`

```
llvm: (a1 & X)  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(X, Y, C0) : T0
  P1 = properties(a1)
  root = (a1 & X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS2169

Source module

Veir.InstSimplify.IS2169.pattern

Source

Builder families: `Veir.InstSimplify.IS2167.pattern`.

```
llvm: (a1 & X)  =>  a1
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(X, Y, C0) : T0
  P1 = properties(a1)
  root = (a1 & X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS2226

Source module

Veir.InstSimplify.IS2226.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((a0 ^ X) & (a0 ^ Y))  =>  mlir.zero()
where
```

```
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((a0 ^ Y) & (a0 ^ X))  =>  mlir.zero()
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((a0 ^ X) & (Y ^ a0))  =>  mlir.zero()
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y ^ a0) & (a0 ^ X))  =>  mlir.zero()
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((X ^ a0) & (a0 ^ Y))  =>  mlir.zero()
where
```

```
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((a0 ^ Y) & (X ^ a0))  =>  mlir.zero()
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((X ^ a0) & (Y ^ a0))  =>  mlir.zero()
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y ^ a0) & (X ^ a0))  =>  mlir.zero()
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((a0 ^ X) & (a0 ^ Y))  =>  mlir.zero()
where
```

```
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((a0 ^ Y) & (a0 ^ X))  =>  mlir.zero()
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((a0 ^ X) & (Y ^ a0))  =>  mlir.zero()
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y ^ a0) & (a0 ^ X))  =>  mlir.zero()
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (a0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((X ^ a0) & (a0 ^ Y))  =>  mlir.zero()
where
```

```
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((a0 ^ Y) & (X ^ a0))  =>  mlir.zero()
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (a0 ^ Y) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((X ^ a0) & (Y ^ a0))  =>  mlir.zero()
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a1 & a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y ^ a0) & (X ^ a0))  =>  mlir.zero()
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  a1 = (X ^ a0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a0) : T0
  P2 = properties(a2)
  root = (a2 & a1) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  P4 = PUnit.unit
```

Veir.InstSimplify.IS2233

Source module

Veir.InstSimplify.IS2233.pattern

Source

Parameters: `c : Int`

```
llvm: ((X ^ c) & (X ^ (-c - 1)))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = (X ^ C1) : T0
  P3 = properties(a3)
  root = (a2 & a3) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS2271

Source module

Veir.InstSimplify.IS2271.pattern

Source

Variant 1: `sw0 = false; sw1 = false`

```
llvm: (X | (X ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (X | a1) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = true`

```
llvm: ((X ^ C0) | X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (a1 | X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = true; sw1 = false`

```
llvm: (X | (C0 ^ X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (X | a1) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = true; sw1 = true`

```
llvm: ((C0 ^ X) | X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (a1 | X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2275

Source module

Veir.InstSimplify.IS2275.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false`

```
llvm: (X | ((X & Y) ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  root = (X | a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = true`

```
llvm: (((X & Y) ^ C0) | X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  root = (a2 | X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = true; sw2 = false`

```
llvm: (X | (C0 ^ (X & Y)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  root = (X | a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = true; sw2 = true`

```
llvm: ((C0 ^ (X & Y)) | X)  =>  C0
where
```

```
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  root = (a2 | X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = true; sw1 = false; sw2 = false`

```
llvm: (X | ((Y & X) ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  root = (X | a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = true; sw1 = false; sw2 = true`

```
llvm: (((Y & X) ^ C0) | X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  root = (a2 | X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = true; sw1 = true; sw2 = false`

```
llvm: (X | (C0 ^ (Y & X)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  root = (X | a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = true; sw1 = true; sw2 = true`

```
llvm: ((C0 ^ (Y & X)) | X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  root = (a2 | X) : T0
  P3 = properties(root)
```

```
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2279

Source module

Veir.InstSimplify.IS2279.pattern

Source

Variant 1: `sw0 = false; sw1 = false`

```
llvm: (X | (X & Y))  =>  X
where
  a0 = (X & Y) : T0
  P0 = properties(a0)
  root = (X | a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 2: `sw0 = false; sw1 = true`

```
llvm: ((X & Y) | X)  =>  X
where
  a0 = (X & Y) : T0
  P0 = properties(a0)
  root = (a0 | X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 3: `sw0 = true; sw1 = false`

```
llvm: (X | (Y & X))  =>  X
where
  a0 = (Y & X) : T0
  P0 = properties(a0)
  root = (X | a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 4: `sw0 = true; sw1 = true`

```
llvm: ((Y & X) | X)  =>  X
where
  a0 = (Y & X) : T0
  P0 = properties(a0)
  root = (a0 | X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2287

Source module

Veir.InstSimplify.IS2287.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false`

```
llvm: ((X ^ Y) | a1)  =>  a1
where
  a0 = (X ^ Y) : T0
  P0 = properties(a0)
  a1 = (X | Y) : T0
```

```
  P1 = properties(a1)
  root = (a0 | a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 2: `sw0 = false; sw1 = false; sw2 = true`

```
llvm: (a1 | (X ^ Y))  =>  a1
where
  a0 = (X ^ Y) : T0
  P0 = properties(a0)
  a1 = (X | Y) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 3: `sw0 = false; sw1 = true; sw2 = false`

```
llvm: ((X ^ Y) | a1)  =>  a1
where
  a0 = (X ^ Y) : T0
  P0 = properties(a0)
  a1 = (Y | X) : T0
  P1 = properties(a1)
  root = (a0 | a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 4: `sw0 = false; sw1 = true; sw2 = true`

```
llvm: (a1 | (X ^ Y))  =>  a1
where
  a0 = (X ^ Y) : T0
  P0 = properties(a0)
  a1 = (Y | X) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 5: `sw0 = true; sw1 = false; sw2 = false`

```
llvm: ((Y ^ X) | a1)  =>  a1
where
  a0 = (Y ^ X) : T0
  P0 = properties(a0)
  a1 = (X | Y) : T0
  P1 = properties(a1)
  root = (a0 | a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 6: `sw0 = true; sw1 = false; sw2 = true`

```
llvm: (a1 | (Y ^ X))  =>  a1
where
  a0 = (Y ^ X) : T0
  P0 = properties(a0)
  a1 = (X | Y) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 7: `sw0 = true; sw1 = true; sw2 = false`

```
llvm: ((Y ^ X) | a1)  =>  a1
where
  a0 = (Y ^ X) : T0
  P0 = properties(a0)
  a1 = (Y | X) : T0
  P1 = properties(a1)
  root = (a0 | a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 8: `sw0 = true; sw1 = true; sw2 = true`

```
llvm: (a1 | (Y ^ X))  =>  a1
where
  a0 = (Y ^ X) : T0
  P0 = properties(a0)
  a1 = (Y | X) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2293

Source module

Veir.InstSimplify.IS2293.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (((X ^ Y) ^ C0) | (X | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X | Y) | ((X ^ Y) ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (((X ^ Y) ^ C0) | (Y | X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y | X) | ((X ^ Y) ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((C0 ^ (X ^ Y)) | (X | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X | Y) | (C0 ^ (X ^ Y)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((C0 ^ (X ^ Y)) | (Y | X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y | X) | (C0 ^ (X ^ Y)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (((Y ^ X) ^ C0) | (X | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X | Y) | ((Y ^ X) ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (((Y ^ X) ^ C0) | (Y | X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y | X) | ((Y ^ X) ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((C0 ^ (Y ^ X)) | (X | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X | Y) | (C0 ^ (Y ^ X)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((C0 ^ (Y ^ X)) | (Y | X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y | X) | (C0 ^ (Y ^ X)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2301

Source module

Veir.InstSimplify.IS2301.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((X & (Y ^ -1)) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: (a3 | (X & (Y ^ -1)))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((X & (Y ^ -1)) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: (a3 | (X & (Y ^ -1)))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (((Y ^ -1) & X) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: (a3 | ((Y ^ -1) & X))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (((Y ^ -1) & X) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: (a3 | ((Y ^ -1) & X))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((X & (-1 ^ Y)) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: (a3 | (X & (-1 ^ Y)))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((X & (-1 ^ Y)) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: (a3 | (X & (-1 ^ Y)))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (X & a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (((-1 ^ Y) & X) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: (a3 | ((-1 ^ Y) & X))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (((-1 ^ Y) & X) | a3)  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: (a3 | ((-1 ^ Y) & X))  =>  a3
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 & X) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2309

Source module

Veir.InstSimplify.IS2309.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
```

```
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
```

```
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
```

```
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
```

```
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2317

Source module

Veir.InstSimplify.IS2317.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (((X ^ C0) | Y) | (X ^ Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | ((X ^ C0) | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (((X ^ C0) | Y) | (Y ^ X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | ((X ^ C0) | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((Y | (X ^ C0)) | (X ^ Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | (Y | (X ^ C0)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((Y | (X ^ C0)) | (Y ^ X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | (Y | (X ^ C0)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (((C0 ^ X) | Y) | (X ^ Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | ((C0 ^ X) | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (((C0 ^ X) | Y) | (Y ^ X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | ((C0 ^ X) | Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((Y | (C0 ^ X)) | (X ^ Y))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | (Y | (C0 ^ X)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((Y | (C0 ^ X)) | (Y ^ X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | (Y | (C0 ^ X)))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2327

Source module

Veir.InstSimplify.IS2327.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false; sw4 = false`

```
llvm: ((a1 & Y) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
```

```
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = false; sw4 = true`

```
llvm: (((X | Y) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = false; sw3 = true; sw4 = false`

```
llvm: ((a1 & Y) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = false; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (X | Y)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = false; sw2 = true; sw3 = false; sw4 = false`

```
llvm: ((a1 & Y) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = false; sw2 = true; sw3 = false; sw4 = true`

```
llvm: (((Y | X) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = false; sw2 = true; sw3 = true; sw4 = false`

```
llvm: ((a1 & Y) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = false; sw2 = true; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (Y | X)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
```

```
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = false; sw1 = true; sw2 = false; sw3 = false; sw4 = false`

```
llvm: ((Y & a1) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = false; sw1 = true; sw2 = false; sw3 = false; sw4 = true`

```
llvm: (((X | Y) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = false; sw1 = true; sw2 = false; sw3 = true; sw4 = false`

```
llvm: ((Y & a1) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = false; sw1 = true; sw2 = false; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (X | Y)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
```

```
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = false; sw1 = true; sw2 = true; sw3 = false; sw4 = false`

```
llvm: ((Y & a1) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = false; sw1 = true; sw2 = true; sw3 = false; sw4 = true`

```
llvm: (((Y | X) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = false; sw1 = true; sw2 = true; sw3 = true; sw4 = false`

```
llvm: ((Y & a1) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = false; sw1 = true; sw2 = true; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (Y | X)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 17: `sw0 = true; sw1 = false; sw2 = false; sw3 = false; sw4 = false`

```
llvm: ((a1 & Y) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 18: `sw0 = true; sw1 = false; sw2 = false; sw3 = false; sw4 = true`

```
llvm: (((X | Y) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 19: `sw0 = true; sw1 = false; sw2 = false; sw3 = true; sw4 = false`

```
llvm: ((a1 & Y) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
```

```
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 20: `sw0 = true; sw1 = false; sw2 = false; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (X | Y)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 21: `sw0 = true; sw1 = false; sw2 = true; sw3 = false; sw4 = false`

```
llvm: ((a1 & Y) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 22: `sw0 = true; sw1 = false; sw2 = true; sw3 = false; sw4 = true`

```
llvm: (((Y | X) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 23: `sw0 = true; sw1 = false; sw2 = true; sw3 = true; sw4 = false`

```
llvm: ((a1 & Y) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 24: `sw0 = true; sw1 = false; sw2 = true; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (Y | X)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 25: `sw0 = true; sw1 = true; sw2 = false; sw3 = false; sw4 = false`

```
llvm: ((Y & a1) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 26: `sw0 = true; sw1 = true; sw2 = false; sw3 = false; sw4 = true`

```
llvm: (((X | Y) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
```

```
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 27: `sw0 = true; sw1 = true; sw2 = false; sw3 = true; sw4 = false`

```
llvm: ((Y & a1) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 28: `sw0 = true; sw1 = true; sw2 = false; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (X | Y)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 29: `sw0 = true; sw1 = true; sw2 = true; sw3 = false; sw4 = false`

```
llvm: ((Y & a1) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 30: `sw0 = true; sw1 = true; sw2 = true; sw3 = false; sw4 = true`

```
llvm: (((Y | X) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
```

```
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 31: `sw0 = true; sw1 = true; sw2 = true; sw3 = true; sw4 = false`

```
llvm: ((Y & a1) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 32: `sw0 = true; sw1 = true; sw2 = true; sw3 = true; sw4 = true`

```
llvm: ((C0 ^ (Y | X)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2334

Source module

Veir.InstSimplify.IS2334.pattern

Source

Variant 1: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((a1 & Y) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 3: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((a1 & Y) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 4: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 5: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 6: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 7: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 8: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 9: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((a1 & Y) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 10: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 11: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((a1 & Y) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 12: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 13: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 14: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 15: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 16: `selectAnd = false; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 17: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((Y & a1) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 18: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 19: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((Y & a1) | ((X | Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 20: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ (X | Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 21: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 22: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 23: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 24: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 25: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((Y & a1) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 26: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 27: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((Y & a1) | ((Y | X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 28: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ (Y | X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 29: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 30: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 31: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 32: `selectAnd = false; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 33: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((a1 & Y) | (select(X, C0, Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 34: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ select(X, C0, Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 35: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((a1 & Y) | (select(X, C0, Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 36: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ select(X, C0, Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 37: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 38: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 39: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 40: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 41: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((a1 & Y) | (select(Y, C0, X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 42: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ select(Y, C0, X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 43: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((a1 & Y) | (select(Y, C0, X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 44: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((a1 & Y) | (C0 ^ select(Y, C0, X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 45: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 46: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 47: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 48: `selectAnd = false; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | (a1 & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 49: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((Y & a1) | (select(X, C0, Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 50: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ select(X, C0, Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 51: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((Y & a1) | (select(X, C0, Y) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 52: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ select(X, C0, Y)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 53: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 54: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 55: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 56: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(X, C0, Y) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 57: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: ((Y & a1) | (select(Y, C0, X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 58: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ select(Y, C0, X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 59: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: ((Y & a1) | (select(Y, C0, X) ^ C0))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 60: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: ((Y & a1) | (C0 ^ select(Y, C0, X)))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a2 | a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 61: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
```

```
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 62: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 63: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
  P3 = properties(a3)
  a4 = (a3 ^ C0) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 64: `selectAnd = false; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | (Y & a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = select(Y, C0, X) : T0
```

```
  P3 = properties(a3)
  a4 = (C0 ^ a3) : T0
  P4 = properties(a4)
  root = (a4 | a2) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 65: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | ((X | Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 66: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ (X | Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 67: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | ((X | Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
```

```
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 68: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ (X | Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 69: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 70: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 71: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 72: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 73: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | ((Y | X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 74: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ (Y | X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 75: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | ((Y | X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 76: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ (Y | X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 77: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 78: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 79: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 80: `selectAnd = true; selectOr = false; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 81: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | ((X | Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 82: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ (X | Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 83: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | ((X | Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 84: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ (X | Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 85: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 86: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 87: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((X | Y) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 88: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (X | Y)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 89: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | ((Y | X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 90: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ (Y | X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 91: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | ((Y | X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 92: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ (Y | X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 93: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 94: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 95: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: (((Y | X) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 96: `selectAnd = true; selectOr = false; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ (Y | X)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 97: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | (select(X, C0, Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 98: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ select(X, C0, Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 99: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | (select(X, C0, Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 100: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ select(X, C0, Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 101: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 102: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 103: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 104: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 105: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | (select(Y, C0, X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 106: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ select(Y, C0, X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 107: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(a2, Y, 0) | (select(Y, C0, X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 108: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(a2, Y, 0) | (C0 ^ select(Y, C0, X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 109: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 110: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 111: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 112: `selectAnd = true; selectOr = true; swapAnd = false; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | select(a2, Y, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(a2, Y, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 113: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | (select(X, C0, Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 114: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ select(X, C0, Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 115: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | (select(X, C0, Y) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 116: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ select(X, C0, Y)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 117: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 118: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 119: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(X, C0, Y) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 120: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = false; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(X, C0, Y)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(X, C0, Y) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 121: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | (select(Y, C0, X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 122: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = false; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ select(Y, C0, X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 123: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = false`

```
llvm: (select(Y, a2, 0) | (select(Y, C0, X) ^ C0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 124: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = false; swapNotA = true; swapNotOr = true`

```
llvm: (select(Y, a2, 0) | (C0 ^ select(Y, C0, X)))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a3 | a5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 125: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
```

```
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 126: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = false; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 127: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = false`

```
llvm: ((select(Y, C0, X) ^ C0) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (a4 ^ C0) : T0
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 128: `selectAnd = true; selectOr = true; swapAnd = true; swapOr = true; swapRoot = true; swapNotA = true; swapNotOr = true`

```
llvm: ((C0 ^ select(Y, C0, X)) | select(Y, a2, 0))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (C0 ^ X) : T0
  P2 = properties(a2)
  a3 = select(Y, a2, C1) : T0
  P3 = properties(a3)
  a4 = select(Y, C0, X) : T0
  P4 = properties(a4)
  a5 = (C0 ^ a4) : T0
```

```
  P5 = properties(a5)
  root = (a5 | a3) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS2342

Source module

Veir.InstSimplify.IS2342.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (a2 | (X & Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X & Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y & X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y & X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y ^ X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2349

Source module

Veir.InstSimplify.IS2349.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (a2 | (X ^ Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y ^ X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (a2 | (X ^ Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y ^ X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (a2 | (X ^ Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y ^ X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: (a2 | (X ^ Y))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X ^ Y) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (X ^ Y) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: (a2 | (Y ^ X))  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
```

```
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y ^ X) | a2)  =>  a2
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (C0 ^ a1) : T0
  P2 = properties(a2)
  a3 = (Y ^ X) : T0
  P3 = properties(a3)
  root = (a3 | a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2363

Source module

Veir.InstSimplify.IS2363.pattern

Source

```
llvm: (X | a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (X | a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2369

Source module

Veir.InstSimplify.IS2369.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X | C0)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = (X | C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = true`

```
llvm: (C0 | X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = (C0 | X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2374

Source module

Veir.InstSimplify.IS2374.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X | X)  =>  X
where
  root = (X | X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Variant 2: `sw0 = true`

```
llvm: (X | X)  =>  X
where
  root = (X | X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2374.Zero.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X | 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (X | C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 2: `sw0 = true`

```
llvm: (0 | X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (C0 | X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS2397

Source module

Veir.InstSimplify.IS2397.pattern

Source

Variant 1: `constant : Int; subOnLeft = false; commute = false`

```
llvm: (shl(C1, X) | lshr(C2, (constant - X)))  =>  C1
where
  C0 = constant : T0
  P0 = properties(C0)
  C1 = -1 : T0
  P1 = properties(C1)
  C2 = -1 : T0
  P2 = properties(C2)
  a3 = (C0 - X) : T0
  P3 = properties(a3)
  a4 = shl(C1, X) : T0
  P4 = properties(a4)
  a5 = lshr(C2, a3) : T0
  P5 = properties(a5)
```

```
  root = (a4 | a5) : T0
  P6 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 constant
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P2 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => decide ((BitVec.ofInt t.bitwidth constant).toNat ≤ t.bitwidth)
  | x => false
```

Variant 2: `constant : Int; subOnLeft = false; commute = true`

```
llvm: (lshr(C2, (constant - X)) | shl(C1, X))  =>  C1
where
  C0 = constant : T0
  P0 = properties(C0)
  C1 = -1 : T0
  P1 = properties(C1)
  C2 = -1 : T0
  P2 = properties(C2)
  a3 = (C0 - X) : T0
  P3 = properties(a3)
  a4 = shl(C1, X) : T0
  P4 = properties(a4)
  a5 = lshr(C2, a3) : T0
  P5 = properties(a5)
  root = (a5 | a4) : T0
  P6 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 constant
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P2 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => decide ((BitVec.ofInt t.bitwidth constant).toNat ≤ t.bitwidth)
  | x => false
```

Variant 3: `constant : Int; subOnLeft = true; commute = false`

```
llvm: (shl(C1, (constant - X)) | lshr(C2, X))  =>  C1
where
  C0 = constant : T0
  P0 = properties(C0)
  C1 = -1 : T0
  P1 = properties(C1)
  C2 = -1 : T0
  P2 = properties(C2)
  a3 = (C0 - X) : T0
  P3 = properties(a3)
  a4 = shl(C1, a3) : T0
  P4 = properties(a4)
  a5 = lshr(C2, X) : T0
  P5 = properties(a5)
  root = (a4 | a5) : T0
  P6 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 constant
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P2 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => decide ((BitVec.ofInt t.bitwidth constant).toNat ≤ t.bitwidth)
  | x => false
```

Variant 4: `constant : Int; subOnLeft = true; commute = true`

```
llvm: (lshr(C2, X) | shl(C1, (constant - X)))  =>  C1
where
  C0 = constant : T0
  P0 = properties(C0)
  C1 = -1 : T0
  P1 = properties(C1)
  C2 = -1 : T0
  P2 = properties(C2)
  a3 = (C0 - X) : T0
```

```
  P3 = properties(a3)
  a4 = shl(C1, a3) : T0
  P4 = properties(a4)
  a5 = lshr(C2, X) : T0
  P5 = properties(a5)
  root = (a5 | a4) : T0
  P6 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 constant
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P2 (-1)
  require match T0.val with
  | Veir.Attribute.integerType t => decide ((BitVec.ofInt t.bitwidth constant).toNat ≤ t.bitwidth)
  | x => false
```

Veir.InstSimplify.IS2406

Source module

Veir.InstSimplify.IS2406.pattern

Source

Variant 1: `commute = false`

```
llvm: (zext(X) | a1)  =>  a1
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = (a0 | a1) : T1
  P2 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

Variant 2: `commute = true`

```
llvm: (a1 | zext(X))  =>  a1
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = (a1 | a0) : T1
  P2 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2408

Source module

Veir.InstSimplify.IS2408.pattern

Source

Builder families: `Veir.InstSimplify.IS2406.pattern`.

```
llvm: (a1 | zext(X))  =>  a1
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = (a1 | a0) : T1
  P2 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2419

Source module

Veir.InstSimplify.IS2419

Source

Variant 1: `swap = false`

```
llvm: (a0 | shl(X, Z))  =>  a0
where
  a0 = intr.fshl(X, Y, Z) : T0
  P0 = properties(a0)
  a1 = shl(X, Z) : T0
  P1 = properties(a1)
  root = (a0 | a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  Z : T0
  T0 : Veir.IntegerType
```

Variant 2: `swap = true`

```
llvm: (shl(X, Z) | a0)  =>  a0
where
  a0 = intr.fshl(X, Y, Z) : T0
  P0 = properties(a0)
  a1 = shl(X, Z) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  Z : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2423

Source module

Veir.InstSimplify.IS2423

Source

Builder families: `Veir.InstSimplify.IS2419`.

```
llvm: (shl(X, Z) | a0)  =>  a0
where
  a0 = intr.fshl(X, Y, Z) : T0
  P0 = properties(a0)
  a1 = shl(X, Z) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  Z : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2430

Source module

Veir.InstSimplify.IS2430

Source

Variant 1: `swap = false`

```
llvm: (a0 | lshr(Y, Z))  =>  a0
where
  a0 = intr.fshr(X, Y, Z) : T0
  P0 = properties(a0)
  a1 = lshr(Y, Z) : T0
  P1 = properties(a1)
  root = (a0 | a1) : T0
```

```
  P2 = properties(root)
  X : T0
  Y : T0
  Z : T0
  T0 : Veir.IntegerType
```

Variant 2: `swap = true`

```
llvm: (lshr(Y, Z) | a0)  =>  a0
where
  a0 = intr.fshr(X, Y, Z) : T0
  P0 = properties(a0)
  a1 = lshr(Y, Z) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  Z : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2434

Source module

Veir.InstSimplify.IS2434

Source

Builder families: `Veir.InstSimplify.IS2430`.

```
llvm: (lshr(Y, Z) | a0)  =>  a0
where
  a0 = intr.fshr(X, Y, Z) : T0
  P0 = properties(a0)
  a1 = lshr(Y, Z) : T0
  P1 = properties(a1)
  root = (a1 | a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  Z : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2468

Source module

Veir.InstSimplify.IS2468.pattern

Source

Variant 1: `commute = false`

```
llvm: (X | a1)  =>  a1
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  root = (X | a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 1
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Variant 2: `commute = true`

```
llvm: (a1 | X)  =>  a1
where
  C0 = 1 : T0
  P0 = properties(C0)
```

```
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  root = (a1 | X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 1
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS2470

Source module

Veir.InstSimplify.IS2470.pattern

Source

Builder families: `Veir.InstSimplify.IS2468.pattern`.

```
llvm: (a1 | X)  =>  a1
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  root = (a1 | X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 1
  require match T0.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
```

Veir.InstSimplify.IS2515

Source module

Veir.InstSimplify.IS2515.pattern

Source

Parameters: `c : Int`

```
llvm: ((X ^ c) | (X ^ (-c - 1)))  =>  C5
where
  C0 = c : T0
  P0 = properties(C0)
  C1 = (-c - 1) : T0
  P1 = properties(C1)
  a2 = (X ^ C0) : T0
  P2 = properties(a2)
  a3 = (X ^ C1) : T0
  P3 = properties(a3)
  root = (a2 | a3) : T0
  P4 = properties(root)
  C5 = mlir.constant() : T0 [created]
  P5 = properties(C5)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-c - 1)
  P5 ← match T0.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt (-1)
t) }
  | x => none
```

Veir.InstSimplify.IS2557

Source module

Veir.InstSimplify.IS2557.pattern

Source

```
llvm: (X ^ a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = (X ^ a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS2565

Source module

Veir.InstSimplify.IS2565.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X ^ 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (X ^ C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 2: `sw0 = true`

```
llvm: (0 ^ X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = (C0 ^ X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS2569

Source module

Veir.InstSimplify.IS2569.pattern

Source

Variant 1: `sw0 = false`

```
llvm: (X ^ X)  =>  mlir.zero()
where
  root = (X ^ X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Variant 2: `sw0 = true`

```
llvm: (X ^ X)  =>  mlir.zero()
where
  root = (X ^ X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Veir.InstSimplify.IS2573

Source module

Veir.InstSimplify.IS2573.pattern

Source

Variant 1: `sw0 = false; sw1 = false`

```
llvm: (X ^ (X ^ C0))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (X ^ a1) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = true`

```
llvm: ((X ^ C0) ^ X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  root = (a1 ^ X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = true; sw1 = false`

```
llvm: (X ^ (C0 ^ X))  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (X ^ a1) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = true; sw1 = true`

```
llvm: ((C0 ^ X) ^ X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  root = (a1 ^ X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2580

Source module

Veir.InstSimplify.IS2580.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (((X ^ -1) & Y) ^ (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X | Y) ^ ((X ^ -1) & Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (((X ^ -1) & Y) ^ (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y | X) ^ ((X ^ -1) & Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((Y & (X ^ -1)) ^ (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X | Y) ^ (Y & (X ^ -1)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((Y & (X ^ -1)) ^ (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y | X) ^ (Y & (X ^ -1)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: (((-1 ^ X) & Y) ^ (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X | Y) ^ ((-1 ^ X) & Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: (((-1 ^ X) & Y) ^ (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y | X) ^ ((-1 ^ X) & Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 & Y) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((Y & (-1 ^ X)) ^ (X | Y))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
```

```
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X | Y) ^ (Y & (-1 ^ X)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((Y & (-1 ^ X)) ^ (Y | X))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y | X) ^ (Y & (-1 ^ X)))  =>  X
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y & a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2589

Source module

Veir.InstSimplify.IS2589.pattern

Source

Variant 1: `sw0 = false; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((a1 | Y) ^ (X & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `sw0 = false; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X & Y) ^ (a1 | Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 3: `sw0 = false; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((a1 | Y) ^ (Y & X))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 4: `sw0 = false; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y & X) ^ (a1 | Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 5: `sw0 = false; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((Y | a1) ^ (X & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 6: `sw0 = false; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X & Y) ^ (Y | a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 7: `sw0 = false; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((Y | a1) ^ (Y & X))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 8: `sw0 = false; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y & X) ^ (Y | a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 9: `sw0 = true; sw1 = false; sw2 = false; sw3 = false`

```
llvm: ((a1 | Y) ^ (X & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 10: `sw0 = true; sw1 = false; sw2 = false; sw3 = true`

```
llvm: ((X & Y) ^ (a1 | Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 11: `sw0 = true; sw1 = false; sw2 = true; sw3 = false`

```
llvm: ((a1 | Y) ^ (Y & X))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 12: `sw0 = true; sw1 = false; sw2 = true; sw3 = true`

```
llvm: ((Y & X) ^ (a1 | Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (a1 | Y) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 13: `sw0 = true; sw1 = true; sw2 = false; sw3 = false`

```
llvm: ((Y | a1) ^ (X & Y))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 14: `sw0 = true; sw1 = true; sw2 = false; sw3 = true`

```
llvm: ((X & Y) ^ (Y | a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (X & Y) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 15: `sw0 = true; sw1 = true; sw2 = true; sw3 = false`

```
llvm: ((Y | a1) ^ (Y & X))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a2 ^ a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 16: `sw0 = true; sw1 = true; sw2 = true; sw3 = true`

```
llvm: ((Y & X) ^ (Y | a1))  =>  a1
where
  C0 = -1 : T0
  P0 = properties(C0)
  a1 = (C0 ^ X) : T0
  P1 = properties(a1)
  a2 = (Y | a1) : T0
  P2 = properties(a2)
  a3 = (Y & X) : T0
  P3 = properties(a3)
  root = (a3 ^ a2) : T0
  P4 = properties(root)
  X : T0
  Y : T0
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS2623

Source module

Veir.InstSimplify.IS2623.pattern

Source

Parameters: `k : Nat`

```
llvm: ((C0 - X) ^ C0)  =>  X
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (C0 - X) : T0
  P1 = properties(a1)
  root = (a1 ^ C0) : T0
  P2 = properties(root)
  X : T0
  require P1.nuw
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require match T0.val with
  | Veir.Attribute.integerType t => decide (k ≤ t.bitwidth)
  | x => false
```

Veir.InstSimplify.IS2928

Source module

Veir.InstSimplify.IS2928

Source

```
llvm: icmp(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T0
  P1 = properties(root)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.ne || P1.predicate == Veir.Data.LLVM.IntPred.ugt ||
    P1.predicate == Veir.Data.LLVM.IntPred.slt
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS2934

Source module

Veir.InstSimplify.IS2934

Source

```
llvm: icmp((X ^ 1), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  root = icmp(a1, C2) : T0
  P3 = properties(root)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq || P3.predicate == Veir.Data.LLVM.IntPred.ule ||
    P3.predicate == Veir.Data.LLVM.IntPred.sge
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS2939

Source module

Veir.InstSimplify.IS2939

Source

```
llvm: icmp(X, C0)  =>  C2
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T0
  P1 = properties(root)
  C2 = 0 : T0 [created]
  P2 = properties(C2)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.ult || P1.predicate == Veir.Data.LLVM.IntPred.sgt
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS2943

Source module

Veir.InstSimplify.IS2943

Source

```
llvm: icmp(X, 0)  =>  1
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T0
  P1 = properties(root)
  C2 = 1 : T0 [created]
  P2 = properties(C2)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.uge || P1.predicate == Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS2953

Source module

Veir.InstSimplify.IS2953

Source

```
llvm: icmp(X, 1)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T0
  P1 = properties(root)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq || P1.predicate == Veir.Data.LLVM.IntPred.uge ||
    P1.predicate == Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS2959

Source module

Veir.InstSimplify.IS2959

Source

```
llvm: icmp((X ^ C0), C2)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
```

```
  a1 = (X ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  root = icmp(a1, C2) : T0
  P3 = properties(root)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne || P3.predicate == Veir.Data.LLVM.IntPred.ult ||
    P3.predicate == Veir.Data.LLVM.IntPred.sgt
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS2964

Source module

Veir.InstSimplify.IS2964

Source

```
llvm: icmp(X, 1)  =>  0
where
  C0 = 1 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T0
  P1 = properties(root)
  C2 = 0 : T0 [created]
  P2 = properties(C2)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.ugt || P1.predicate == Veir.Data.LLVM.IntPred.slt
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS2968

Source module

Veir.InstSimplify.IS2968

Source

```
llvm: icmp(X, C0)  =>  C2
where
  C0 = 1 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T0
  P1 = properties(root)
  C2 = 1 : T0 [created]
  P2 = properties(C2)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.ule || P1.predicate == Veir.Data.LLVM.IntPred.sge
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3074

Source module

Veir.InstSimplify.IS3074

Source

```
llvm: icmp(X, C0)  =>  C2
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
```

```
  require P1.predicate == Veir.Data.LLVM.IntPred.ult
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3076

Source module

Veir.InstSimplify.IS3076

Source

```
llvm: icmp(X, 0)  =>  1
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.uge
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3146

Source module

Veir.InstSimplify.IS3146

Source

Parameters: `pred : Veir.Data.LLVM.IntPred; constant : Int`

```
llvm: icmp(X, constant)  =>  0
where
  C0 = constant : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  require P1.predicate == pred
  require Veir.InstSimplify.intConstantGuard P0 constant
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (0 < t.bitwidth) && Veir.InstSimplify.icmpEmptyGuard pred
t.bitwidth constant
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3148

Source module

Veir.InstSimplify.IS3148

Source

Parameters: `pred : Veir.Data.LLVM.IntPred; constant : Int`

```
llvm: icmp(X, constant)  =>  1
where
  C0 = constant : T0
  P0 = properties(C0)
  root = icmp(X, C0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  require P1.predicate == pred
  require Veir.InstSimplify.intConstantGuard P0 constant
```

```
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (0 < t.bitwidth) && Veir.InstSimplify.icmpFullGuard pred t.bitwidth
constant
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3166

Source module

Veir.InstSimplify.IS3166

Source

Variant 1: `multiplier : Int; constant : Int; unsigned = false; equal = false`

```
llvm: icmp((X * multiplier), constant)  =>  1
where
  C0 = multiplier : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require P2.nsw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 multiplier
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth multiplier != 0 &&
    (BitVec.ofInt t.bitwidth constant).srem (BitVec.ofInt t.bitwidth multiplier) != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `multiplier : Int; constant : Int; unsigned = false; equal = true`

```
llvm: icmp((X * multiplier), constant)  =>  0
where
  C0 = multiplier : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require P2.nsw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 multiplier
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth multiplier != 0 &&
    (BitVec.ofInt t.bitwidth constant).srem (BitVec.ofInt t.bitwidth multiplier) != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `multiplier : Int; constant : Int; unsigned = true; equal = false`

```
llvm: icmp((X * multiplier), constant)  =>  1
where
  C0 = multiplier : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
```

```
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require P2.nuw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 multiplier
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth multiplier != 0 && BitVec.ofInt t.bitwidth
constant % BitVec.ofInt t.bitwidth multiplier != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `multiplier : Int; constant : Int; unsigned = true; equal = true`

```
llvm: icmp((X * multiplier), constant)  =>  0
where
  C0 = multiplier : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require P2.nuw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 multiplier
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth multiplier != 0 && BitVec.ofInt t.bitwidth
constant % BitVec.ofInt t.bitwidth multiplier != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3280

Source module

Veir.InstSimplify.IS3280

Source

```
llvm: icmp(urem(X, Y), Y)  =>  0
where
  a0 = urem(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, Y) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq || P1.predicate == Veir.Data.LLVM.IntPred.ugt ||
    P1.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3291

Source module

Veir.InstSimplify.IS3291

Source

```
llvm: icmp(urem(X, Y), Y)  =>  1
where
  a0 = urem(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, Y) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
```

```
  require P1.predicate == Veir.Data.LLVM.IntPred.ne || P1.predicate == Veir.Data.LLVM.IntPred.ult ||
    P1.predicate == Veir.Data.LLVM.IntPred.ule
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3341

Source module

Veir.InstSimplify.IS3341_mulUdiv

Source

Parameters: `a : Int; b : Int`

```
llvm: icmp(udiv((X * a), b), X)  =>  0
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3341_mulLshr

Source

Parameters: `a : Int; b : Int`

```
llvm: icmp(lshr((X * a), b), X)  =>  0
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  a3 = lshr(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.twoPow t.bitwidth (BitVec.ofInt
t.bitwidth b).toNat)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3341_shlUdiv

Source

Parameters: `a : Int; b : Int`

```
llvm: icmp(udiv(shl(X, a), b), X)  =>  0
where
  C0 = a : T0
  P0 = properties(C0)
```

```
  C1 = b : T0
  P1 = properties(C1)
  a2 = shl(X, C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.twoPow t.bitwidth (BitVec.ofInt t.bitwidth a).toNat).ule
(BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3341

Source

Builder families: `Veir.InstSimplify.IS3341_mulLshr`, `Veir.InstSimplify.IS3341_mulUdiv`, `Veir.InstSimplify.IS3341_shlUdiv`.

Variant 1: `shape = .mulUdiv; a : Int; b : Int`

```
llvm: icmp(udiv((X * a), b), X)  =>  0
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `shape = .mulLshr; a : Int; b : Int`

```
llvm: icmp(lshr((X * a), b), X)  =>  0
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  a3 = lshr(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.twoPow t.bitwidth (BitVec.ofInt
t.bitwidth b).toNat)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `shape = .shlUdiv; a : Int; b : Int`

```
llvm: icmp(udiv(shl(X, a), b), X)  =>  0
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = shl(X, C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.twoPow t.bitwidth (BitVec.ofInt t.bitwidth a).toNat).ule
(BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3343

Source module

Veir.InstSimplify.IS3343_mulUdiv

Source

Parameters: `a : Int; b : Int`

```
llvm: icmp(udiv((X * a), b), X)  =>  1
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3343_mulLshr

Source

Parameters: `a : Int; b : Int`

```
llvm: icmp(lshr((X * a), b), X)  =>  1
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  a3 = lshr(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
```

```
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.twoPow t.bitwidth (BitVec.ofInt
t.bitwidth b).toNat)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3343_shlUdiv

Source

Parameters: `a : Int; b : Int`

```
llvm: icmp(udiv(shl(X, a), b), X)  =>  1
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = shl(X, C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.twoPow t.bitwidth (BitVec.ofInt t.bitwidth a).toNat).ule
(BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3343

Source

Builder families: `Veir.InstSimplify.IS3343_mulLshr`, `Veir.InstSimplify.IS3343_mulUdiv`, `Veir.InstSimplify.IS3343_shlUdiv`.

Variant 1: `shape = .mulUdiv; a : Int; b : Int`

```
llvm: icmp(udiv((X * a), b), X)  =>  1
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `shape = .mulLshr; a : Int; b : Int`

```
llvm: icmp(lshr((X * a), b), X)  =>  1
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = (X * C0) : T0
  P2 = properties(a2)
```

```
  a3 = lshr(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth a).ule (BitVec.twoPow t.bitwidth (BitVec.ofInt
t.bitwidth b).toNat)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `shape = .shlUdiv; a : Int; b : Int`

```
llvm: icmp(udiv(shl(X, a), b), X)  =>  1
where
  C0 = a : T0
  P0 = properties(C0)
  C1 = b : T0
  P1 = properties(C1)
  a2 = shl(X, C0) : T0
  P2 = properties(a2)
  a3 = udiv(a2, C1) : T0
  P3 = properties(a3)
  root = icmp(a3, X) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 b
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.twoPow t.bitwidth (BitVec.ofInt t.bitwidth a).toNat).ule
(BitVec.ofInt t.bitwidth b)
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3350

Source module

Veir.InstSimplify.IS3350

Source

Variant 1: `constant : Int; equal = false`

```
llvm: icmp((constant - X), X)  =>  1
where
  C0 = constant : T0
  P0 = properties(C0)
  a1 = (C0 - X) : T0
  P1 = properties(a1)
  root = icmp(a1, X) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P0 constant
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth constant).getLsbD 0
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `constant : Int; equal = true`

```
llvm: icmp((constant - X), X)  =>  0
where
  C0 = constant : T0
  P0 = properties(C0)
  a1 = (C0 - X) : T0
  P1 = properties(a1)
  root = icmp(a1, X) : T1
```

```
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P0 constant
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: (BitVec.ofInt t.bitwidth constant).getLsbD 0
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3481

Source module

Veir.InstSimplify.IS3481

Source

Parameters: `constant : Int`

```
llvm: icmp((0 - zext(X)), constant)  =>  1
where
  C0 = 0 : T1
  P0 = properties(C0)
  C1 = constant : T1
  P1 = properties(C1)
  a2 = zext(X) : T1
  P2 = properties(a2)
  a3 = (C0 - a2) : T1
  P3 = properties(a3)
  root = icmp(a3, C1) : T2
  P4 = properties(root)
  C5 = 1 : T2 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.slt || P4.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 0
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType, where t = T1: decide (0 < (BitVec.ofInt t.bitwidth constant).toInt)
  T0 : Veir.IntegerType
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3483

Source module

Veir.InstSimplify.IS3483

Source

Parameters: `constant : Int`

```
llvm: icmp((C0 - zext(X)), constant)  =>  C5
where
  C0 = 0 : T1
  P0 = properties(C0)
  C1 = constant : T1
  P1 = properties(C1)
  a2 = zext(X) : T1
  P2 = properties(a2)
  a3 = (C0 - a2) : T1
  P3 = properties(a3)
  root = icmp(a3, C1) : T2
  P4 = properties(root)
  C5 = 0 : T2 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sge || P4.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 0
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType, where t = T1: decide (0 < (BitVec.ofInt t.bitwidth constant).toInt)
  T0 : Veir.IntegerType
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3487

Source module

Veir.InstSimplify.IS3487

Source

Parameters: `constant : Int`

```
llvm: icmp((0 - zext(X)), constant)  =>  1
where
  C0 = 0 : T1
  P0 = properties(C0)
  C1 = constant : T1
  P1 = properties(C1)
  a2 = zext(X) : T1
  P2 = properties(a2)
  a3 = (C0 - a2) : T1
  P3 = properties(a3)
  root = icmp(a3, C1) : T2
  P4 = properties(root)
  C5 = 1 : T2 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 0
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType, where t = T1: decide (0 ≤ (BitVec.ofInt t.bitwidth constant).toInt)
  T0 : Veir.IntegerType
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3489

Source module

Veir.InstSimplify.IS3489

Source

Parameters: `constant : Int`

```
llvm: icmp((C0 - zext(X)), constant)  =>  C5
where
  C0 = 0 : T1
  P0 = properties(C0)
  C1 = constant : T1
  P1 = properties(C1)
  a2 = zext(X) : T1
  P2 = properties(a2)
  a3 = (C0 - a2) : T1
  P3 = properties(a3)
  root = icmp(a3, C1) : T2
  P4 = properties(root)
  C5 = 0 : T2 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 0
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType, where t = T1: decide (0 ≤ (BitVec.ofInt t.bitwidth constant).toInt)
  T0 : Veir.IntegerType
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3512

Source module

Veir.InstSimplify.IS3512_noSignedWrap

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
```

```
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require P2.nsw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3512_noUnsignedWrap

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require P2.nuw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3512_baseOne

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
```

```
    BitVec.ofInt t.bitwidth base == 1
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3512_nonzeroRHS

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
    BitVec.ofInt t.bitwidth constant != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3512

Source

Builder families: `Veir.InstSimplify.IS3512_baseOne`, `Veir.InstSimplify.IS3512_noSignedWrap`, `Veir.InstSimplify.IS3512_noUnsignedWrap`, `Veir.InstSimplify.IS3512_nonzeroRHS`.

Variant 1: `safety = .noSignedWrap; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require P2.nsw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `safety = .noUnsignedWrap; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
```

```
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require P2.nuw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `safety = .baseOne; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
    BitVec.ofInt t.bitwidth base == 1
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `safety = .nonzeroRHS; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
    BitVec.ofInt t.bitwidth constant != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3514

Source module

Veir.InstSimplify.IS3514_noSignedWrap

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
```

```
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require P2.nsw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3514_noUnsignedWrap

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require P2.nuw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3514_baseOne

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
    BitVec.ofInt t.bitwidth base == 1
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3514_nonzeroRHS

Source

Parameters: `base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
    BitVec.ofInt t.bitwidth constant != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3514

Source

Builder families: `Veir.InstSimplify.IS3514_baseOne`, `Veir.InstSimplify.IS3514_noSignedWrap`, `Veir.InstSimplify.IS3514_noUnsignedWrap`, `Veir.InstSimplify.IS3514_nonzeroRHS`.

Variant 1: `safety = .noSignedWrap; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require P2.nsw
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `safety = .noUnsignedWrap; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require P2.nuw
```

```
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `safety = .baseOne; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
    BitVec.ofInt t.bitwidth base == 1
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `safety = .nonzeroRHS; base : Int; constant : Int; bit : Nat`

```
llvm: icmp(shl(base, X), constant)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = constant : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P1 constant
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
      !decide (BitVec.ofInt t.bitwidth constant).toNat.isPowerOfTwo &&
    BitVec.ofInt t.bitwidth constant != 0
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3523

Source module

Veir.InstSimplify.IS3523

Source

Parameters: `base : Int; mask : Int; bit : Nat`

```
llvm: icmp(shl(base, X), mask)  =>  0
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = mask : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
```

```
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 0 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 mask
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    BitVec.ofInt t.bitwidth mask == BitVec.intMin t.bitwidth
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3525

Source module

Veir.InstSimplify.IS3525

Source

Parameters: `base : Int; mask : Int; bit : Nat`

```
llvm: icmp(shl(base, X), mask)  =>  1
where
  C0 = base : T0
  P0 = properties(C0)
  C1 = mask : T0
  P1 = properties(C1)
  a2 = shl(C0, X) : T0
  P2 = properties(a2)
  root = icmp(a2, C1) : T1
  P3 = properties(root)
  C4 = 1 : T1 [created]
  P4 = properties(C4)
  X : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 mask
  require Veir.InstSimplify.intConstantGuard P0 base
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth base == BitVec.twoPow
t.bitwidth bit &&
    BitVec.ofInt t.bitwidth mask == BitVec.intMin t.bitwidth
  P4 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3563

Source module

Veir.InstSimplify.IS3563

Source

Variant 1: `small : Int; large : Int; useOr = false; reverse = false`

```
llvm: icmp((X & small), (X & large))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
```

```
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `small : Int; large : Int; useOr = false; reverse = true`

```
llvm: icmp((X & large), (X & small))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `small : Int; large : Int; useOr = true; reverse = false`

```
llvm: icmp((X | small), (X | large))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `small : Int; large : Int; useOr = true; reverse = true`

```
llvm: icmp((X | large), (X | small))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
```

```
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3565

Source module

Veir.InstSimplify.IS3565

Source

Variant 1: `small : Int; large : Int; useOr = false; reverse = false`

```
llvm: icmp((X & small), (X & large))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `small : Int; large : Int; useOr = false; reverse = true`

```
llvm: icmp((X & large), (X & small))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `small : Int; large : Int; useOr = true; reverse = false`

```
llvm: icmp((X | small), (X | large))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
```

```
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `small : Int; large : Int; useOr = true; reverse = true`

```
llvm: icmp((X | large), (X | small))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ugt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3568

Source module

Veir.InstSimplify.IS3568

Source

Variant 1: `small : Int; large : Int; useOr = false; reverse = false`

```
llvm: icmp((X & small), (X & large))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `small : Int; large : Int; useOr = false; reverse = true`

```
llvm: icmp((X & large), (X & small))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
```

```
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `small : Int; large : Int; useOr = true; reverse = false`

```
llvm: icmp((X | small), (X | large))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `small : Int; large : Int; useOr = true; reverse = true`

```
llvm: icmp((X | large), (X | small))  =>  1
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 1 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3570

Source module

Veir.InstSimplify.IS3570

Source

Variant 1: `small : Int; large : Int; useOr = false; reverse = false`

```
llvm: icmp((X & small), (X & large))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `small : Int; large : Int; useOr = false; reverse = true`

```
llvm: icmp((X & large), (X & small))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sgt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `small : Int; large : Int; useOr = true; reverse = false`

```
llvm: icmp((X | small), (X | large))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a2, a3) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.sgt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `small : Int; large : Int; useOr = true; reverse = true`

```
llvm: icmp((X | large), (X | small))  =>  0
where
  C0 = small : T0
  P0 = properties(C0)
  C1 = large : T0
  P1 = properties(C1)
  a2 = (X | C0) : T0
  P2 = properties(a2)
  a3 = (X | C1) : T0
  P3 = properties(a3)
  root = icmp(a3, a2) : T1
  P4 = properties(root)
  C5 = 0 : T1 [created]
  P5 = properties(C5)
  X : T0
  require P4.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sgt
  require Veir.InstSimplify.intConstantGuard P1 large
  require Veir.InstSimplify.intConstantGuard P0 small
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth small &&& BitVec.ofInt t.bitwidth large ==
BitVec.ofInt t.bitwidth small &&
    (BitVec.ofInt t.bitwidth small).msb == (BitVec.ofInt t.bitwidth large).msb
  P5 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3700

Source module

Veir.InstSimplify.IS3700

Source

Variant 1: `isMin = false; commute = false; onRhs = false`

```
llvm: icmp(intr.smax(X, Y), X)  =>  1
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `isMin = false; commute = false; onRhs = true`

```
llvm: icmp(X, intr.smax(X, Y))  =>  1
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `isMin = false; commute = true; onRhs = false`

```
llvm: icmp(intr.smax(Y, X), X)  =>  1
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `isMin = false; commute = true; onRhs = true`

```
llvm: icmp(X, intr.smax(Y, X))  =>  1
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 5: `isMin = true; commute = false; onRhs = false`

```
llvm: icmp(intr.smin(X, Y), X)  =>  1
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 6: `isMin = true; commute = false; onRhs = true`

```
llvm: icmp(X, intr.smin(X, Y))  =>  1
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
```

```
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 7: `isMin = true; commute = true; onRhs = false`

```
llvm: icmp(intr.smin(Y, X), X)  =>  1
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 8: `isMin = true; commute = true; onRhs = true`

```
llvm: icmp(X, intr.smin(Y, X))  =>  1
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
    | false => Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3703

Source module

Veir.InstSimplify.IS3703

Source

Variant 1: `isMin = false; commute = false; onRhs = false`

```
llvm: icmp(intr.smax(X, Y), X)  =>  0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `isMin = false; commute = false; onRhs = true`

```
llvm: icmp(X, intr.smax(X, Y))  =>  0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `isMin = false; commute = true; onRhs = false`

```
llvm: icmp(intr.smax(Y, X), X)  =>  0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `isMin = false; commute = true; onRhs = true`

```
llvm: icmp(X, intr.smax(Y, X))  =>  0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 5: `isMin = true; commute = false; onRhs = false`

```
llvm: icmp(intr.smin(X, Y), X)  =>  0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 6: `isMin = true; commute = false; onRhs = true`

```
llvm: icmp(X, intr.smin(X, Y))  =>  0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 7: `isMin = true; commute = true; onRhs = false`

```
llvm: icmp(intr.smin(Y, X), X)  =>  0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 8: `isMin = true; commute = true; onRhs = true`

```
llvm: icmp(X, intr.smin(Y, X))  =>  0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
    | false => Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3773

Source module

Veir.InstSimplify.IS3773

Source

Variant 1: `isMin = false; commute = false; onRhs = false`

```
llvm: icmp(intr.umax(X, Y), X)  =>  1
where
  a0 = intr.umax(X, Y) : T0
```

```
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `isMin = false; commute = false; onRhs = true`

```
llvm: icmp(X, intr.umax(X, Y))  =>  1
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `isMin = false; commute = true; onRhs = false`

```
llvm: icmp(intr.umax(Y, X), X)  =>  1
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `isMin = false; commute = true; onRhs = true`

```
llvm: icmp(X, intr.umax(Y, X))  =>  1
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 5: `isMin = true; commute = false; onRhs = false`

```
llvm: icmp(intr.umin(X, Y), X)  =>  1
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 6: `isMin = true; commute = false; onRhs = true`

```
llvm: icmp(X, intr.umin(X, Y))  =>  1
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 7: `isMin = true; commute = true; onRhs = false`

```
llvm: icmp(intr.umin(Y, X), X)  =>  1
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 8: `isMin = true; commute = true; onRhs = true`

```
llvm: icmp(X, intr.umin(Y, X))  =>  1
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 1 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
    | false => Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3775

Source module

Veir.InstSimplify.IS3775

Source

Variant 1: `isMin = false; commute = false; onRhs = false`

```
llvm: icmp(intr.umax(X, Y), X)  =>  0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `isMin = false; commute = false; onRhs = true`

```
llvm: icmp(X, intr.umax(X, Y))  =>  0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `isMin = false; commute = true; onRhs = false`

```
llvm: icmp(intr.umax(Y, X), X)  =>  0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `isMin = false; commute = true; onRhs = true`

```
llvm: icmp(X, intr.umax(Y, X))  =>  0
where
  a0 = intr.umax(Y, X) : T0
```

```
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match false != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 5: `isMin = true; commute = false; onRhs = false`

```
llvm: icmp(intr.umin(X, Y), X)  =>  0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 6: `isMin = true; commute = false; onRhs = true`

```
llvm: icmp(X, intr.umin(X, Y))  =>  0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 7: `isMin = true; commute = true; onRhs = false`

```
llvm: icmp(intr.umin(Y, X), X)  =>  0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(a0, X) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != false with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 8: `isMin = true; commute = true; onRhs = true`

```
llvm: icmp(X, intr.umin(Y, X))  =>  0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  C2 = 0 : T1 [created]
  P2 = properties(C2)
  X : T0
  Y : T0
  require P1.predicate ==
    match true != true with
    | true => Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
    | false => Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3793

Source module

Veir.InstSimplify.IS3793

Source

Variant 1: `maxCommute = false; minCommute = false; reverse = false`

```
llvm: icmp(intr.smax(X, Y), intr.smin(X, Z))  =>  1
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `maxCommute = false; minCommute = false; reverse = true`

```
llvm: icmp(intr.smin(X, Z), intr.smax(X, Y))  =>  1
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `maxCommute = false; minCommute = true; reverse = false`

```
llvm: icmp(intr.smax(X, Y), intr.smin(Z, X))  =>  1
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
```

```
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `maxCommute = false; minCommute = true; reverse = true`

```
llvm: icmp(intr.smin(Z, X), intr.smax(X, Y))  =>  1
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 5: `maxCommute = true; minCommute = false; reverse = false`

```
llvm: icmp(intr.smax(Y, X), intr.smin(X, Z))  =>  1
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 6: `maxCommute = true; minCommute = false; reverse = true`

```
llvm: icmp(intr.smin(X, Z), intr.smax(Y, X))  =>  1
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 7: `maxCommute = true; minCommute = true; reverse = false`

```
llvm: icmp(intr.smax(Y, X), intr.smin(Z, X))  =>  1
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
```

```
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 8: `maxCommute = true; minCommute = true; reverse = true`

```
llvm: icmp(intr.smin(Z, X), intr.smax(Y, X))  =>  1
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.sge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3796

Source module

Veir.InstSimplify.IS3796

Source

Variant 1: `maxCommute = false; minCommute = false; reverse = false`

```
llvm: icmp(intr.smax(X, Y), intr.smin(X, Z))  =>  0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `maxCommute = false; minCommute = false; reverse = true`

```
llvm: icmp(intr.smin(X, Z), intr.smax(X, Y))  =>  0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
```

```
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `maxCommute = false; minCommute = true; reverse = false`

```
llvm: icmp(intr.smax(X, Y), intr.smin(Z, X))  =>  0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `maxCommute = false; minCommute = true; reverse = true`

```
llvm: icmp(intr.smin(Z, X), intr.smax(X, Y))  =>  0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 5: `maxCommute = true; minCommute = false; reverse = false`

```
llvm: icmp(intr.smax(Y, X), intr.smin(X, Z))  =>  0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 6: `maxCommute = true; minCommute = false; reverse = true`

```
llvm: icmp(intr.smin(X, Z), intr.smax(Y, X))  =>  0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.smin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
```

```
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 7: `maxCommute = true; minCommute = true; reverse = false`

```
llvm: icmp(intr.smax(Y, X), intr.smin(Z, X))  =>  0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 8: `maxCommute = true; minCommute = true; reverse = true`

```
llvm: icmp(intr.smin(Z, X), intr.smax(Y, X))  =>  0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.smin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.slt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3802

Source module

Veir.InstSimplify.IS3802

Source

Variant 1: `maxCommute = false; minCommute = false; reverse = false`

```
llvm: icmp(intr.umax(X, Y), intr.umin(X, Z))  =>  1
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `maxCommute = false; minCommute = false; reverse = true`

```
llvm: icmp(intr.umin(X, Z), intr.umax(X, Y))  =>  1
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `maxCommute = false; minCommute = true; reverse = false`

```
llvm: icmp(intr.umax(X, Y), intr.umin(Z, X))  =>  1
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `maxCommute = false; minCommute = true; reverse = true`

```
llvm: icmp(intr.umin(Z, X), intr.umax(X, Y))  =>  1
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 5: `maxCommute = true; minCommute = false; reverse = false`

```
llvm: icmp(intr.umax(Y, X), intr.umin(X, Z))  =>  1
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
```

```
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 6: `maxCommute = true; minCommute = false; reverse = true`

```
llvm: icmp(intr.umin(X, Z), intr.umax(Y, X))  =>  1
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 7: `maxCommute = true; minCommute = true; reverse = false`

```
llvm: icmp(intr.umax(Y, X), intr.umin(Z, X))  =>  1
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 8: `maxCommute = true; minCommute = true; reverse = true`

```
llvm: icmp(intr.umin(Z, X), intr.umax(Y, X))  =>  1
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3805

Source module

Veir.InstSimplify.IS3805

Source

Variant 1: `maxCommute = false; minCommute = false; reverse = false`

```
llvm: icmp(intr.umax(X, Y), intr.umin(X, Z))  =>  0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `maxCommute = false; minCommute = false; reverse = true`

```
llvm: icmp(intr.umin(X, Z), intr.umax(X, Y))  =>  0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `maxCommute = false; minCommute = true; reverse = false`

```
llvm: icmp(intr.umax(X, Y), intr.umin(Z, X))  =>  0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `maxCommute = false; minCommute = true; reverse = true`

```
llvm: icmp(intr.umin(Z, X), intr.umax(X, Y))  =>  0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 5: `maxCommute = true; minCommute = false; reverse = false`

```
llvm: icmp(intr.umax(Y, X), intr.umin(X, Z))  =>  0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 6: `maxCommute = true; minCommute = false; reverse = true`

```
llvm: icmp(intr.umin(X, Z), intr.umax(Y, X))  =>  0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(X, Z) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 7: `maxCommute = true; minCommute = true; reverse = false`

```
llvm: icmp(intr.umax(Y, X), intr.umin(Z, X))  =>  0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  Z : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 8: `maxCommute = true; minCommute = true; reverse = true`

```
llvm: icmp(intr.umin(Z, X), intr.umax(Y, X))  =>  0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = intr.umin(Z, X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
```

```
  Y : T0
  Z : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3846

Source module

Veir.InstSimplify.IS3846

Source

Variant 1: `commute = false; reverse = false`

```
llvm: icmp(intr.uadd.sat(X, Y), (X + Y))  =>  1
where
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X + Y) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `commute = false; reverse = true`

```
llvm: icmp((X + Y), intr.uadd.sat(X, Y))  =>  1
where
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X + Y) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 3: `commute = true; reverse = false`

```
llvm: icmp(intr.uadd.sat(X, Y), (Y + X))  =>  1
where
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y + X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 4: `commute = true; reverse = true`

```
llvm: icmp((Y + X), intr.uadd.sat(X, Y))  =>  1
where
```

```
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y + X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.uge
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3848

Source module

Veir.InstSimplify.IS3848

Source

Variant 1: `commute = false; reverse = false`

```
llvm: icmp(intr.uadd.sat(X, Y), (X + Y))  =>  0
where
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X + Y) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `commute = false; reverse = true`

```
llvm: icmp((X + Y), intr.uadd.sat(X, Y))  =>  0
where
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X + Y) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 3: `commute = true; reverse = false`

```
llvm: icmp(intr.uadd.sat(X, Y), (Y + X))  =>  0
where
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y + X) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ult
```

```
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 4: `commute = true; reverse = true`

```
llvm: icmp((Y + X), intr.uadd.sat(X, Y))  =>  0
where
  a0 = intr.uadd.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (Y + X) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ult
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3856

Source module

Veir.InstSimplify.IS3856

Source

Variant 1: `reverse = false`

```
llvm: icmp(intr.usub.sat(X, Y), (X - Y))  =>  1
where
  a0 = intr.usub.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ule
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Variant 2: `reverse = true`

```
llvm: icmp((X - Y), intr.usub.sat(X, Y))  =>  1
where
  a0 = intr.usub.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 1 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ule
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 1, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3858

Source module

Veir.InstSimplify.IS3858

Source

Variant 1: `reverse = false`

```
llvm: icmp(intr.usub.sat(X, Y), (X - Y))  =>  0
where
  a0 = intr.usub.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ugt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Variant 2: `reverse = true`

```
llvm: icmp((X - Y), intr.usub.sat(X, Y))  =>  0
where
  a0 = intr.usub.sat(X, Y) : T0
  P0 = properties(a0)
  a1 = (X - Y) : T0
  P1 = properties(a1)
  root = icmp(a1, a0) : T1
  P2 = properties(root)
  C3 = 0 : T1 [created]
  P3 = properties(C3)
  X : T0
  Y : T0
  require P2.predicate == Veir.InstSimplify.swapCmpPred Veir.Data.LLVM.IntPred.ugt
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P3 = { value := Veir.LLVMConstantValue.integer { value := Int.ofNat 0, type := { bitwidth := 1 } } }
```

Veir.InstSimplify.IS3891

Source module

Veir.InstSimplify.IS3891

Source

Variant 1: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .integer lhs_value, lhs_value : Int; rhs = .integer rhs_value, rhs_value : Int`

```
llvm: icmp(lhs_value, rhs_value)  =>  (if pred.eval (BitVec.ofInt width lhs_value) (BitVec.ofInt width rhs_value)
= true then 1 else 0)
where
  C0 = lhs_value : T0
  P0 = properties(C0)
  C1 = rhs_value : T0
  P1 = properties(C1)
  root = icmp(C0, C1) : T1
  P2 = properties(root)
  C3 = (if pred.eval (BitVec.ofInt width lhs_value) (BitVec.ofInt width rhs_value) = true then 1 else 0) : T1
[created]
  P3 = properties(C3)
  require P2.predicate == pred
  require Veir.InstSimplify.intConstantGuard P1 rhs_value
  require Veir.InstSimplify.intConstantGuard P0 lhs_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = {
    value :=
      Veir.LLVMConstantValue.integer
        { value := if pred.eval (BitVec.ofInt width lhs_value) (BitVec.ofInt width rhs_value) = true then 1 else
0,
          type := Veir.IntegerType.signless 1 } }
```

Variant 2: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .integer lhs_value, lhs_value : Int; rhs = .zero`

```
llvm: icmp(lhs_value, mlir.zero())  =>  (if pred.eval (BitVec.ofInt width lhs_value) 0#width = true then 1 else
0)
where
  C0 = lhs_value : T0
  P0 = properties(C0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  root = icmp(C0, a1) : T1
  P2 = properties(root)
  C3 = (if pred.eval (BitVec.ofInt width lhs_value) 0#width = true then 1 else 0) : T1 [created]
  P3 = properties(C3)
  require P2.predicate == pred
  require Veir.InstSimplify.intConstantGuard P0 lhs_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = {
    value :=
      Veir.LLVMConstantValue.integer
        { value := if pred.eval (BitVec.ofInt width lhs_value) 0#width = true then 1 else 0,
          type := Veir.IntegerType.signless 1 } }
```

Variant 3: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .integer lhs_value, lhs_value : Int; rhs = .poison`

```
llvm: icmp(lhs_value, a1)  =>  new3
where
  C0 = lhs_value : T0
  P0 = properties(C0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  root = icmp(C0, a1) : T1
  P2 = properties(root)
  new3 = mlir.poison() : T1 [created]
  P3 = properties(new3)
  require P2.predicate == pred
  require Veir.InstSimplify.intConstantGuard P0 lhs_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = PUnit.unit
```

Variant 4: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .zero; rhs = .integer rhs_value, rhs_value : Int`

```
llvm: icmp(mlir.zero(), rhs_value)  =>  (if pred.eval (0#width) (BitVec.ofInt width rhs_value) = true then 1 else
0)
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  C1 = rhs_value : T0
  P1 = properties(C1)
  root = icmp(a0, C1) : T1
  P2 = properties(root)
  C3 = (if pred.eval (0#width) (BitVec.ofInt width rhs_value) = true then 1 else 0) : T1 [created]
  P3 = properties(C3)
  require P2.predicate == pred
  require Veir.InstSimplify.intConstantGuard P1 rhs_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = {
    value :=
      Veir.LLVMConstantValue.integer
        { value := if pred.eval (0#width) (BitVec.ofInt width rhs_value) = true then 1 else 0,
          type := Veir.IntegerType.signless 1 } }
```

Variant 5: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .zero; rhs = .zero`

```
llvm: icmp(a0, a1)  =>  (if pred.eval 0#width 0#width = true then 1 else 0)
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  C3 = (if pred.eval 0#width 0#width = true then 1 else 0) : T1 [created]
  P3 = properties(C3)
  require P2.predicate == pred
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = {
    value :=
      Veir.LLVMConstantValue.integer
        { value := if pred.eval 0#width 0#width = true then 1 else 0, type := Veir.IntegerType.signless 1 } }
```

Variant 6: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .zero; rhs = .poison`

```
llvm: icmp(mlir.zero(), a1)  =>  new3
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  new3 = mlir.poison() : T1 [created]
  P3 = properties(new3)
  require P2.predicate == pred
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = PUnit.unit
```

Variant 7: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .poison; rhs = .integer rhs_value, rhs_value : Int`

```
llvm: icmp(a0, rhs_value)  =>  new3
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  C1 = rhs_value : T0
  P1 = properties(C1)
  root = icmp(a0, C1) : T1
  P2 = properties(root)
  new3 = mlir.poison() : T1 [created]
  P3 = properties(new3)
  require P2.predicate == pred
  require Veir.InstSimplify.intConstantGuard P1 rhs_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = PUnit.unit
```

Variant 8: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .poison; rhs = .zero`

```
llvm: icmp(a0, mlir.zero())  =>  new3
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  new3 = mlir.poison() : T1 [created]
  P3 = properties(new3)
  require P2.predicate == pred
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = PUnit.unit
```

Variant 9: `width : Nat; pred : Veir.Data.LLVM.IntPred; lhs = .poison; rhs = .poison`

```
llvm: icmp(a0, a1)  =>  new3
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  root = icmp(a0, a1) : T1
  P2 = properties(root)
  new3 = mlir.poison() : T1 [created]
  P3 = properties(new3)
  require P2.predicate == pred
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == width
  P3 = PUnit.unit
```

Veir.InstSimplify.IS3907

Source module

Veir.InstSimplify.IS3907

Source

```
llvm: icmp(X, a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = icmp(X, a0) : T1
  P1 = properties(root)
  new2 = mlir.poison() : T1 [created]
  P2 = properties(new2)
  X : T0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS3918

Source module

Veir.InstSimplify.IS3918

Source

Parameters: `pred : Veir.Data.LLVM.IntPred`

```
llvm: icmp(X, X)  =>  (if
    (pred == Veir.Data.LLVM.IntPred.eq || pred == Veir.Data.LLVM.IntPred.uge || pred ==
Veir.Data.LLVM.IntPred.ule ||
          pred == Veir.Data.LLVM.IntPred.sge ||
        pred == Veir.Data.LLVM.IntPred.sle) =
      true then
  1
else 0)
where
  root = icmp(X, X) : T1
  P0 = properties(root)
  C1 = (if
      (pred == Veir.Data.LLVM.IntPred.eq || pred == Veir.Data.LLVM.IntPred.uge || pred ==
Veir.Data.LLVM.IntPred.ule ||
            pred == Veir.Data.LLVM.IntPred.sge ||
          pred == Veir.Data.LLVM.IntPred.sle) =
        true then
    1
  else 0) : T1 [created]
  P1 = properties(C1)
  X : T0
  require P0.predicate == pred
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P1 = {
    value :=
      Veir.LLVMConstantValue.integer
        {
          value :=
            if
                (pred == Veir.Data.LLVM.IntPred.eq || pred == Veir.Data.LLVM.IntPred.uge ||
                        pred == Veir.Data.LLVM.IntPred.ule ||
                      pred == Veir.Data.LLVM.IntPred.sge ||
                    pred == Veir.Data.LLVM.IntPred.sle) =
                  true then
              1
            else 0,
          type := Veir.IntegerType.signless 1 } }
```

Veir.InstSimplify.IS3985

Source module

Veir.InstSimplify.IS3985.pattern

Source

Variant 1: `signed = false; flipped = false`

```
llvm: icmp(zext(X), sext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a0, a1) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif false then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3985.pred false)
    else Veir.InstSimplify.IS3985.pred false
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `signed = false; flipped = true`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3985.pred false)
    else Veir.InstSimplify.IS3985.pred false
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 3: `signed = true; flipped = false`

```
llvm: icmp(zext(X), sext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a0, a1) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif false then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3985.pred true)
    else Veir.InstSimplify.IS3985.pred true
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
```

```
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 4: `signed = true; flipped = true`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3985.pred true)
    else Veir.InstSimplify.IS3985.pred true
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS3987

Source module

Veir.InstSimplify.IS3987.pattern

Source

Variant 1: `signed = false; flipped = false`

```
llvm: icmp(zext(X), sext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a0, a1) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif false then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3987.pred false)
    else Veir.InstSimplify.IS3987.pred false
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `signed = false; flipped = true`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
```

```
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3987.pred false)
    else Veir.InstSimplify.IS3987.pred false
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 3: `signed = true; flipped = false`

```
llvm: icmp(zext(X), sext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a0, a1) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif false then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3987.pred true)
    else Veir.InstSimplify.IS3987.pred true
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 4: `signed = true; flipped = true`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3987.pred true)
    else Veir.InstSimplify.IS3987.pred true
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS4026

Source module

Veir.InstSimplify.IS4026

Source

Parameters: `constant : Int`

```
llvm: icmp(zext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = zext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq || P2.predicate == Veir.Data.LLVM.IntPred.ugt ||
    P2.predicate == Veir.Data.LLVM.IntPred.uge
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard false constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.falseValue.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4031

Source module

Veir.InstSimplify.IS4031

Source

Parameters: `constant : Int`

```
llvm: icmp(zext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = zext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne || P2.predicate == Veir.Data.LLVM.IntPred.ult ||
    P2.predicate == Veir.Data.LLVM.IntPred.ule
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard false constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.trueValue.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4037

Source module

Veir.InstSimplify.IS4037

Source

Parameters: `constant : Int`

```
llvm: icmp(zext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = zext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.sgt || P2.predicate == Veir.Data.LLVM.IntPred.sge
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard false constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.negative.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4042

Source module

Veir.InstSimplify.IS4042

Source

Parameters: `constant : Int`

```
llvm: icmp(zext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = zext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.slt || P2.predicate == Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard false constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.nonnegative.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4064

Source module

Veir.InstSimplify.IS4064.pattern

Source

Builder families: `Veir.InstSimplify.IS3985.pattern`.

Variant 1: `signed = false`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3985.pred false)
    else Veir.InstSimplify.IS3985.pred false
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Variant 2: `signed = true`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3985.pred true)
    else Veir.InstSimplify.IS3985.pred true
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 1 t) }
  | x => none
```

Veir.InstSimplify.IS4066

Source module

Veir.InstSimplify.IS4066.pattern

Source

Builder families: `Veir.InstSimplify.IS3987.pattern`.

Variant 1: `signed = false`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3987.pred false)
    else Veir.InstSimplify.IS3987.pred false
```

```
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Variant 2: `signed = true`

```
llvm: icmp(sext(X), zext(X))  =>  C3
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, a0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate ==
    bif true then Veir.InstSimplify.swapCmpPred (Veir.InstSimplify.IS3987.pred true)
    else Veir.InstSimplify.IS3987.pred true
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T2.val with
  | Veir.Attribute.integerType t => t.bitwidth == 1
  | x => false
  P3 ← match T2.val with
  | Veir.Attribute.integerType t => some { value := Veir.LLVMConstantValue.integer (Veir.IntegerAttr.ofInt 0 t) }
  | x => none
```

Veir.InstSimplify.IS4100

Source module

Veir.InstSimplify.IS4100

Source

Parameters: `constant : Int`

```
llvm: icmp(sext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard true constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.falseValue.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4102

Source module

Veir.InstSimplify.IS4102

Source

Parameters: `constant : Int`

```
llvm: icmp(sext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard true constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.trueValue.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4108

Source module

Veir.InstSimplify.IS4108

Source

Parameters: `constant : Int`

```
llvm: icmp(sext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.sgt || P2.predicate == Veir.Data.LLVM.IntPred.sge
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard true constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.negative.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4113

Source module

Veir.InstSimplify.IS4113

Source

Parameters: `constant : Int`

```
llvm: icmp(sext(X), constant)  =>  C3
where
  C0 = constant : T1
  P0 = properties(C0)
  a1 = sext(X) : T1
  P1 = properties(a1)
  root = icmp(a1, C0) : T2
  P2 = properties(root)
  C3 = mlir.constant() : T2 [created]
  P3 = properties(C3)
  X : T0
  require P2.predicate == Veir.Data.LLVM.IntPred.slt || P2.predicate == Veir.Data.LLVM.IntPred.sle
  require Veir.InstSimplify.intConstantGuard P0 constant
  T2 : Veir.IntegerType, where t = T2: t.bitwidth == 1
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require Veir.InstSimplify.castNonroundtripGuard true constant T0 T1
  P3 ← match T1.val with
  | Veir.Attribute.integerType it =>
    some
      {
        value :=
          Veir.LLVMConstantValue.integer
            { value := Veir.InstSimplify.CastCompareLiteral.nonnegative.value constant it.bitwidth,
              type := Veir.IntegerType.signless 1 } }
  | x => none
```

Veir.InstSimplify.IS4705

Source module

Veir.InstSimplify.IS4705

Source

Variant 1: `mask : Int; complement : Int; trueWhenUnset = false`

```
llvm: select(icmp((X & complement), 0), a4, X)  =>  a4
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = complement : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  a4 = (X & C0) : T0
  P4 = properties(a4)
  a5 = icmp(a3, C2) : T1
  P5 = properties(a5)
  root = select(a5, a4, X) : T0
  P6 = properties(root)
  X : T0
  require P5.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P1 complement
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth complement == ~~~BitVec.ofInt t.bitwidth mask
```

Variant 2: `mask : Int; complement : Int; trueWhenUnset = true`

```
llvm: select(icmp((X & complement), 0), (X & mask), X)  =>  X
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = complement : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  a4 = (X & C0) : T0
```

```
  P4 = properties(a4)
  a5 = icmp(a3, C2) : T1
  P5 = properties(a5)
  root = select(a5, a4, X) : T0
  P6 = properties(root)
  X : T0
  require P5.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P1 complement
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth complement == ~~~BitVec.ofInt t.bitwidth mask
```

Veir.InstSimplify.IS4711

Source module

Veir.InstSimplify.IS4711

Source

Variant 1: `mask : Int; complement : Int; trueWhenUnset = false`

```
llvm: select(icmp((X & complement), 0), X, (X & mask))  =>  X
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = complement : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  a4 = (X & C0) : T0
  P4 = properties(a4)
  a5 = icmp(a3, C2) : T1
  P5 = properties(a5)
  root = select(a5, X, a4) : T0
  P6 = properties(root)
  X : T0
  require P5.predicate == Veir.Data.LLVM.IntPred.ne
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P1 complement
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth complement == ~~~BitVec.ofInt t.bitwidth mask
```

Variant 2: `mask : Int; complement : Int; trueWhenUnset = true`

```
llvm: select(icmp((X & complement), 0), X, a4)  =>  a4
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = complement : T0
  P1 = properties(C1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = (X & C1) : T0
  P3 = properties(a3)
  a4 = (X & C0) : T0
  P4 = properties(a4)
  a5 = icmp(a3, C2) : T1
  P5 = properties(a5)
  root = select(a5, X, a4) : T0
  P6 = properties(root)
  X : T0
  require P5.predicate == Veir.Data.LLVM.IntPred.eq
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P1 complement
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: BitVec.ofInt t.bitwidth complement == ~~~BitVec.ofInt t.bitwidth mask
```

Veir.InstSimplify.IS4721

Source module

Veir.InstSimplify.IS4721

Source

Variant 1: `mask : Int; bit : Nat; trueWhenUnset = false`

```
llvm: select(icmp((X & C0), 0), (X | C0), X)  =>  X
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X | C0) : T0
  P3 = properties(a3)
  a4 = icmp(a2, C1) : T1
  P4 = properties(a4)
  root = select(a4, a3, X) : T0
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require !false || !P3.disjoint
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth mask == BitVec.twoPow
t.bitwidth bit
```

Variant 2: `mask : Int; bit : Nat; trueWhenUnset = true`

```
llvm: select(icmp((X & C0), 0), a3, X)  =>  a3
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X | C0) : T0
  P3 = properties(a3)
  a4 = icmp(a2, C1) : T1
  P4 = properties(a4)
  root = select(a4, a3, X) : T0
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require !true || !P3.disjoint
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth mask == BitVec.twoPow
t.bitwidth bit
```

Veir.InstSimplify.IS4731

Source module

Veir.InstSimplify.IS4731

Source

Variant 1: `mask : Int; bit : Nat; trueWhenUnset = false`

```
llvm: select(icmp((X & C0), 0), X, a3)  =>  a3
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X | C0) : T0
  P3 = properties(a3)
  a4 = icmp(a2, C1) : T1
  P4 = properties(a4)
  root = select(a4, X, a3) : T0
  P5 = properties(root)
```

```
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.ne
  require false || !P3.disjoint
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth mask == BitVec.twoPow
t.bitwidth bit
```

Variant 2: `mask : Int; bit : Nat; trueWhenUnset = true`

```
llvm: select(icmp((X & C0), 0), X, (X | C0))  =>  X
where
  C0 = mask : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = (X & C0) : T0
  P2 = properties(a2)
  a3 = (X | C0) : T0
  P3 = properties(a3)
  a4 = icmp(a2, C1) : T1
  P4 = properties(a4)
  root = select(a4, X, a3) : T0
  P5 = properties(root)
  X : T0
  require P4.predicate == Veir.Data.LLVM.IntPred.eq
  require true || !P3.disjoint
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 mask
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (bit < t.bitwidth) && BitVec.ofInt t.bitwidth mask == BitVec.twoPow
t.bitwidth bit
```

Veir.InstSimplify.IS4785

Source module

Veir.InstSimplify.IS4785

Source

Variant 1: `kind = .smax; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 2: `kind = .smax; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 3: `kind = .smax; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 4: `kind = .smax; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 5: `kind = .smax; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 6: `kind = .smax; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 7: `kind = .smax; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 8: `kind = .smax; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 9: `kind = .smax; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 10: `kind = .smax; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 11: `kind = .smax; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 12: `kind = .smax; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 13: `kind = .smax; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 14: `kind = .smax; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 15: `kind = .smax; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 16: `kind = .smax; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 17: `kind = .smin; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 18: `kind = .smin; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 19: `kind = .smin; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 20: `kind = .smin; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 21: `kind = .smin; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 22: `kind = .smin; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 23: `kind = .smin; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 24: `kind = .smin; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 25: `kind = .smin; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 26: `kind = .smin; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 27: `kind = .smin; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 28: `kind = .smin; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 29: `kind = .smin; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 30: `kind = .smin; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 31: `kind = .smin; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 32: `kind = .smin; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 33: `kind = .umax; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 34: `kind = .umax; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 35: `kind = .umax; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 36: `kind = .umax; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 37: `kind = .umax; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 38: `kind = .umax; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 39: `kind = .umax; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 40: `kind = .umax; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 41: `kind = .umax; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 42: `kind = .umax; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 43: `kind = .umax; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 44: `kind = .umax; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 45: `kind = .umax; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 46: `kind = .umax; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 47: `kind = .umax; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 48: `kind = .umax; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 49: `kind = .umin; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 50: `kind = .umin; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 51: `kind = .umin; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 52: `kind = .umin; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 53: `kind = .umin; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 54: `kind = .umin; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 55: `kind = .umin; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 56: `kind = .umin; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 57: `kind = .umin; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred false
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 58: `kind = .umin; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 59: `kind = .umin; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred false
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 60: `kind = .umin; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 61: `kind = .umin; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 62: `kind = .umin; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred true
false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 63: `kind = .umin; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.strictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 64: `kind = .umin; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.nonStrictPred true
true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS4793

Source module

Veir.InstSimplify.IS4793

Source

Variant 1: `kind = .smax; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
```

```
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 2: `kind = .smax; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 3: `kind = .smax; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 4: `kind = .smax; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 5: `kind = .smax; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 6: `kind = .smax; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
```

```
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 7: `kind = .smax; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 8: `kind = .smax; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 9: `kind = .smin; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 10: `kind = .smin; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 11: `kind = .smin; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 12: `kind = .smin; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 13: `kind = .smin; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 14: `kind = .smin; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 15: `kind = .smin; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
```

```
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 16: `kind = .smin; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 17: `kind = .umax; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 18: `kind = .umax; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 19: `kind = .umax; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 20: `kind = .umax; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
```

```
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 21: `kind = .umax; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 22: `kind = .umax; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 23: `kind = .umax; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 24: `kind = .umax; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 25: `kind = .umin; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
```

```
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 26: `kind = .umin; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 27: `kind = .umin; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 28: `kind = .umin; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 29: `kind = .umin; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, a0)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 30: `kind = .umin; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), a0, X)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 31: `kind = .umin; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, a0)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 32: `kind = .umin; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), a0, X)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.eq true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS4797

Source module

Veir.InstSimplify.IS4797

Source

Variant 1: `kind = .smax; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.smax(X, Y))  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 2: `kind = .smax; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.smax(X, Y), X)  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
```

```
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 3: `kind = .smax; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.smax(X, Y))  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 4: `kind = .smax; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.smax(X, Y), X)  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 5: `kind = .smax; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.smax(Y, X))  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 6: `kind = .smax; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.smax(Y, X), X)  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 7: `kind = .smax; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.smax(Y, X))  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 8: `kind = .smax; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.smax(Y, X), X)  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 9: `kind = .smin; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.smin(X, Y))  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 10: `kind = .smin; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.smin(X, Y), X)  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 11: `kind = .smin; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.smin(X, Y))  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
```

```
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 12: `kind = .smin; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.smin(X, Y), X)  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 13: `kind = .smin; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.smin(Y, X))  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 14: `kind = .smin; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.smin(Y, X), X)  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 15: `kind = .smin; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.smin(Y, X))  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 16: `kind = .smin; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.smin(Y, X), X)  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
```

```
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 17: `kind = .umax; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.umax(X, Y))  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 18: `kind = .umax; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.umax(X, Y), X)  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 19: `kind = .umax; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.umax(X, Y))  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 20: `kind = .umax; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.umax(X, Y), X)  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 21: `kind = .umax; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.umax(Y, X))  =>  X
where
  a0 = intr.umax(Y, X) : T0
```

```
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 22: `kind = .umax; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.umax(Y, X), X)  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 23: `kind = .umax; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.umax(Y, X))  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 24: `kind = .umax; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.umax(Y, X), X)  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 25: `kind = .umin; commute = false; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.umin(X, Y))  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 26: `kind = .umin; commute = false; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.umin(X, Y), X)  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 27: `kind = .umin; commute = false; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.umin(X, Y))  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 28: `kind = .umin; commute = false; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.umin(X, Y), X)  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 29: `kind = .umin; commute = true; swapCmp = false; swapArms = false`

```
llvm: select(icmp(X, Y), X, intr.umin(Y, X))  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 30: `kind = .umin; commute = true; swapCmp = false; swapArms = true`

```
llvm: select(icmp(X, Y), intr.umin(Y, X), X)  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne false true
```

```
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 31: `kind = .umin; commute = true; swapCmp = true; swapArms = false`

```
llvm: select(icmp(Y, X), X, intr.umin(Y, X))  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 32: `kind = .umin; commute = true; swapCmp = true; swapArms = true`

```
llvm: select(icmp(Y, X), intr.umin(Y, X), X)  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.Data.LLVM.IntPred.ne true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS4805

Source module

Veir.InstSimplify.IS4805

Source

Variant 1: `kind = .smax; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.smax(X, Y))  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 2: `kind = .smax; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.smax(X, Y))  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 3: `kind = .smax; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.smax(X, Y), X)  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 4: `kind = .smax; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.smax(X, Y), X)  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 5: `kind = .smax; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.smax(X, Y))  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 6: `kind = .smax; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.smax(X, Y))  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 7: `kind = .smax; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.smax(X, Y), X)  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
```

```
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 8: `kind = .smax; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.smax(X, Y), X)  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 9: `kind = .smax; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.smax(Y, X))  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 10: `kind = .smax; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.smax(Y, X))  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 11: `kind = .smax; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.smax(Y, X), X)  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 12: `kind = .smax; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.smax(Y, X), X)  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 13: `kind = .smax; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.smax(Y, X))  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 14: `kind = .smax; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.smax(Y, X))  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 15: `kind = .smax; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.smax(Y, X), X)  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smax.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 16: `kind = .smax; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.smax(Y, X), X)  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
```

```
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smax.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 17: `kind = .smin; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.smin(X, Y))  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 18: `kind = .smin; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.smin(X, Y))  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 19: `kind = .smin; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.smin(X, Y), X)  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 20: `kind = .smin; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.smin(X, Y), X)  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 21: `kind = .smin; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.smin(X, Y))  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 22: `kind = .smin; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.smin(X, Y))  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 23: `kind = .smin; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.smin(X, Y), X)  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 24: `kind = .smin; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.smin(X, Y), X)  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 25: `kind = .smin; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.smin(Y, X))  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
```

```
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 26: `kind = .smin; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.smin(Y, X))  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 27: `kind = .smin; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.smin(Y, X), X)  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 28: `kind = .smin; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.smin(Y, X), X)  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 29: `kind = .smin; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.smin(Y, X))  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 30: `kind = .smin; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.smin(Y, X))  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 31: `kind = .smin; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.smin(Y, X), X)  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.smin.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 32: `kind = .smin; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.smin(Y, X), X)  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.smin.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 33: `kind = .umax; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.umax(X, Y))  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 34: `kind = .umax; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.umax(X, Y))  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
```

```
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 35: `kind = .umax; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.umax(X, Y), X)  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 36: `kind = .umax; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.umax(X, Y), X)  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 37: `kind = .umax; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.umax(X, Y))  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 38: `kind = .umax; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.umax(X, Y))  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 39: `kind = .umax; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.umax(X, Y), X)  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 40: `kind = .umax; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.umax(X, Y), X)  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 41: `kind = .umax; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.umax(Y, X))  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 42: `kind = .umax; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.umax(Y, X))  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 43: `kind = .umax; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.umax(Y, X), X)  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
```

```
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 44: `kind = .umax; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.umax(Y, X), X)  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 45: `kind = .umax; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.umax(Y, X))  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 46: `kind = .umax; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.umax(Y, X))  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 47: `kind = .umax; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.umax(Y, X), X)  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umax.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 48: `kind = .umax; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.umax(Y, X), X)  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umax.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 49: `kind = .umin; commute = false; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.umin(X, Y))  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 50: `kind = .umin; commute = false; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.umin(X, Y))  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 51: `kind = .umin; commute = false; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.umin(X, Y), X)  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 52: `kind = .umin; commute = false; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.umin(X, Y), X)  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
```

```
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 53: `kind = .umin; commute = false; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.umin(X, Y))  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 54: `kind = .umin; commute = false; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.umin(X, Y))  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 55: `kind = .umin; commute = false; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.umin(X, Y), X)  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 56: `kind = .umin; commute = false; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.umin(X, Y), X)  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 57: `kind = .umin; commute = true; swapCmp = false; swapArms = false; inclusive = false`

```
llvm: select(icmp(X, Y), X, intr.umin(Y, X))  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 58: `kind = .umin; commute = true; swapCmp = false; swapArms = false; inclusive = true`

```
llvm: select(icmp(X, Y), X, intr.umin(Y, X))  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred false false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 59: `kind = .umin; commute = true; swapCmp = false; swapArms = true; inclusive = false`

```
llvm: select(icmp(X, Y), intr.umin(Y, X), X)  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 60: `kind = .umin; commute = true; swapCmp = false; swapArms = true; inclusive = true`

```
llvm: select(icmp(X, Y), intr.umin(Y, X), X)  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(X, Y) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred false true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 61: `kind = .umin; commute = true; swapCmp = true; swapArms = false; inclusive = false`

```
llvm: select(icmp(Y, X), X, intr.umin(Y, X))  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
```

```
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 62: `kind = .umin; commute = true; swapCmp = true; swapArms = false; inclusive = true`

```
llvm: select(icmp(Y, X), X, intr.umin(Y, X))  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, X, a0) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred true false
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 63: `kind = .umin; commute = true; swapCmp = true; swapArms = true; inclusive = false`

```
llvm: select(icmp(Y, X), intr.umin(Y, X), X)  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred Veir.InstSimplify.MinMaxKind.umin.inverseStrictPred
true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Variant 64: `kind = .umin; commute = true; swapCmp = true; swapArms = true; inclusive = true`

```
llvm: select(icmp(Y, X), intr.umin(Y, X), X)  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  a1 = icmp(Y, X) : T1
  P1 = properties(a1)
  root = select(a1, a0, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.InstSimplify.canonicalCmpPred
Veir.InstSimplify.MinMaxKind.umin.inverseNonStrictPred true true
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS4898

Source module

Veir.InstSimplify.IS4898

Source

Variant 1: `left = false; swapcmp = false`

```
llvm: select(icmp(Z, 0), intr.fshr(Y, X, Z), X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Z, C0) : T1
  P1 = properties(a1)
```

```
  a2 = intr.fshr(Y, X, Z) : T0
  P2 = properties(a2)
  root = select(a1, a2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  Z : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 2: `left = false; swapcmp = true`

```
llvm: select(icmp(0, Z), intr.fshr(Y, X, Z), X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(C0, Z) : T1
  P1 = properties(a1)
  a2 = intr.fshr(Y, X, Z) : T0
  P2 = properties(a2)
  root = select(a1, a2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  Z : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 3: `left = true; swapcmp = false`

```
llvm: select(icmp(Z, 0), intr.fshl(X, Y, Z), X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Z, C0) : T1
  P1 = properties(a1)
  a2 = intr.fshl(X, Y, Z) : T0
  P2 = properties(a2)
  root = select(a1, a2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  Z : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 4: `left = true; swapcmp = true`

```
llvm: select(icmp(0, Z), intr.fshl(X, Y, Z), X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(C0, Z) : T1
  P1 = properties(a1)
  a2 = intr.fshl(X, Y, Z) : T0
  P2 = properties(a2)
  root = select(a1, a2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  Z : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS4912

Source module

Veir.InstSimplify.IS4912

Source

Variant 1: `left = false; swapcmp = false`

```
llvm: select(icmp(Y, 0), X, a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, C0) : T1
  P1 = properties(a1)
  a2 = intr.fshr(X, X, Y) : T0
  P2 = properties(a2)
  root = select(a1, X, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 2: `left = false; swapcmp = true`

```
llvm: select(icmp(0, Y), X, a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(C0, Y) : T1
  P1 = properties(a1)
  a2 = intr.fshr(X, X, Y) : T0
  P2 = properties(a2)
  root = select(a1, X, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 3: `left = true; swapcmp = false`

```
llvm: select(icmp(Y, 0), X, a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(Y, C0) : T1
  P1 = properties(a1)
  a2 = intr.fshl(X, X, Y) : T0
  P2 = properties(a2)
  root = select(a1, X, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 4: `left = true; swapcmp = true`

```
llvm: select(icmp(0, Y), X, a2)  =>  a2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(C0, Y) : T1
  P1 = properties(a1)
  a2 = intr.fshl(X, X, Y) : T0
  P2 = properties(a2)
  root = select(a1, X, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS4923

Source module

Veir.InstSimplify.IS4923

Source

Variant 1: `k : Nat; left = false; swapcmp = false; swapand = false`

```
llvm: select(icmp((Y & (Int.ofNat (2 ^ k - 1))), 0), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (Y & C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a1, C2) : T1
  P3 = properties(a3)
  a4 = intr.fshr(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 2: `k : Nat; left = false; swapcmp = false; swapand = true`

```
llvm: select(icmp(((Int.ofNat (2 ^ k - 1)) & Y), 0), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (C0 & Y) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a1, C2) : T1
  P3 = properties(a3)
  a4 = intr.fshr(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 3: `k : Nat; left = false; swapcmp = true; swapand = false`

```
llvm: select(icmp(0, (Y & (Int.ofNat (2 ^ k - 1)))), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (Y & C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C2, a1) : T1
  P3 = properties(a3)
  a4 = intr.fshr(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
```

```
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 4: `k : Nat; left = false; swapcmp = true; swapand = true`

```
llvm: select(icmp(0, ((Int.ofNat (2 ^ k - 1)) & Y)), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (C0 & Y) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C2, a1) : T1
  P3 = properties(a3)
  a4 = intr.fshr(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 5: `k : Nat; left = true; swapcmp = false; swapand = false`

```
llvm: select(icmp((Y & (Int.ofNat (2 ^ k - 1))), 0), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (Y & C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a1, C2) : T1
  P3 = properties(a3)
  a4 = intr.fshl(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 6: `k : Nat; left = true; swapcmp = false; swapand = true`

```
llvm: select(icmp(((Int.ofNat (2 ^ k - 1)) & Y), 0), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (C0 & Y) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a1, C2) : T1
  P3 = properties(a3)
  a4 = intr.fshl(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
```

```
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 7: `k : Nat; left = true; swapcmp = true; swapand = false`

```
llvm: select(icmp(0, (Y & (Int.ofNat (2 ^ k - 1)))), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (Y & C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C2, a1) : T1
  P3 = properties(a3)
  a4 = intr.fshl(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 8: `k : Nat; left = true; swapcmp = true; swapand = true`

```
llvm: select(icmp(0, ((Int.ofNat (2 ^ k - 1)) & Y)), X, a4)  =>  a4
where
  C0 = (Int.ofNat (2 ^ k - 1)) : T0
  P0 = properties(C0)
  a1 = (C0 & Y) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C2, a1) : T1
  P3 = properties(a3)
  a4 = intr.fshl(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 2 ^ k
  require Veir.InstSimplify.bitConstantGuard T0 P0 (Int.ofNat (2 ^ k - 1))
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Veir.InstSimplify.IS4927

Source module

Veir.InstSimplify.IS4927

Source

Variant 1: `w : Nat; left = false; swapcmp = false`

```
llvm: select(icmp(urem(Y, (↑w)), 0), X, a4)  =>  a4
where
  C0 = (↑w) : T0
  P0 = properties(C0)
  a1 = urem(Y, C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a1, C2) : T1
  P3 = properties(a3)
  a4 = intr.fshr(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
```

```
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == w
  require Veir.InstSimplify.bitConstantGuard T0 P0 ↑w
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 2: `w : Nat; left = false; swapcmp = true`

```
llvm: select(icmp(0, urem(Y, (↑w))), X, a4)  =>  a4
where
  C0 = (↑w) : T0
  P0 = properties(C0)
  a1 = urem(Y, C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C2, a1) : T1
  P3 = properties(a3)
  a4 = intr.fshr(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == w
  require Veir.InstSimplify.bitConstantGuard T0 P0 ↑w
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 3: `w : Nat; left = true; swapcmp = false`

```
llvm: select(icmp(urem(Y, (↑w)), 0), X, a4)  =>  a4
where
  C0 = (↑w) : T0
  P0 = properties(C0)
  a1 = urem(Y, C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(a1, C2) : T1
  P3 = properties(a3)
  a4 = intr.fshl(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == w
  require Veir.InstSimplify.bitConstantGuard T0 P0 ↑w
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Variant 4: `w : Nat; left = true; swapcmp = true`

```
llvm: select(icmp(0, urem(Y, (↑w))), X, a4)  =>  a4
where
  C0 = (↑w) : T0
  P0 = properties(C0)
  a1 = urem(Y, C0) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  a3 = icmp(C2, a1) : T1
  P3 = properties(a3)
  a4 = intr.fshl(X, X, Y) : T0
  P4 = properties(a4)
  root = select(a3, X, a4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require P3.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == w
```

```
  require Veir.InstSimplify.bitConstantGuard T0 P0 ↑w
  require Veir.InstSimplify.bitConstantGuard T0 P2 0
```

Veir.InstSimplify.IS4934

Source module

Veir.InstSimplify.IS4934

Source

Variant 1: `swapcmp = false`

```
llvm: select(icmp(X, C0), a2, a5)  =>  a5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, C0) : T1
  P1 = properties(a1)
  a2 = intr.abs(X) : T0
  P2 = properties(a2)
  a3 = intr.abs(X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  a5 = (C4 - a3) : T0
  P5 = properties(a5)
  root = select(a1, a2, a5) : T0
  P6 = properties(root)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (0 < t.bitwidth)
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require Veir.InstSimplify.bitConstantGuard T0 P4 0
```

Variant 2: `swapcmp = true`

```
llvm: select(icmp(C0, X), a2, a5)  =>  a5
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(C0, X) : T1
  P1 = properties(a1)
  a2 = intr.abs(X) : T0
  P2 = properties(a2)
  a3 = intr.abs(X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  a5 = (C4 - a3) : T0
  P5 = properties(a5)
  root = select(a1, a2, a5) : T0
  P6 = properties(root)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (0 < t.bitwidth)
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require Veir.InstSimplify.bitConstantGuard T0 P4 0
```

Veir.InstSimplify.IS4938

Source module

Veir.InstSimplify.IS4938

Source

Variant 1: `swapcmp = false`

```
llvm: select(icmp(X, C0), (C4 - a2), a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(X, C0) : T1
```

```
  P1 = properties(a1)
  a2 = intr.abs(X) : T0
  P2 = properties(a2)
  a3 = intr.abs(X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  a5 = (C4 - a2) : T0
  P5 = properties(a5)
  root = select(a1, a5, a3) : T0
  P6 = properties(root)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (0 < t.bitwidth)
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require Veir.InstSimplify.bitConstantGuard T0 P4 0
```

Variant 2: `swapcmp = true`

```
llvm: select(icmp(C0, X), (C4 - a2), a3)  =>  a3
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = icmp(C0, X) : T1
  P1 = properties(a1)
  a2 = intr.abs(X) : T0
  P2 = properties(a2)
  a3 = intr.abs(X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  a5 = (C4 - a2) : T0
  P5 = properties(a5)
  root = select(a1, a5, a3) : T0
  P6 = properties(root)
  X : T0
  require P1.predicate == Veir.Data.LLVM.IntPred.eq
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: decide (0 < t.bitwidth)
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require Veir.InstSimplify.bitConstantGuard T0 P4 0
```

Veir.InstSimplify.IS5122

Source module

Veir.InstSimplify.IS5122

Source

Variant 1: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(0, trueVal_value, C2)  =>  C4
where
  C0 = 0 : T1
  P0 = properties(C0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(C0, C1, C2) : T0
  P3 = properties(root)
  C4 = falseVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := falseVal_value, type := resultType } }
```

Variant 2: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .zero`

```
llvm: select(0, trueVal_value, a2)  =>  new4
where
  C0 = 0 : T1
  P0 = properties(C0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(C0, C1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 3: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .poison`

```
llvm: select(0, trueVal_value, a2)  =>  new4
where
  C0 = 0 : T1
  P0 = properties(C0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(C0, C1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 4: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .zero; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(0, mlir.zero(), C2)  =>  C4
where
  C0 = 0 : T1
  P0 = properties(C0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(C0, a1, C2) : T0
  P3 = properties(root)
  C4 = falseVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := falseVal_value, type := resultType } }
```

Variant 5: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .zero; falseVal = .zero`

```
llvm: select(0, a1, a2)  =>  new4
where
  C0 = 0 : T1
  P0 = properties(C0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 6: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .zero; falseVal = .poison`

```
llvm: select(0, mlir.zero(), a2)  =>  new4
where
  C0 = 0 : T1
  P0 = properties(C0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 7: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .poison; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(0, mlir.poison(), C2)  =>  C4
where
  C0 = 0 : T1
  P0 = properties(C0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(C0, a1, C2) : T0
  P3 = properties(root)
  C4 = falseVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := falseVal_value, type := resultType } }
```

Variant 8: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .poison; falseVal = .zero`

```
llvm: select(0, mlir.poison(), a2)  =>  new4
where
  C0 = 0 : T1
  P0 = properties(C0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 9: `resultType : Veir.IntegerType; cond = .integer 0; trueVal = .poison; falseVal = .poison`

```
llvm: select(0, a1, a2)  =>  new4
where
  C0 = 0 : T1
  P0 = properties(C0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
```

```
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 10: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(1, C1, falseVal_value)  =>  C4
where
  C0 = 1 : T1
  P0 = properties(C0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(C0, C1, C2) : T0
  P3 = properties(root)
  C4 = trueVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := trueVal_value, type := resultType } }
```

Variant 11: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .zero`

```
llvm: select(1, C1, mlir.zero())  =>  C4
where
  C0 = 1 : T1
  P0 = properties(C0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(C0, C1, a2) : T0
  P3 = properties(root)
  C4 = trueVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := trueVal_value, type := resultType } }
```

Variant 12: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .poison`

```
llvm: select(1, C1, mlir.poison())  =>  C4
where
  C0 = 1 : T1
  P0 = properties(C0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(C0, C1, a2) : T0
  P3 = properties(root)
  C4 = trueVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := trueVal_value, type := resultType } }
```

Variant 13: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .zero; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(1, a1, falseVal_value)  =>  new4
where
  C0 = 1 : T1
```

```
  P0 = properties(C0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(C0, a1, C2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 14: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .zero; falseVal = .zero`

```
llvm: select(1, a1, a2)  =>  new4
where
  C0 = 1 : T1
  P0 = properties(C0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 15: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .zero; falseVal = .poison`

```
llvm: select(1, a1, mlir.poison())  =>  new4
where
  C0 = 1 : T1
  P0 = properties(C0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 16: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .poison; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(1, a1, falseVal_value)  =>  new4
where
  C0 = 1 : T1
  P0 = properties(C0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(C0, a1, C2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 17: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .poison; falseVal = .zero`

```
llvm: select(1, a1, mlir.zero())  =>  new4
where
  C0 = 1 : T1
  P0 = properties(C0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 18: `resultType : Veir.IntegerType; cond = .integer 1; trueVal = .poison; falseVal = .poison`

```
llvm: select(1, a1, a2)  =>  new4
where
  C0 = 1 : T1
  P0 = properties(C0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(C0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 19: `resultType : Veir.IntegerType; cond = .zero; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(mlir.zero(), trueVal_value, C2)  =>  C4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(a0, C1, C2) : T0
  P3 = properties(root)
  C4 = falseVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := falseVal_value, type := resultType } }
```

Variant 20: `resultType : Veir.IntegerType; cond = .zero; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .zero`

```
llvm: select(a0, trueVal_value, a2)  =>  new4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(a0, C1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 21: `resultType : Veir.IntegerType; cond = .zero; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .poison`

```
llvm: select(mlir.zero(), trueVal_value, a2)  =>  new4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(a0, C1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 22: `resultType : Veir.IntegerType; cond = .zero; trueVal = .zero; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(a0, a1, C2)  =>  C4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(a0, a1, C2) : T0
  P3 = properties(root)
  C4 = falseVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := falseVal_value, type := resultType } }
```

Variant 23: `resultType : Veir.IntegerType; cond = .zero; trueVal = .zero; falseVal = .zero`

```
llvm: select(a0, a1, a2)  =>  new4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 24: `resultType : Veir.IntegerType; cond = .zero; trueVal = .zero; falseVal = .poison`

```
llvm: select(a0, a1, a2)  =>  new4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 25: `resultType : Veir.IntegerType; cond = .zero; trueVal = .poison; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(mlir.zero(), mlir.poison(), C2)  =>  C4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(a0, a1, C2) : T0
  P3 = properties(root)
  C4 = falseVal_value : T0 [created]
  P4 = properties(C4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = { value := Veir.LLVMConstantValue.integer { value := falseVal_value, type := resultType } }
```

Variant 26: `resultType : Veir.IntegerType; cond = .zero; trueVal = .poison; falseVal = .zero`

```
llvm: select(a0, mlir.poison(), a2)  =>  new4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.zero() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 27: `resultType : Veir.IntegerType; cond = .zero; trueVal = .poison; falseVal = .poison`

```
llvm: select(mlir.zero(), a1, a2)  =>  new4
where
  a0 = mlir.zero() : T1
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 28: `resultType : Veir.IntegerType; cond = .poison; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(a0, trueVal_value, falseVal_value)  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(a0, C1, C2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 29: `resultType : Veir.IntegerType; cond = .poison; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .zero`

```
llvm: select(a0, trueVal_value, mlir.zero())  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(a0, C1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 30: `resultType : Veir.IntegerType; cond = .poison; trueVal = .integer trueVal_value, trueVal_value : Int; falseVal = .poison`

```
llvm: select(a0, trueVal_value, a2)  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  C1 = trueVal_value : T0
  P1 = properties(C1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(a0, C1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P1 trueVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 31: `resultType : Veir.IntegerType; cond = .poison; trueVal = .zero; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(a0, mlir.zero(), falseVal_value)  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(a0, a1, C2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 32: `resultType : Veir.IntegerType; cond = .poison; trueVal = .zero; falseVal = .zero`

```
llvm: select(a0, a1, a2)  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
```

```
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 33: `resultType : Veir.IntegerType; cond = .poison; trueVal = .zero; falseVal = .poison`

```
llvm: select(a0, mlir.zero(), a2)  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  a1 = mlir.zero() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 34: `resultType : Veir.IntegerType; cond = .poison; trueVal = .poison; falseVal = .integer falseVal_value, falseVal_value : Int`

```
llvm: select(a0, a1, falseVal_value)  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  C2 = falseVal_value : T0
  P2 = properties(C2)
  root = select(a0, a1, C2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  require Veir.InstSimplify.intConstantGuard P2 falseVal_value
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 35: `resultType : Veir.IntegerType; cond = .poison; trueVal = .poison; falseVal = .zero`

```
llvm: select(a0, a1, mlir.zero())  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.zero() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Variant 36: `resultType : Veir.IntegerType; cond = .poison; trueVal = .poison; falseVal = .poison`

```
llvm: select(a0, a1, a2)  =>  new4
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  a1 = mlir.poison() : T0
  P1 = properties(a1)
  a2 = mlir.poison() : T0
  P2 = properties(a2)
  root = select(a0, a1, a2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType, where t = T0: t == resultType
  P4 = PUnit.unit
```

Veir.InstSimplify.IS5126

Source module

Veir.InstSimplify.IS5126

Source

```
llvm: select(a0, X, Y)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = select(a0, X, Y) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5137

Source module

Veir.InstSimplify.IS5137

Source

```
llvm: select(1, X, Y)  =>  X
where
  C0 = 1 : T1
  P0 = properties(C0)
  root = select(C0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5139

Source module

Veir.InstSimplify.IS5139

Source

```
llvm: select(0, X, Y)  =>  Y
where
  C0 = 0 : T1
  P0 = properties(C0)
  root = select(C0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5150

Source module

Veir.InstSimplify.IS5150

Source

```
llvm: select(X, 1, 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
```

```
  root = select(X, C0, C1) : T0
  P2 = properties(root)
  X : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5154

Source module

Veir.InstSimplify.IS5154

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select((X & Y), X, Y)  =>  Y
where
  a0 = (X & Y) : T0
  P0 = properties(a0)
  root = select(a0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select(select(X, Y, 0), X, Y)  =>  Y
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(X, Y, C0) : T0
  P1 = properties(a1)
  root = select(a1, X, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select((Y & X), X, Y)  =>  Y
where
  a0 = (Y & X) : T0
  P0 = properties(a0)
  root = select(a0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select(select(Y, X, 0), X, Y)  =>  Y
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(Y, X, C0) : T0
  P1 = properties(a1)
  root = select(a1, X, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5158

Source module

Veir.InstSimplify.IS5158

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select((X | Y), X, Y)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  root = select(a0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select(select(X, 1, Y), X, Y)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  root = select(a1, X, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select((Y | X), X, Y)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  root = select(a0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select(select(Y, 1, X), X, Y)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  root = select(a1, X, Y) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5163

Source module

Veir.InstSimplify.IS5163

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select((X | Y), C1, X)  =>  C1
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = select(a0, C1, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select(select(X, 1, Y), C2, X)  =>  C2
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  root = select(a1, C2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select((Y | X), C1, X)  =>  C1
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = select(a0, C1, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select(select(Y, 1, X), C2, X)  =>  C2
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  root = select(a1, C2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5169

Source module

Veir.InstSimplify.IS5169

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select(((X | Y) ^ 1), X, C3)  =>  C3
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (a0 ^ C1) : T0
  P2 = properties(a2)
  C3 = 0 : T0
  P3 = properties(C3)
  root = select(a2, X, C3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P3 0
```

```
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select((select(X, C0, Y) ^ C2), X, C4)  =>  C4
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (a1 ^ C2) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a3, X, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select(((Y | X) ^ 1), X, C3)  =>  C3
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (a0 ^ C1) : T0
  P2 = properties(a2)
  C3 = 0 : T0
  P3 = properties(C3)
  root = select(a2, X, C3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P3 0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select((select(Y, C0, X) ^ C2), X, C4)  =>  C4
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (a1 ^ C2) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a3, X, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5172

Source module

Veir.InstSimplify.IS5172

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select(X, ((X | Y) ^ 1), C3)  =>  C3
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (a0 ^ C1) : T0
  P2 = properties(a2)
  C3 = 0 : T0
  P3 = properties(C3)
  root = select(X, a2, C3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P3 0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select(X, (select(X, C0, Y) ^ C2), C4)  =>  C4
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (a1 ^ C2) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(X, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select(X, ((Y | X) ^ 1), C3)  =>  C3
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (a0 ^ C1) : T0
  P2 = properties(a2)
  C3 = 0 : T0
  P3 = properties(C3)
  root = select(X, a2, C3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P3 0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select(X, (select(Y, C0, X) ^ C2), C4)  =>  C4
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (a1 ^ C2) : T0
  P3 = properties(a3)
  C4 = 0 : T0
```

```
  P4 = properties(C4)
  root = select(X, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5176

Source module

Veir.InstSimplify.IS5176

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select((X | Y), X, 0)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = select(a0, X, C1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select(select(X, 1, Y), X, 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  root = select(a1, X, C2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select((Y | X), X, 0)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = select(a0, X, C1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select(select(Y, 1, X), X, 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
```

```
  root = select(a1, X, C2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5179

Source module

Veir.InstSimplify.IS5179

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select(X, (Y | X), 0)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = select(X, a0, C1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select(X, select(Y, 1, X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  root = select(X, a1, C2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select(X, (X | Y), 0)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = select(X, a0, C1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select(X, select(X, 1, Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 0 : T0
  P2 = properties(C2)
  root = select(X, a1, C2) : T0
  P3 = properties(root)
```

```
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5185

Source module

Veir.InstSimplify.IS5185

Source

Variant 1: `swapn4 = false; logicaln4 = false; swapn5 = false; logicaln5 = false`

```
llvm: select((X | (Y ^ 1)), (X | Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a2, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn4 = false; logicaln4 = false; swapn5 = false; logicaln5 = true`

```
llvm: select((X | (Y ^ C0)), select(X, C3, Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
  a4 = select(X, C3, Y) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a2, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn4 = false; logicaln4 = false; swapn5 = true; logicaln5 = false`

```
llvm: select((X | (Y ^ 1)), (Y | X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a2, a3, C4) : T0
```

```
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn4 = false; logicaln4 = false; swapn5 = true; logicaln5 = true`

```
llvm: select((X | (Y ^ C0)), select(Y, C3, X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (X | a1) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
  a4 = select(Y, C3, X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a2, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 5: `swapn4 = false; logicaln4 = true; swapn5 = false; logicaln5 = false`

```
llvm: select(select(X, C2, (Y ^ C0)), (X | Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(X, C2, a1) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a3, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 6: `swapn4 = false; logicaln4 = true; swapn5 = false; logicaln5 = true`

```
llvm: select(select(X, C2, (Y ^ C0)), select(X, C4, Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(X, C2, a1) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
  a5 = select(X, C4, Y) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a3, a5, C6) : T0
```

```
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 7: `swapn4 = false; logicaln4 = true; swapn5 = true; logicaln5 = false`

```
llvm: select(select(X, C2, (Y ^ C0)), (Y | X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(X, C2, a1) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a3, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 8: `swapn4 = false; logicaln4 = true; swapn5 = true; logicaln5 = true`

```
llvm: select(select(X, C2, (Y ^ C0)), select(Y, C4, X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(X, C2, a1) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
  a5 = select(Y, C4, X) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a3, a5, C6) : T0
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 9: `swapn4 = true; logicaln4 = false; swapn5 = false; logicaln5 = false`

```
llvm: select(((Y ^ 1) | X), (X | Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (X | Y) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
```

```
  root = select(a2, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 10: `swapn4 = true; logicaln4 = false; swapn5 = false; logicaln5 = true`

```
llvm: select(((Y ^ C0) | X), select(X, C3, Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
  a4 = select(X, C3, Y) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a2, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 11: `swapn4 = true; logicaln4 = false; swapn5 = true; logicaln5 = false`

```
llvm: select(((Y ^ 1) | X), (Y | X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  a3 = (Y | X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a2, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 12: `swapn4 = true; logicaln4 = false; swapn5 = true; logicaln5 = true`

```
llvm: select(((Y ^ C0) | X), select(Y, C3, X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  a2 = (a1 | X) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
  a4 = select(Y, C3, X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a2, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
```

```
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 13: `swapn4 = true; logicaln4 = true; swapn5 = false; logicaln5 = false`

```
llvm: select(select((Y ^ C0), C2, X), (X | Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(a1, C2, X) : T0
  P3 = properties(a3)
  a4 = (X | Y) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a3, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 14: `swapn4 = true; logicaln4 = true; swapn5 = false; logicaln5 = true`

```
llvm: select(select((Y ^ C0), C2, X), select(X, C4, Y), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(a1, C2, X) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
  a5 = select(X, C4, Y) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a3, a5, C6) : T0
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 15: `swapn4 = true; logicaln4 = true; swapn5 = true; logicaln5 = false`

```
llvm: select(select((Y ^ C0), C2, X), (Y | X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(a1, C2, X) : T0
  P3 = properties(a3)
  a4 = (Y | X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a3, a4, C5) : T0
  P6 = properties(root)
  X : T0
```

```
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 16: `swapn4 = true; logicaln4 = true; swapn5 = true; logicaln5 = true`

```
llvm: select(select((Y ^ C0), C2, X), select(Y, C4, X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y ^ C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = select(a1, C2, X) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
  a5 = select(Y, C4, X) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a3, a5, C6) : T0
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5188

Source module

Veir.InstSimplify.IS5188

Source

Variant 1: `swapn2 = false; logicaln2 = false; swapn5 = false; logicaln5 = false`

```
llvm: select((X | Y), (X | (Y ^ 1)), 0)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  a3 = (X | a2) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a0, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = false; swapn5 = false; logicaln5 = true`

```
llvm: select((X | Y), select(X, C3, (Y ^ C1)), 0)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
```

```
  a4 = select(X, C3, a2) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a0, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = false; logicaln2 = false; swapn5 = true; logicaln5 = false`

```
llvm: select((X | Y), ((Y ^ 1) | X), 0)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  a3 = (a2 | X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a0, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = false; logicaln2 = false; swapn5 = true; logicaln5 = true`

```
llvm: select((X | Y), select((Y ^ C1), C3, X), 0)  =>  X
where
  a0 = (X | Y) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
  a4 = select(a2, C3, X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a0, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 5: `swapn2 = false; logicaln2 = true; swapn5 = false; logicaln5 = false`

```
llvm: select(select(X, C0, Y), (X | (Y ^ C2)), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  a4 = (X | a3) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
```

```
  root = select(a1, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 6: `swapn2 = false; logicaln2 = true; swapn5 = false; logicaln5 = true`

```
llvm: select(select(X, C0, Y), select(X, C4, (Y ^ C2)), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
  a5 = select(X, C4, a3) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a1, a5, C6) : T0
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 7: `swapn2 = false; logicaln2 = true; swapn5 = true; logicaln5 = false`

```
llvm: select(select(X, C0, Y), ((Y ^ C2) | X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  a4 = (a3 | X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a1, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 8: `swapn2 = false; logicaln2 = true; swapn5 = true; logicaln5 = true`

```
llvm: select(select(X, C0, Y), select((Y ^ C2), C4, X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(X, C0, Y) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
```

```
  a5 = select(a3, C4, X) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a1, a5, C6) : T0
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 9: `swapn2 = true; logicaln2 = false; swapn5 = false; logicaln5 = false`

```
llvm: select((Y | X), (X | (Y ^ 1)), 0)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  a3 = (X | a2) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a0, a3, C4) : T0
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 10: `swapn2 = true; logicaln2 = false; swapn5 = false; logicaln5 = true`

```
llvm: select((Y | X), select(X, C3, (Y ^ C1)), 0)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
  a4 = select(X, C3, a2) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a0, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 11: `swapn2 = true; logicaln2 = false; swapn5 = true; logicaln5 = false`

```
llvm: select((Y | X), ((Y ^ 1) | X), 0)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  a3 = (a2 | X) : T0
  P3 = properties(a3)
  C4 = 0 : T0
  P4 = properties(C4)
  root = select(a0, a3, C4) : T0
```

```
  P5 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P4 0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 12: `swapn2 = true; logicaln2 = false; swapn5 = true; logicaln5 = true`

```
llvm: select((Y | X), select((Y ^ C1), C3, X), 0)  =>  X
where
  a0 = (Y | X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (Y ^ C1) : T0
  P2 = properties(a2)
  C3 = 1 : T0
  P3 = properties(C3)
  a4 = select(a2, C3, X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a0, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P3 1
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 13: `swapn2 = true; logicaln2 = true; swapn5 = false; logicaln5 = false`

```
llvm: select(select(Y, C0, X), (X | (Y ^ C2)), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  a4 = (X | a3) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a1, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 14: `swapn2 = true; logicaln2 = true; swapn5 = false; logicaln5 = true`

```
llvm: select(select(Y, C0, X), select(X, C4, (Y ^ C2)), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
  a5 = select(X, C4, a3) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a1, a5, C6) : T0
```

```
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 15: `swapn2 = true; logicaln2 = true; swapn5 = true; logicaln5 = false`

```
llvm: select(select(Y, C0, X), ((Y ^ C2) | X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  a4 = (a3 | X) : T0
  P4 = properties(a4)
  C5 = 0 : T0
  P5 = properties(C5)
  root = select(a1, a4, C5) : T0
  P6 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P5 0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 16: `swapn2 = true; logicaln2 = true; swapn5 = true; logicaln5 = true`

```
llvm: select(select(Y, C0, X), select((Y ^ C2), C4, X), 0)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = select(Y, C0, X) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (Y ^ C2) : T0
  P3 = properties(a3)
  C4 = 1 : T0
  P4 = properties(C4)
  a5 = select(a3, C4, X) : T0
  P5 = properties(a5)
  C6 = 0 : T0
  P6 = properties(C6)
  root = select(a1, a5, C6) : T0
  P7 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P6 0
  require Veir.InstSimplify.intConstantGuard P4 1
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5195

Source module

Veir.InstSimplify.IS5195

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select(((X & Y) ^ C1), C1, X)  =>  C1
where
  a0 = (X & Y) : T0
  P0 = properties(a0)
```

```
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (a0 ^ C1) : T0
  P2 = properties(a2)
  root = select(a2, C1, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select((select(X, Y, 0) ^ C2), C2, X)  =>  C2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(X, Y, C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (a1 ^ C2) : T0
  P3 = properties(a3)
  root = select(a3, C2, X) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select(((Y & X) ^ C1), C1, X)  =>  C1
where
  a0 = (Y & X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  a2 = (a0 ^ C1) : T0
  P2 = properties(a2)
  root = select(a2, C1, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select((select(Y, X, 0) ^ C2), C2, X)  =>  C2
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(Y, X, C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  a3 = (a1 ^ C2) : T0
  P3 = properties(a3)
  root = select(a3, C2, X) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5198

Source module

Veir.InstSimplify.IS5198

Source

Variant 1: `swapn3 = false; logicaln3 = false`

```
llvm: select(X, C0, ((X & Y) ^ C0))  =>  C0
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  root = select(X, C0, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn3 = false; logicaln3 = true`

```
llvm: select(X, C0, (select(X, Y, 0) ^ C0))  =>  C0
where
  C0 = 1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = select(X, Y, C1) : T0
  P2 = properties(a2)
  a3 = (a2 ^ C0) : T0
  P3 = properties(a3)
  root = select(X, C0, a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn3 = true; logicaln3 = false`

```
llvm: select(X, C0, ((Y & X) ^ C0))  =>  C0
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  a2 = (a1 ^ C0) : T0
  P2 = properties(a2)
  root = select(X, C0, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn3 = true; logicaln3 = true`

```
llvm: select(X, C0, (select(Y, X, 0) ^ C0))  =>  C0
where
  C0 = 1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = select(Y, X, C1) : T0
  P2 = properties(a2)
  a3 = (a2 ^ C0) : T0
  P3 = properties(a3)
  root = select(X, C0, a3) : T0
  P4 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5202

Source module

Veir.InstSimplify.IS5202

Source

Variant 1: `swapn2 = false; logicaln2 = false`

```
llvm: select((X & Y), 1, X)  =>  X
where
  a0 = (X & Y) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  root = select(a0, C1, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn2 = false; logicaln2 = true`

```
llvm: select(select(X, Y, 0), 1, X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(X, Y, C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  root = select(a1, C2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn2 = true; logicaln2 = false`

```
llvm: select((Y & X), 1, X)  =>  X
where
  a0 = (Y & X) : T0
  P0 = properties(a0)
  C1 = 1 : T0
  P1 = properties(C1)
  root = select(a0, C1, X) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn2 = true; logicaln2 = true`

```
llvm: select(select(Y, X, 0), 1, X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  a1 = select(Y, X, C0) : T0
  P1 = properties(a1)
  C2 = 1 : T0
  P2 = properties(C2)
  root = select(a1, C2, X) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P2 1
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5205

Source module

Veir.InstSimplify.IS5205

Source

Variant 1: `swapn3 = false; logicaln3 = false`

```
llvm: select(X, 1, (X & Y))  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (X & Y) : T0
  P1 = properties(a1)
  root = select(X, C0, a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 2: `swapn3 = false; logicaln3 = true`

```
llvm: select(X, 1, select(X, Y, 0))  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = select(X, Y, C1) : T0
  P2 = properties(a2)
  root = select(X, C0, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 3: `swapn3 = true; logicaln3 = false`

```
llvm: select(X, 1, (Y & X))  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = (Y & X) : T0
  P1 = properties(a1)
  root = select(X, C0, a1) : T0
  P2 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Variant 4: `swapn3 = true; logicaln3 = true`

```
llvm: select(X, 1, select(Y, X, 0))  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  a2 = select(Y, X, C1) : T0
  P2 = properties(a2)
  root = select(X, C0, a2) : T0
  P3 = properties(root)
  X : T0
  Y : T0
  require Veir.InstSimplify.intConstantGuard P1 0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5211

Source module

Veir.InstSimplify.IS5211

Source

```
llvm: select(X, Y, Y)  =>  Y
where
```

```
  root = select(X, Y, Y) : T0
  P0 = properties(root)
  X : T1
  Y : T0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5216

Source module

Veir.InstSimplify.IS5216

Source

```
llvm: select(X, X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = select(X, X, C0) : T0
  P1 = properties(root)
  X : T0
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5219

Source module

Veir.InstSimplify.IS5219

Source

```
llvm: select(X, X, C0)  =>  C0
where
  C0 = 1 : T0
  P0 = properties(C0)
  root = select(X, X, C0) : T0
  P1 = properties(root)
  X : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5224

Source module

Veir.InstSimplify.IS5224

Source

```
llvm: select(X, 1, X)  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  root = select(X, C0, X) : T0
  P1 = properties(root)
  X : T0
  require Veir.InstSimplify.intConstantGuard P0 1
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5227

Source module

Veir.InstSimplify.IS5227

Source

```
llvm: select(X, C0, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = select(X, C0, X) : T0
```

```
  P1 = properties(root)
  X : T0
  require Veir.InstSimplify.intConstantGuard P0 0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 1
```

Veir.InstSimplify.IS5237a

Source module

Veir.InstSimplify.IS5237a

Source

```
llvm: select(X, mlir.poison(), Y)  =>  Y
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = select(X, a0, Y) : T0
  P1 = properties(root)
  X : T1
  Y : T0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5242a

Source module

Veir.InstSimplify.IS5242a

Source

```
llvm: select(X, Y, mlir.poison())  =>  Y
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = select(X, Y, a0) : T0
  P1 = properties(root)
  X : T1
  Y : T0
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 1
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5317

Source module

Veir.InstSimplify.IS5317

Source

```
llvm: getelementptr(X)  =>  X
where
  root = getelementptr(X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.LLVM.PointerType
  require P0.rawConstantIndices.values == #[]
```

Veir.InstSimplify.IS5335

Source module

Veir.InstSimplify.IS5335_static

Source

```
llvm: getelementptr(X)  =>  X
where
  root = getelementptr(X) : T0
  P0 = properties(root)
  X : T0
  require !P0.rawConstantIndices.values.isEmpty && P0.rawConstantIndices.values.all fun x => x == 0
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5335

Source

Builder families: `Veir.InstSimplify.gepAllZeroFamily`.

Variant 1: `payloads = []; representative length 0, a b : Int`

```
llvm: getelementptr(X)  =>  X
where
  root = getelementptr(X) : T0
  P0 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P0.rawConstantIndices.values).size == 0 &&
    P0.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  T0 : Veir.LLVM.PointerType
```

Variant 2: `payloads = [a]; representative length 1, a b : Int`

```
llvm: getelementptr(X, a)  =>  X
where
  C0 = a : T1
  P0 = properties(C0)
  root = getelementptr(X, C0) : T0
  P1 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 &&
    P1.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: BitVec.ofInt t.bitwidth a == 0
  T0 : Veir.LLVM.PointerType
```

Variant 3: `payloads = [a, b]; representative length 2, a b : Int`

```
llvm: getelementptr(X, a, b)  =>  X
where
  C0 = a : T1
  P0 = properties(C0)
  C1 = b : T2
  P1 = properties(C1)
  root = getelementptr(X, C0, C1) : T0
  P2 = properties(root)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P2.rawConstantIndices.values).size == 1 + 1 &&
    P2.rawConstantIndices.values.all fun index => index == 0 || index == -2147483648
  require Veir.InstSimplify.intConstantGuard P1 b
  T2 : Veir.IntegerType, where t = T2: BitVec.ofInt t.bitwidth b == 0
  require Veir.InstSimplify.intConstantGuard P0 a
  T1 : Veir.IntegerType, where t = T1: BitVec.ofInt t.bitwidth a == 0
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5340

Source module

Veir.InstSimplify.IS5340_static

Source

```
llvm: getelementptr(a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  require P1.rawConstantIndices.values.all fun x => x != -2147483648
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5340

Source

Builder families: `Veir.InstSimplify.gepPoisonBaseFamily`.

Variant 1: `cause = .base 0`

```
llvm: getelementptr(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0) : T0
  P1 = properties(root)
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0
  T0 : Veir.LLVM.PointerType
```

Variant 2: `cause = .base 1`

```
llvm: getelementptr(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X) : T0
  P1 = properties(root)
  X : T1
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Variant 3: `cause = .base 2`

```
llvm: getelementptr(a0, X, Y)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X, Y) : T0
  P1 = properties(root)
  X : T1
  Y : T2
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Variant 4: `cause = .base 3`

```
llvm: getelementptr(a0, X, Y, Z)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = getelementptr(a0, X, Y, Z) : T0
  P1 = properties(root)
  X : T1
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 3
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Variant 5: `cause = .index 0 0`

```
llvm: getelementptr(X, a0)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0 + 1 + 0
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 6: `cause = .index 0 1`

```
llvm: getelementptr(X, a0, Y)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
```

```
  root = getelementptr(X, a0, Y) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0 + 1 + 1
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 7: `cause = .index 0 2`

```
llvm: getelementptr(X, a0, Y, Z)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, a0, Y, Z) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 0 + 1 + 2
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 8: `cause = .index 1 0`

```
llvm: getelementptr(X, Y, a0)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 + 1 + 0
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 9: `cause = .index 1 1`

```
llvm: getelementptr(X, Y, a0, Z)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, a0, Z) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 + 1 + 1
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 10: `cause = .index 1 2`

```
llvm: getelementptr(X, Y, a0, Z, X3)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
```

```
  root = getelementptr(X, Y, a0, Z, X3) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  X3 : T4
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 1 + 1 + 2
  T4 : Veir.IntegerType
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 11: `cause = .index 2 0`

```
llvm: getelementptr(X, Y, Z, a0)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, Z, a0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2 + 1 + 0
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 12: `cause = .index 2 1`

```
llvm: getelementptr(X, Y, Z, a0, X3)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, Z, a0, X3) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  X3 : T4
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2 + 1 + 1
  T4 : Veir.IntegerType
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Variant 13: `cause = .index 2 2`

```
llvm: getelementptr(X, Y, Z, a0, X3, X4)  =>  new2
where
  a0 = mlir.poison() : T1
  P0 = properties(a0)
  root = getelementptr(X, Y, Z, a0, X3, X4) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  X : T0
  Y : T2
  Z : T3
  X3 : T4
  X4 : T5
  require (Array.filter (fun x => x == -2147483648) P1.rawConstantIndices.values).size == 2 + 1 + 2
  T5 : Veir.IntegerType
  T4 : Veir.IntegerType
```

```
  T3 : Veir.IntegerType
  T2 : Veir.IntegerType
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5359

Source module

Veir.InstSimplify.IS5359_static

Source

Parameters: `index : Int`

```
llvm: getelementptr(X)  =>  X
where
  root = getelementptr(X) : T0
  P0 = properties(root)
  X : T0
  require P0.rawConstantIndices.values == #[index] && index != -2147483648 &&
    Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val == some 0
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5359_dynamic

Source

```
llvm: getelementptr(X, Y)  =>  X
where
  root = getelementptr(X, Y) : T0
  P0 = properties(root)
  X : T0
  Y : T1
  require P0.rawConstantIndices.values == #[-2147483648] && Veir.DataLayout.riscv64.getTypeAllocSize
P0.elem_type.val == some 0
  T1 : Veir.IntegerType
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5681

Source module

Veir.InstSimplify.IS5681.trunc

Source

```
llvm: trunc(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = trunc(C0) : T1
  P1 = properties(root)
  C2 = mlir.constant() : T1 [created]
  P2 = properties(C2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (n.bitwidth < w.bitwidth)
  | x, x_1 => false
  P2 ← Veir.InstSimplify.IS5681.fold_trunc T0 T1 P0
```

Veir.InstSimplify.IS5681.zext

Source

```
llvm: zext(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = zext(C0) : T1
  P1 = properties(root)
  C2 = mlir.constant() : T1 [created]
  P2 = properties(C2)
```

```
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
  P2 ← Veir.InstSimplify.IS5681.fold_zext T0 T1 P0
```

Veir.InstSimplify.IS5681.sext

Source

```
llvm: sext(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = sext(C0) : T1
  P1 = properties(root)
  C2 = mlir.constant() : T1 [created]
  P2 = properties(C2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
  P2 ← Veir.InstSimplify.IS5681.fold_sext T0 T1 P0
```

Veir.InstSimplify.IS5681.bitcast

Source

```
llvm: bitcast(C0)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = bitcast(C0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5681.ptrtoint_null

Source

```
llvm: ptrtoint(a0)  =>  new2
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  root = ptrtoint(a0) : T1
  P1 = properties(root)
  new2 = mlir.zero() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 64
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.trunc_zero

Source

```
llvm: trunc(a0)  =>  new2
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  root = trunc(a0) : T1
  P1 = properties(root)
  new2 = mlir.zero() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (n.bitwidth < w.bitwidth)
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.trunc_poison

Source

```
llvm: trunc(a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = trunc(a0) : T1
  P1 = properties(root)
  new2 = mlir.poison() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (n.bitwidth < w.bitwidth)
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.zext_zero

Source

```
llvm: zext(a0)  =>  new2
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  root = zext(a0) : T1
  P1 = properties(root)
  new2 = mlir.zero() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.zext_poison

Source

```
llvm: zext(a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = zext(a0) : T1
  P1 = properties(root)
  new2 = mlir.poison() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.sext_zero

Source

```
llvm: sext(a0)  =>  new2
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  root = sext(a0) : T1
  P1 = properties(root)
  new2 = mlir.zero() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.sext_poison

Source

```
llvm: sext(a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = sext(a0) : T1
  P1 = properties(root)
  new2 = mlir.poison() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.bitcast_int_zero

Source

```
llvm: bitcast(a0)  =>  a0
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  root = bitcast(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5681.bitcast_int_poison

Source

```
llvm: bitcast(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = bitcast(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5681.bitcast_ptr_zero

Source

```
llvm: bitcast(a0)  =>  a0
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  root = bitcast(a0) : T0
  P1 = properties(root)
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5681.bitcast_ptr_poison

Source

```
llvm: bitcast(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = bitcast(a0) : T0
  P1 = properties(root)
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5681.inttoptr_poison

Source

```
llvm: inttoptr(a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = inttoptr(a0) : T1
```

```
  P1 = properties(root)
  new2 = mlir.poison() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.LLVM.PointerType
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 64
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.ptrtoint_poison

Source

```
llvm: ptrtoint(a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = ptrtoint(a0) : T1
  P1 = properties(root)
  new2 = mlir.poison() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType, where t = T1: t.bitwidth == 64
  T0 : Veir.LLVM.PointerType
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.bitcast_integer

Source

```
llvm: bitcast(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = bitcast(C0) : T1
  P1 = properties(root)
  C2 = mlir.constant() : T1 [created]
  P2 = properties(C2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType a, Veir.Attribute.integerType b => a.bitwidth == b.bitwidth
  | x, x_1 => false
  P2 ← Veir.InstSimplify.IS5681.fold_bitcast_integer T1 P0
```

Veir.InstSimplify.IS5681.bitcast_integer_zero

Source

```
llvm: bitcast(a0)  =>  new2
where
  a0 = mlir.zero() : T0
  P0 = properties(a0)
  root = bitcast(a0) : T1
  P1 = properties(root)
  new2 = mlir.zero() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType a, Veir.Attribute.integerType b => a.bitwidth == b.bitwidth
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5681.bitcast_integer_poison

Source

```
llvm: bitcast(a0)  =>  new2
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = bitcast(a0) : T1
  P1 = properties(root)
  new2 = mlir.poison() : T1 [created]
  P2 = properties(new2)
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
```

```
  require match T0.val, T1.val with
  | Veir.Attribute.integerType a, Veir.Attribute.integerType b => a.bitwidth == b.bitwidth
  | x, x_1 => false
  P2 = PUnit.unit
```

Veir.InstSimplify.IS5693

Source module

Veir.InstSimplify.IS5693_extend_trunc

Source

Variant 1: `signed = false`

```
llvm: trunc(zext(X))  =>  X
where
  a0 = zext(X) : T1
  P0 = properties(a0)
  root = trunc(a0) : T0
  P1 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
```

Variant 2: `signed = true`

```
llvm: trunc(sext(X))  =>  X
where
  a0 = sext(X) : T1
  P0 = properties(a0)
  root = trunc(a0) : T0
  P1 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (w.bitwidth < n.bitwidth)
  | x, x_1 => false
```

Veir.InstSimplify.IS5693_bitcast_int

Source

```
llvm: bitcast(bitcast(X))  =>  X
where
  a0 = bitcast(X) : T1
  P0 = properties(a0)
  root = bitcast(a0) : T0
  P1 = properties(root)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType a, Veir.Attribute.integerType b => a.bitwidth == b.bitwidth
  | x, x_1 => false
```

Veir.InstSimplify.IS5693_bitcast_ptr

Source

```
llvm: bitcast(bitcast(X))  =>  X
where
  a0 = bitcast(X) : T0
  P0 = properties(a0)
  root = bitcast(a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5693_bitcast_byte

Source

```
llvm: bitcast(bitcast(X))  =>  X
where
  a0 = bitcast(X) : T0
  P0 = properties(a0)
  root = bitcast(a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.LLVM.ByteType
```

Veir.InstSimplify.IS5700

Source module

Veir.InstSimplify.IS5700_int

Source

```
llvm: bitcast(X)  =>  X
where
  root = bitcast(X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS5700_byte

Source

```
llvm: bitcast(X)  =>  X
where
  root = bitcast(X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.LLVM.ByteType
```

Veir.InstSimplify.IS5700_ptr

Source

```
llvm: bitcast(X)  =>  X
where
  root = bitcast(X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.LLVM.PointerType
```

Veir.InstSimplify.IS5722

Source module

Veir.InstSimplify.IS5722

Source

Variant 1: `signed = false`

```
llvm: zext(trunc(X))  =>  X
where
  a0 = trunc(X) : T1
  P0 = properties(a0)
  root = zext(a0) : T0
  P1 = properties(root)
  X : T0
  require P0.nuw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (n.bitwidth < w.bitwidth)
  | x, x_1 => false
```

Variant 2: `signed = true`

```
llvm: sext(trunc(X))  =>  X
where
  a0 = trunc(X) : T1
```

```
  P0 = properties(a0)
  root = sext(a0) : T0
  P1 = properties(root)
  X : T0
  require P0.nsw
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  require match T0.val, T1.val with
  | Veir.Attribute.integerType w, Veir.Attribute.integerType n => decide (n.bitwidth < w.bitwidth)
  | x, x_1 => false
```

Veir.InstSimplify.IS6666

Source module

Veir.InstSimplify.IS6666

Source

```
llvm: intr.bswap(intr.bswap(X))  =>  X
where
  a0 = intr.bswap(X) : T0
  P0 = properties(a0)
  root = intr.bswap(a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 16 || t.bitwidth == 32 || t.bitwidth == 64
```

Veir.InstSimplify.IS6671

Source module

Veir.InstSimplify.IS6671

Source

```
llvm: intr.bitreverse(intr.bitreverse(X))  =>  X
where
  a0 = intr.bitreverse(X) : T0
  P0 = properties(a0)
  root = intr.bitreverse(a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS6760

Source module

Veir.InstSimplify.IS6760

Source

Variant 1: `kind = .smax; si = false; so = false`

```
llvm: intr.smax(a0, X)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = intr.smax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 2: `kind = .smax; si = false; so = true`

```
llvm: intr.smax(X, a0)  =>  a0
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = intr.smax(X, a0) : T0
  P1 = properties(root)
  X : T0
```

```
  Y : T0
  T0 : Veir.IntegerType
```

Variant 3: `kind = .smax; si = true; so = false`

```
llvm: intr.smax(a0, X)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = intr.smax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 4: `kind = .smax; si = true; so = true`

```
llvm: intr.smax(X, a0)  =>  a0
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = intr.smax(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 5: `kind = .smin; si = false; so = false`

```
llvm: intr.smin(a0, X)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = intr.smin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 6: `kind = .smin; si = false; so = true`

```
llvm: intr.smin(X, a0)  =>  a0
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = intr.smin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 7: `kind = .smin; si = true; so = false`

```
llvm: intr.smin(a0, X)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = intr.smin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 8: `kind = .smin; si = true; so = true`

```
llvm: intr.smin(X, a0)  =>  a0
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = intr.smin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 9: `kind = .umax; si = false; so = false`

```
llvm: intr.umax(a0, X)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  root = intr.umax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 10: `kind = .umax; si = false; so = true`

```
llvm: intr.umax(X, a0)  =>  a0
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  root = intr.umax(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 11: `kind = .umax; si = true; so = false`

```
llvm: intr.umax(a0, X)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  root = intr.umax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 12: `kind = .umax; si = true; so = true`

```
llvm: intr.umax(X, a0)  =>  a0
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  root = intr.umax(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 13: `kind = .umin; si = false; so = false`

```
llvm: intr.umin(a0, X)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = intr.umin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 14: `kind = .umin; si = false; so = true`

```
llvm: intr.umin(X, a0)  =>  a0
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = intr.umin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 15: `kind = .umin; si = true; so = false`

```
llvm: intr.umin(a0, X)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
```

```
  root = intr.umin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 16: `kind = .umin; si = true; so = true`

```
llvm: intr.umin(X, a0)  =>  a0
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  root = intr.umin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS6763

Source module

Veir.InstSimplify.IS6763

Source

Variant 1: `kind = .smax; si = false; so = false`

```
llvm: intr.smax(intr.smin(X, Y), X)  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = intr.smax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 2: `kind = .smax; si = false; so = true`

```
llvm: intr.smax(X, intr.smin(X, Y))  =>  X
where
  a0 = intr.smin(X, Y) : T0
  P0 = properties(a0)
  root = intr.smax(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 3: `kind = .smax; si = true; so = false`

```
llvm: intr.smax(intr.smin(Y, X), X)  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = intr.smax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 4: `kind = .smax; si = true; so = true`

```
llvm: intr.smax(X, intr.smin(Y, X))  =>  X
where
  a0 = intr.smin(Y, X) : T0
  P0 = properties(a0)
  root = intr.smax(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 5: `kind = .smin; si = false; so = false`

```
llvm: intr.smin(intr.smax(X, Y), X)  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = intr.smin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 6: `kind = .smin; si = false; so = true`

```
llvm: intr.smin(X, intr.smax(X, Y))  =>  X
where
  a0 = intr.smax(X, Y) : T0
  P0 = properties(a0)
  root = intr.smin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 7: `kind = .smin; si = true; so = false`

```
llvm: intr.smin(intr.smax(Y, X), X)  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = intr.smin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 8: `kind = .smin; si = true; so = true`

```
llvm: intr.smin(X, intr.smax(Y, X))  =>  X
where
  a0 = intr.smax(Y, X) : T0
  P0 = properties(a0)
  root = intr.smin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 9: `kind = .umax; si = false; so = false`

```
llvm: intr.umax(intr.umin(X, Y), X)  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = intr.umax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 10: `kind = .umax; si = false; so = true`

```
llvm: intr.umax(X, intr.umin(X, Y))  =>  X
where
  a0 = intr.umin(X, Y) : T0
  P0 = properties(a0)
  root = intr.umax(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 11: `kind = .umax; si = true; so = false`

```
llvm: intr.umax(intr.umin(Y, X), X)  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
```

```
  root = intr.umax(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 12: `kind = .umax; si = true; so = true`

```
llvm: intr.umax(X, intr.umin(Y, X))  =>  X
where
  a0 = intr.umin(Y, X) : T0
  P0 = properties(a0)
  root = intr.umax(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 13: `kind = .umin; si = false; so = false`

```
llvm: intr.umin(intr.umax(X, Y), X)  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  root = intr.umin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 14: `kind = .umin; si = false; so = true`

```
llvm: intr.umin(X, intr.umax(X, Y))  =>  X
where
  a0 = intr.umax(X, Y) : T0
  P0 = properties(a0)
  root = intr.umin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 15: `kind = .umin; si = true; so = false`

```
llvm: intr.umin(intr.umax(Y, X), X)  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  root = intr.umin(a0, X) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Variant 16: `kind = .umin; si = true; so = true`

```
llvm: intr.umin(X, intr.umax(Y, X))  =>  X
where
  a0 = intr.umax(Y, X) : T0
  P0 = properties(a0)
  root = intr.umin(X, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS6990

Source module

Veir.InstSimplify.IS6990

Source

```
llvm: intr.abs(a0)  =>  a0
where
  a0 = intr.abs(X) : T0
  P0 = properties(a0)
  root = intr.abs(a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS6996

Source module

Veir.InstSimplify.IS6996

Source

```
llvm: intr.cttz(shl(1, X))  =>  X
where
  C0 = 1 : T0
  P0 = properties(C0)
  a1 = shl(C0, X) : T0
  P1 = properties(a1)
  root = intr.cttz(a1) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 1
```

Veir.InstSimplify.IS7002

Source module

Veir.InstSimplify.IS7002

Source

Parameters: `n : Int`

```
llvm: intr.ctlz(lshr(n, X))  =>  X
where
  C0 = n : T0
  P0 = properties(C0)
  a1 = lshr(C0, X) : T0
  P1 = properties(a1)
  root = intr.ctlz(a1) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType t => (BitVec.ofInt t.bitwidth n).msb
  | x => false
```

Veir.InstSimplify.IS7004

Source module

Veir.InstSimplify.IS7004

Source

Parameters: `n : Int`

```
llvm: intr.ctlz(ashr(n, X))  =>  mlir.zero()
where
  C0 = n : T0
  P0 = properties(C0)
  a1 = ashr(C0, X) : T0
  P1 = properties(a1)
  root = intr.ctlz(a1) : T0
  P2 = properties(root)
  new3 = mlir.zero() : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType
```

```
  require Veir.InstSimplify.bitConstantGuard T0 P0 n
  require match T0.val with
  | Veir.Attribute.integerType t => (BitVec.ofInt t.bitwidth n).msb
  | x => false
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7066

Source module

Veir.InstSimplify.IS7066

Source

Variant 1: `kind = .smax`

```
llvm: intr.smax(X, X)  =>  X
where
  root = intr.smax(X, X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Variant 2: `kind = .smin`

```
llvm: intr.smin(X, X)  =>  X
where
  root = intr.smin(X, X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Variant 3: `kind = .umax`

```
llvm: intr.umax(X, X)  =>  X
where
  root = intr.umax(X, X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Variant 4: `kind = .umin`

```
llvm: intr.umin(X, X)  =>  X
where
  root = intr.umin(X, X) : T0
  P0 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7082

Source module

Veir.InstSimplify.IS7082

Source

Variant 1: `kind = .smax; swap = false`

```
llvm: intr.smax(X, C0)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smax(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smax.saturation t.bitwidth
  | x, x_1 => false
```

Variant 2: `kind = .smax; swap = true`

```
llvm: intr.smax(C0, X)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smax(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smax.saturation t.bitwidth
  | x, x_1 => false
```

Variant 3: `kind = .smin; swap = false`

```
llvm: intr.smin(X, C0)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smin(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smin.saturation t.bitwidth
  | x, x_1 => false
```

Variant 4: `kind = .smin; swap = true`

```
llvm: intr.smin(C0, X)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smin(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smin.saturation t.bitwidth
  | x, x_1 => false
```

Variant 5: `kind = .umax; swap = false`

```
llvm: intr.umax(X, C0)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umax(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umax.saturation t.bitwidth
  | x, x_1 => false
```

Variant 6: `kind = .umax; swap = true`

```
llvm: intr.umax(C0, X)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umax(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umax.saturation t.bitwidth
  | x, x_1 => false
```

Variant 7: `kind = .umin; swap = false`

```
llvm: intr.umin(X, C0)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umin(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umin.saturation t.bitwidth
  | x, x_1 => false
```

Variant 8: `kind = .umin; swap = true`

```
llvm: intr.umin(C0, X)  =>  C0
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umin(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umin.saturation t.bitwidth
  | x, x_1 => false
```

Veir.InstSimplify.IS7089

Source module

Veir.InstSimplify.IS7089

Source

Variant 1: `kind = .smax; swap = false`

```
llvm: intr.smax(X, C0)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smax(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smax.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Variant 2: `kind = .smax; swap = true`

```
llvm: intr.smax(C0, X)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smax(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smax.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Variant 3: `kind = .smin; swap = false`

```
llvm: intr.smin(X, C0)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smin(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

```
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smin.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Variant 4: `kind = .smin; swap = true`

```
llvm: intr.smin(C0, X)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.smin(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.smin.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Variant 5: `kind = .umax; swap = false`

```
llvm: intr.umax(X, C0)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umax(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umax.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Variant 6: `kind = .umax; swap = true`

```
llvm: intr.umax(C0, X)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umax(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umax.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Variant 7: `kind = .umin; swap = false`

```
llvm: intr.umin(X, C0)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umin(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umin.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Variant 8: `kind = .umin; swap = true`

```
llvm: intr.umin(C0, X)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.umin(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
```

```
  | Veir.Attribute.integerType t, Veir.LLVMConstantValue.integer a =>
    BitVec.ofInt t.bitwidth a.value == Veir.InstSimplify.MinMaxKind.umin.inverse.saturation t.bitwidth
  | x, x_1 => false
```

Veir.InstSimplify.IS7102

Source module

Veir.InstSimplify.IS7102

Source

Variant 1: `kind = .smax; c1 : Int; c2 : Int; si = false; so = false`

```
llvm: intr.smax(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smax(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smax(a1, C2) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.inverse.saturation t.bitwidth
  | x => false
```

Variant 2: `kind = .smax; c1 : Int; c2 : Int; si = false; so = true`

```
llvm: intr.smax(c2, a1)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smax(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smax(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.inverse.saturation t.bitwidth
  | x => false
```

Variant 3: `kind = .smax; c1 : Int; c2 : Int; si = true; so = false`

```
llvm: intr.smax(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smax(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smax(a1, C2) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.inverse.saturation t.bitwidth
  | x => false
```

Variant 4: `kind = .smax; c1 : Int; c2 : Int; si = true; so = true`

```
llvm: intr.smax(c2, a1)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smax(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smax(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smax.inverse.saturation t.bitwidth
  | x => false
```

Variant 5: `kind = .smin; c1 : Int; c2 : Int; si = false; so = false`

```
llvm: intr.smin(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smin(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smin(a1, C2) : T0
  P3 = properties(root)
```

```
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.inverse.saturation t.bitwidth
  | x => false
```

Variant 6: `kind = .smin; c1 : Int; c2 : Int; si = false; so = true`

```
llvm: intr.smin(c2, a1)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smin(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smin(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.inverse.saturation t.bitwidth
  | x => false
```

Variant 7: `kind = .smin; c1 : Int; c2 : Int; si = true; so = false`

```
llvm: intr.smin(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smin(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smin(a1, C2) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
```

```
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.inverse.saturation t.bitwidth
  | x => false
```

Variant 8: `kind = .smin; c1 : Int; c2 : Int; si = true; so = true`

```
llvm: intr.smin(c2, a1)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.smin(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.smin(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.smin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.smin.inverse.saturation t.bitwidth
  | x => false
```

Variant 9: `kind = .umax; c1 : Int; c2 : Int; si = false; so = false`

```
llvm: intr.umax(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umax(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umax(a1, C2) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.inverse.saturation t.bitwidth
  | x => false
```

Variant 10: `kind = .umax; c1 : Int; c2 : Int; si = false; so = true`

```
llvm: intr.umax(c2, a1)  =>  a1
where
```

```
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umax(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umax(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.inverse.saturation t.bitwidth
  | x => false
```

Variant 11: `kind = .umax; c1 : Int; c2 : Int; si = true; so = false`

```
llvm: intr.umax(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umax(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umax(a1, C2) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.inverse.saturation t.bitwidth
  | x => false
```

Variant 12: `kind = .umax; c1 : Int; c2 : Int; si = true; so = true`

```
llvm: intr.umax(c2, a1)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umax(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umax(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
```

```
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umax.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umax.inverse.saturation t.bitwidth
  | x => false
```

Variant 13: `kind = .umin; c1 : Int; c2 : Int; si = false; so = false`

```
llvm: intr.umin(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umin(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umin(a1, C2) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.inverse.saturation t.bitwidth
  | x => false
```

Variant 14: `kind = .umin; c1 : Int; c2 : Int; si = false; so = true`

```
llvm: intr.umin(c2, a1)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umin(X, C0) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umin(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!false) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
```

```
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.inverse.saturation t.bitwidth
  | x => false
```

Variant 15: `kind = .umin; c1 : Int; c2 : Int; si = true; so = false`

```
llvm: intr.umin(a1, c2)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umin(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umin(a1, C2) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.inverse.saturation t.bitwidth
  | x => false
```

Variant 16: `kind = .umin; c1 : Int; c2 : Int; si = true; so = true`

```
llvm: intr.umin(c2, a1)  =>  a1
where
  C0 = c1 : T0
  P0 = properties(C0)
  a1 = intr.umin(C0, X) : T0
  P1 = properties(a1)
  C2 = c2 : T0
  P2 = properties(C2)
  root = intr.umin(C2, a1) : T0
  P3 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 c1
  () ← inspect a1 using fun ctx op =>
    if (!true) = true then
      match (op.getOperand! ctx 0).definingOp? with
      | some defop =>
        (guard ((defop.getOpType! ctx != Veir.OpCode.llvm Veir.Llvm.mlir__constant) = true)).bind fun __r => some
()
      | x => some ()
    else some ()
  require Veir.InstSimplify.bitConstantGuard T0 P2 c2
  require match T0.val with
  | Veir.Attribute.integerType t =>
    Veir.InstSimplify.MinMaxKind.umin.nonStrictPred.eval (BitVec.ofInt t.bitwidth c1) (BitVec.ofInt t.bitwidth
c2) &&
        !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.saturation t.bitwidth &&
      !BitVec.ofInt t.bitwidth c2 == Veir.InstSimplify.MinMaxKind.umin.inverse.saturation t.bitwidth
  | x => false
```

Veir.InstSimplify.IS7188

Source module

Veir.InstSimplify.IS7188

Source

Variant 1: `swap = false`

```
llvm: intr.uadd.sat(X, C0)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = intr.uadd.sat(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Variant 2: `swap = true`

```
llvm: intr.uadd.sat(C0, X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  root = intr.uadd.sat(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
```

Veir.InstSimplify.IS7200

Source module

Veir.InstSimplify.IS7200

Source

Variant 1: `signed = false`

```
llvm: intr.uadd.sat(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = intr.uadd.sat(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 2: `signed = true`

```
llvm: intr.sadd.sat(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = intr.sadd.sat(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS7203

Source module

Veir.InstSimplify.IS7203

Source

Variant 1: `signed = false`

```
llvm: intr.uadd.sat(0, X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = intr.uadd.sat(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 2: `signed = true`

```
llvm: intr.sadd.sat(0, X)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = intr.sadd.sat(C0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS7208

Source module

Veir.InstSimplify.IS7208

Source

Variant 1: `swap = false`

```
llvm: intr.usub.sat((if false = true then -1 else 0), X)  =>  mlir.zero()
where
  C0 = (if false = true then -1 else 0) : T0
  P0 = properties(C0)
  root = intr.usub.sat(C0, X) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (if false = true then -1 else 0)
  P2 = PUnit.unit
```

Variant 2: `swap = true`

```
llvm: intr.usub.sat(X, (if true = true then -1 else 0))  =>  mlir.zero()
where
  C0 = (if true = true then -1 else 0) : T0
  P0 = properties(C0)
  root = intr.usub.sat(X, C0) : T0
  P1 = properties(root)
  new2 = mlir.zero() : T0 [created]
  P2 = properties(new2)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (if true = true then -1 else 0)
  P2 = PUnit.unit
```

Veir.InstSimplify.IS7213

Source module

Veir.InstSimplify.IS7213

Source

Variant 1: `signed = false`

```
llvm: intr.usub.sat(X, X)  =>  mlir.zero()
where
  root = intr.usub.sat(X, X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
  X : T0
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Variant 2: `signed = true`

```
llvm: intr.ssub.sat(X, X)  =>  mlir.zero()
where
  root = intr.ssub.sat(X, X) : T0
  P0 = properties(root)
  new1 = mlir.zero() : T0 [created]
  P1 = properties(new1)
```

```
  X : T0
  T0 : Veir.IntegerType
  P1 = PUnit.unit
```

Veir.InstSimplify.IS7216

Source module

Veir.InstSimplify.IS7216

Source

Variant 1: `signed = false`

```
llvm: intr.usub.sat(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = intr.usub.sat(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Variant 2: `signed = true`

```
llvm: intr.ssub.sat(X, 0)  =>  X
where
  C0 = 0 : T0
  P0 = properties(C0)
  root = intr.ssub.sat(X, C0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
```

Veir.InstSimplify.IS7406

Source module

Veir.InstSimplify.IS7406.bswap_0

Source

```
llvm: intr.bswap(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.bswap(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 16 || t.bitwidth == 32 || t.bitwidth == 64
```

Veir.InstSimplify.IS7406.bitreverse_0

Source

```
llvm: intr.bitreverse(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.bitreverse(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.ctpop_0

Source

```
llvm: intr.ctpop(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.ctpop(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.ctlz_0

Source

```
llvm: intr.ctlz(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.ctlz(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.cttz_0

Source

```
llvm: intr.cttz(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.cttz(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.abs_0

Source

```
llvm: intr.abs(a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.abs(a0) : T0
  P1 = properties(root)
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.smax_0

Source

```
llvm: intr.smax(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.smax(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.smax_1

Source

```
llvm: intr.smax(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.smax(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.smin_0

Source

```
llvm: intr.smin(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.smin(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.smin_1

Source

```
llvm: intr.smin(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.smin(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.umax_0

Source

```
llvm: intr.umax(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.umax(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.umax_1

Source

```
llvm: intr.umax(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.umax(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.umin_0

Source

```
llvm: intr.umin(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.umin(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.umin_1

Source

```
llvm: intr.umin(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.umin(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.saddSat_0

Source

```
llvm: intr.sadd.sat(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.sadd.sat(a0, X) : T0
  P1 = properties(root)
```

```
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.saddSat_1

Source

```
llvm: intr.sadd.sat(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.sadd.sat(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.uaddSat_0

Source

```
llvm: intr.uadd.sat(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.uadd.sat(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.uaddSat_1

Source

```
llvm: intr.uadd.sat(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.uadd.sat(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.ssubSat_0

Source

```
llvm: intr.ssub.sat(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.ssub.sat(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.ssubSat_1

Source

```
llvm: intr.ssub.sat(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.ssub.sat(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.usubSat_0

Source

```
llvm: intr.usub.sat(a0, X)  =>  a0
where
```

```
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.usub.sat(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.usubSat_1

Source

```
llvm: intr.usub.sat(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.usub.sat(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.sshlSat_0

Source

```
llvm: intr.sshl.sat(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.sshl.sat(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.sshlSat_1

Source

```
llvm: intr.sshl.sat(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.sshl.sat(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.ushlSat_0

Source

```
llvm: intr.ushl.sat(a0, X)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.ushl.sat(a0, X) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.ushlSat_1

Source

```
llvm: intr.ushl.sat(X, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.ushl.sat(X, a0) : T0
  P1 = properties(root)
  X : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.fshl_0

Source

```
llvm: intr.fshl(a0, X, Y)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.fshl(a0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.fshl_1

Source

```
llvm: intr.fshl(X, a0, Y)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.fshl(X, a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.fshl_2

Source

```
llvm: intr.fshl(X, Y, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.fshl(X, Y, a0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.fshr_0

Source

```
llvm: intr.fshr(a0, X, Y)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.fshr(a0, X, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.fshr_1

Source

```
llvm: intr.fshr(X, a0, Y)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.fshr(X, a0, Y) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7406.fshr_2

Source

```
llvm: intr.fshr(X, Y, a0)  =>  a0
where
  a0 = mlir.poison() : T0
  P0 = properties(a0)
  root = intr.fshr(X, Y, a0) : T0
```

```
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
```

Veir.InstSimplify.IS7413

Source module

Veir.InstSimplify.IS7413.bswap

Source

```
llvm: intr.bswap(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.bswap(C0) : T0
  P1 = properties(root)
  C2 = mlir.constant() : T0 [created]
  P2 = properties(C2)
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 16 || t.bitwidth == 32 || t.bitwidth == 64
  require !Veir.InstSimplify.IS7413.poison_bswap T0 P0 P1
  P2 ← Veir.InstSimplify.IS7413.fold_bswap T0 P0 P1
```

Veir.InstSimplify.IS7413.bswap_poison

Source

```
llvm: intr.bswap(C0)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.bswap(C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType, where t = T0: t.bitwidth == 16 || t.bitwidth == 32 || t.bitwidth == 64
  require Veir.InstSimplify.IS7413.poison_bswap T0 P0 P1
  P2 = PUnit.unit
```

Veir.InstSimplify.IS7413.bitreverse

Source

```
llvm: intr.bitreverse(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.bitreverse(C0) : T0
  P1 = properties(root)
  C2 = mlir.constant() : T0 [created]
  P2 = properties(C2)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_bitreverse T0 P0 P1
  P2 ← Veir.InstSimplify.IS7413.fold_bitreverse T0 P0 P1
```

Veir.InstSimplify.IS7413.bitreverse_poison

Source

```
llvm: intr.bitreverse(C0)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.bitreverse(C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_bitreverse T0 P0 P1
  P2 = PUnit.unit
```

Veir.InstSimplify.IS7413.ctpop

Source

```
llvm: intr.ctpop(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.ctpop(C0) : T0
  P1 = properties(root)
  C2 = mlir.constant() : T0 [created]
  P2 = properties(C2)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_ctpop T0 P0 P1
  P2 ← Veir.InstSimplify.IS7413.fold_ctpop T0 P0 P1
```

Veir.InstSimplify.IS7413.ctpop_poison

Source

```
llvm: intr.ctpop(C0)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.ctpop(C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_ctpop T0 P0 P1
  P2 = PUnit.unit
```

Veir.InstSimplify.IS7413.ctlz

Source

```
llvm: intr.ctlz(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.ctlz(C0) : T0
  P1 = properties(root)
  C2 = mlir.constant() : T0 [created]
  P2 = properties(C2)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_ctlz T0 P0 P1
  P2 ← Veir.InstSimplify.IS7413.fold_ctlz T0 P0 P1
```

Veir.InstSimplify.IS7413.ctlz_poison

Source

```
llvm: intr.ctlz(C0)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.ctlz(C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_ctlz T0 P0 P1
  P2 = PUnit.unit
```

Veir.InstSimplify.IS7413.cttz

Source

```
llvm: intr.cttz(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.cttz(C0) : T0
  P1 = properties(root)
  C2 = mlir.constant() : T0 [created]
  P2 = properties(C2)
```

```
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_cttz T0 P0 P1
  P2 ← Veir.InstSimplify.IS7413.fold_cttz T0 P0 P1
```

Veir.InstSimplify.IS7413.cttz_poison

Source

```
llvm: intr.cttz(C0)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.cttz(C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_cttz T0 P0 P1
  P2 = PUnit.unit
```

Veir.InstSimplify.IS7413.abs

Source

```
llvm: intr.abs(C0)  =>  C2
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.abs(C0) : T0
  P1 = properties(root)
  C2 = mlir.constant() : T0 [created]
  P2 = properties(C2)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_abs T0 P0 P1
  P2 ← Veir.InstSimplify.IS7413.fold_abs T0 P0 P1
```

Veir.InstSimplify.IS7413.abs_poison

Source

```
llvm: intr.abs(C0)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.abs(C0) : T0
  P1 = properties(root)
  new2 = mlir.poison() : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_abs T0 P0 P1
  P2 = PUnit.unit
```

Veir.InstSimplify.IS7413.smax

Source

```
llvm: intr.smax(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.smax(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_smax T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_smax T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.smax_poison

Source

```
llvm: intr.smax(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.smax(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_smax T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.smin

Source

```
llvm: intr.smin(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.smin(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_smin T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_smin T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.smin_poison

Source

```
llvm: intr.smin(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.smin(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_smin T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.umax

Source

```
llvm: intr.umax(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.umax(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_umax T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_umax T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.umax_poison

Source

```
llvm: intr.umax(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
```

```
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.umax(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_umax T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.umin

Source

```
llvm: intr.umin(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.umin(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_umin T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_umin T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.umin_poison

Source

```
llvm: intr.umin(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.umin(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_umin T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.saddSat

Source

```
llvm: intr.sadd.sat(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.sadd.sat(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_saddSat T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_saddSat T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.saddSat_poison

Source

```
llvm: intr.sadd.sat(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
```

```
  root = intr.sadd.sat(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_saddSat T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.uaddSat

Source

```
llvm: intr.uadd.sat(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.uadd.sat(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_uaddSat T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_uaddSat T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.uaddSat_poison

Source

```
llvm: intr.uadd.sat(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.uadd.sat(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_uaddSat T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.ssubSat

Source

```
llvm: intr.ssub.sat(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.ssub.sat(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_ssubSat T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_ssubSat T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.ssubSat_poison

Source

```
llvm: intr.ssub.sat(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.ssub.sat(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
```

```
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_ssubSat T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.usubSat

Source

```
llvm: intr.usub.sat(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.usub.sat(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_usubSat T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_usubSat T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.usubSat_poison

Source

```
llvm: intr.usub.sat(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.usub.sat(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_usubSat T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.sshlSat

Source

```
llvm: intr.sshl.sat(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.sshl.sat(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_sshlSat T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_sshlSat T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.sshlSat_poison

Source

```
llvm: intr.sshl.sat(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.sshl.sat(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
```

```
  require Veir.InstSimplify.IS7413.poison_sshlSat T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.ushlSat

Source

```
llvm: intr.ushl.sat(C0, C1)  =>  C3
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.ushl.sat(C0, C1) : T0
  P2 = properties(root)
  C3 = mlir.constant() : T0 [created]
  P3 = properties(C3)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_ushlSat T0 P0 P1 P2
  P3 ← Veir.InstSimplify.IS7413.fold_ushlSat T0 P0 P1 P2
```

Veir.InstSimplify.IS7413.ushlSat_poison

Source

```
llvm: intr.ushl.sat(C0, C1)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  root = intr.ushl.sat(C0, C1) : T0
  P2 = properties(root)
  new3 = mlir.poison() : T0 [created]
  P3 = properties(new3)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_ushlSat T0 P0 P1 P2
  P3 = PUnit.unit
```

Veir.InstSimplify.IS7413.fshl

Source

```
llvm: intr.fshl(C0, C1, C2)  =>  C4
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  C2 = mlir.constant() : T0
  P2 = properties(C2)
  root = intr.fshl(C0, C1, C2) : T0
  P3 = properties(root)
  C4 = mlir.constant() : T0 [created]
  P4 = properties(C4)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_fshl T0 P0 P1 P2 P3
  P4 ← Veir.InstSimplify.IS7413.fold_fshl T0 P0 P1 P2 P3
```

Veir.InstSimplify.IS7413.fshl_poison

Source

```
llvm: intr.fshl(C0, C1, C2)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  C2 = mlir.constant() : T0
  P2 = properties(C2)
  root = intr.fshl(C0, C1, C2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
```

```
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_fshl T0 P0 P1 P2 P3
  P4 = PUnit.unit
```

Veir.InstSimplify.IS7413.fshr

Source

```
llvm: intr.fshr(C0, C1, C2)  =>  C4
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  C2 = mlir.constant() : T0
  P2 = properties(C2)
  root = intr.fshr(C0, C1, C2) : T0
  P3 = properties(root)
  C4 = mlir.constant() : T0 [created]
  P4 = properties(C4)
  T0 : Veir.IntegerType
  require !Veir.InstSimplify.IS7413.poison_fshr T0 P0 P1 P2 P3
  P4 ← Veir.InstSimplify.IS7413.fold_fshr T0 P0 P1 P2 P3
```

Veir.InstSimplify.IS7413.fshr_poison

Source

```
llvm: intr.fshr(C0, C1, C2)  =>  mlir.poison()
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  C2 = mlir.constant() : T0
  P2 = properties(C2)
  root = intr.fshr(C0, C1, C2) : T0
  P3 = properties(root)
  new4 = mlir.poison() : T0 [created]
  P4 = properties(new4)
  T0 : Veir.IntegerType
  require Veir.InstSimplify.IS7413.poison_fshr T0 P0 P1 P2 P3
  P4 = PUnit.unit
```

Veir.InstSimplify.IS7471

Source module

Veir.InstSimplify.IS7471

Source

Variant 1: `left = false`

```
llvm: intr.fshr(X, Y, C0)  =>  Y
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.fshr(X, Y, C0) : T0
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType w, Veir.LLVMConstantValue.integer attr =>
    (BitVec.ofInt w.bitwidth attr.value).toNat % w.bitwidth == 0
  | x, x_1 => false
```

Variant 2: `left = true`

```
llvm: intr.fshl(X, Y, C0)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  root = intr.fshl(X, Y, C0) : T0
```

```
  P1 = properties(root)
  X : T0
  Y : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value with
  | Veir.Attribute.integerType w, Veir.LLVMConstantValue.integer attr =>
    (BitVec.ofInt w.bitwidth attr.value).toNat % w.bitwidth == 0
  | x, x_1 => false
```

Veir.InstSimplify.IS7480

Source module

Veir.InstSimplify.IS7480

Source

Variant 1: `left = false`

```
llvm: intr.fshr(lshr(X, C0), shl(X, C1), C2)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  C2 = mlir.constant() : T0
  P2 = properties(C2)
  a3 = lshr(X, C0) : T0
  P3 = properties(a3)
  a4 = shl(X, C1) : T0
  P4 = properties(a4)
  root = intr.fshr(a3, a4, C2) : T0
  P5 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value, P1.value, P2.value with
  | Veir.Attribute.integerType w, Veir.LLVMConstantValue.integer a, Veir.LLVMConstantValue.integer b,
    Veir.LLVMConstantValue.integer c =>
    (BitVec.ofInt w.bitwidth c.value).toNat % w.bitwidth != 0 &&
        BitVec.ofInt w.bitwidth a.value + BitVec.ofInt w.bitwidth b.value == BitVec.ofNat w.bitwidth w.bitwidth
&&
      (BitVec.ofInt w.bitwidth c.value).toNat % w.bitwidth == (BitVec.ofInt w.bitwidth b.value).toNat
  | x, x_1, x_2, x_3 => false
```

Variant 2: `left = true`

```
llvm: intr.fshl(lshr(X, C0), shl(X, C1), C2)  =>  X
where
  C0 = mlir.constant() : T0
  P0 = properties(C0)
  C1 = mlir.constant() : T0
  P1 = properties(C1)
  C2 = mlir.constant() : T0
  P2 = properties(C2)
  a3 = lshr(X, C0) : T0
  P3 = properties(a3)
  a4 = shl(X, C1) : T0
  P4 = properties(a4)
  root = intr.fshl(a3, a4, C2) : T0
  P5 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require match T0.val, P0.value, P1.value, P2.value with
  | Veir.Attribute.integerType w, Veir.LLVMConstantValue.integer a, Veir.LLVMConstantValue.integer b,
    Veir.LLVMConstantValue.integer c =>
    (BitVec.ofInt w.bitwidth c.value).toNat % w.bitwidth != 0 &&
        BitVec.ofInt w.bitwidth a.value + BitVec.ofInt w.bitwidth b.value == BitVec.ofNat w.bitwidth w.bitwidth
&&
      (BitVec.ofInt w.bitwidth c.value).toNat % w.bitwidth == (BitVec.ofInt w.bitwidth a.value).toNat
  | x, x_1, x_2, x_3 => false
```

Veir.InstSimplify.IS7485

Source module

Veir.InstSimplify.IS7485

Source

Variant 1: `left = false`

```
llvm: intr.fshr(C0, C1, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = intr.fshr(C0, C1, X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
```

Variant 2: `left = true`

```
llvm: intr.fshl(C0, C1, X)  =>  C0
where
  C0 = 0 : T0
  P0 = properties(C0)
  C1 = 0 : T0
  P1 = properties(C1)
  root = intr.fshl(C0, C1, X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 0
  require Veir.InstSimplify.bitConstantGuard T0 P1 0
```

Veir.InstSimplify.IS7489

Source module

Veir.InstSimplify.IS7489

Source

Variant 1: `left = false`

```
llvm: intr.fshr(C0, C1, X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = -1 : T0
  P1 = properties(C1)
  root = intr.fshr(C0, C1, X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-1)
```

Variant 2: `left = true`

```
llvm: intr.fshl(C0, C1, X)  =>  C0
where
  C0 = -1 : T0
  P0 = properties(C0)
  C1 = -1 : T0
  P1 = properties(C1)
  root = intr.fshl(C0, C1, X) : T0
  P2 = properties(root)
  X : T0
  T0 : Veir.IntegerType
  require Veir.InstSimplify.bitConstantGuard T0 P0 (-1)
  require Veir.InstSimplify.bitConstantGuard T0 P1 (-1)
```

RISC-V instruction selection

Veir.Passes.InstructionSelection.RISCV64

Source module

Veir.ctlz32_pattern

Source

Builder families: `Veir.lowerUnary`.

```
llvm.intr.ctlz(X)  =>  builtin.unrealized_conversion_cast(riscv.clzw(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.intr.ctlz(X) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = riscv.clzw(new1) : T1 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.ctlz64_pattern

Source

Builder families: `Veir.lowerUnary`.

```
llvm.intr.ctlz(X)  =>  builtin.unrealized_conversion_cast(riscv.clz(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.intr.ctlz(X) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = riscv.clz(new1) : T1 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.cttz32_pattern

Source

Builder families: `Veir.lowerUnary`.

```
llvm.intr.cttz(X)  =>  builtin.unrealized_conversion_cast(riscv.ctzw(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.intr.cttz(X) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = riscv.ctzw(new1) : T1 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.cttz64_pattern

Source

Builder families: `Veir.lowerUnary`.

```
llvm.intr.cttz(X)  =>  builtin.unrealized_conversion_cast(riscv.ctz(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.intr.cttz(X) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = riscv.ctz(new1) : T1 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.ctpop32_pattern

Source

Builder families: `Veir.lowerUnary`.

```
llvm.intr.ctpop(X)  =>  builtin.unrealized_conversion_cast(riscv.cpopw(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.intr.ctpop(X) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = riscv.cpopw(new1) : T1 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.ctpop64_pattern

Source

Builder families: `Veir.lowerUnary`.

```
llvm.intr.ctpop(X)  =>  builtin.unrealized_conversion_cast(riscv.cpop(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.intr.ctpop(X) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = riscv.cpop(new1) : T1 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T0 [created]
  P3 = properties(new3)
  X : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.sext8_pattern

Source

Builder families: `Veir.lowerExt`.

```
gmir.g_sext(X)  =>  builtin.unrealized_conversion_cast(riscv.sextb(builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_sext(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.sextb(new1) : T2 [created]
```

```
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.IntegerType, where t = T1: decide (8 < t.bitwidth ∧ t.bitwidth ≤ 64)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 8)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_sext #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.sext16_pattern

Source

Builder families: `Veir.lowerExt`.

```
gmir.g_sext(X)  =>  builtin.unrealized_conversion_cast(riscv.sexth(builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_sext(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.sexth(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.IntegerType, where t = T1: decide (16 < t.bitwidth ∧ t.bitwidth ≤ 64)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 16)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_sext #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.sext32_pattern

Source

Builder families: `Veir.lowerExt`.

```
gmir.g_sext(X)  =>  builtin.unrealized_conversion_cast(riscv.sextw(builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_sext(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.sextw(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.IntegerType, where t = T1: decide (32 < t.bitwidth ∧ t.bitwidth ≤ 64)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_sext #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.zext8_pattern

Source

Builder families: `Veir.lowerExt`.

```
gmir.g_zext(X)  =>  builtin.unrealized_conversion_cast(riscv.zextb(builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_zext(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.zextb(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
```

```
  P3 = properties(new3)
  X : T0
  T1 : Veir.IntegerType, where t = T1: decide (8 < t.bitwidth ∧ t.bitwidth ≤ 64)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 8)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_zext #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.zext16_pattern

Source

Builder families: `Veir.lowerExt`.

```
gmir.g_zext(X)  =>  builtin.unrealized_conversion_cast(riscv.zexth(builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_zext(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.zexth(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.IntegerType, where t = T1: decide (16 < t.bitwidth ∧ t.bitwidth ≤ 64)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 16)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_zext #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.zext32_pattern

Source

Builder families: `Veir.lowerExt`.

```
gmir.g_zext(X)  =>  builtin.unrealized_conversion_cast(riscv.zextw(builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_zext(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.zextw(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.IntegerType, where t = T1: decide (32 < t.bitwidth ∧ t.bitwidth ≤ 64)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_zext #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.add64_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.add(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.add(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.add(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.add(new1, new2) : T1 [created]
  P3 = properties(new3)
```

```
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.add32_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.add(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.addw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.add(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.addw(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.sub64_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.sub(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sub(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.sub(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.sub(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.sub32_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.sub(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.subw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.sub(X, Y) : T0
```

```
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.subw(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.umax_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.intr.umax(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.maxu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.umax(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.maxu(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64 ∨ t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.umin_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.intr.umin(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.minu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.umin(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.minu(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64 ∨ t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.shl64_pattern

Source

Builder families: `Veir.lowerByteBinaryW`.

```
llvm.shl(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sll(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.shl(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sll(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.getIntByteTypeBitwidth t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.shl32_pattern

Source

Builder families: `Veir.lowerByteBinaryW`.

```
llvm.shl(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sllw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.shl(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sllw(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.getIntByteTypeBitwidth t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.lshr64_pattern

Source

Builder families: `Veir.lowerByteBinaryW`.

```
llvm.lshr(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.srl(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.lshr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.srl(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
```

```
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.getIntByteTypeBitwidth t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.lshr32_pattern

Source

Builder families: `Veir.lowerByteBinaryW`.

```
llvm.lshr(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.srlw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.lshr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.srlw(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.getIntByteTypeBitwidth t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.smax64_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.intr.smax(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.max(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.smax(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.max(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.smax32_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.intr.smax(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.max(riscv.sextw(builtin.unrealized_conversion_cast(X)),
riscv.sextw(builtin.unrealized_conversion_cast(Y))))
where
```

```
  root = llvm.intr.smax(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T1 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T1 [created]
  P3 = properties(new4)
  new5 = riscv.max(new3, new4) : T1 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T0 [created]
  P5 = properties(new6)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Veir.smin64_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.intr.smin(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.min(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.smin(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.min(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.smin32_pattern

Source

Builder families: `Veir.lowerBinary`.

```
llvm.intr.smin(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.min(riscv.sextw(builtin.unrealized_conversion_cast(X)),
riscv.sextw(builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.intr.smin(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T1 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T1 [created]
  P3 = properties(new4)
  new5 = riscv.min(new3, new4) : T1 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T0 [created]
  P5 = properties(new6)
```

```
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Veir.fshl64_pattern

Source

Builder families: `Veir.lowerRotate`.

```
llvm.intr.fshl(X, X, Y)  =>  builtin.unrealized_conversion_cast(riscv.rol(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.fshl(X, X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.rol(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.fshl32_pattern

Source

Builder families: `Veir.lowerRotate`.

```
llvm.intr.fshl(X, X, Y)  =>  builtin.unrealized_conversion_cast(riscv.rolw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.fshl(X, X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.rolw(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.fshr64_pattern

Source

Builder families: `Veir.lowerRotate`.

```
llvm.intr.fshr(X, X, Y)  =>  builtin.unrealized_conversion_cast(riscv.ror(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.fshr(X, X, Y) : T0
  P0 = properties(root)
```

```
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.ror(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.fshr32_pattern

Source

Builder families: `Veir.lowerRotate`.

```
llvm.intr.fshr(X, X, Y)  =>  builtin.unrealized_conversion_cast(riscv.rorw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.intr.fshr(X, X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.rorw(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.bswap64

Source

Builder families: `Veir.lowerBswap`.

Pattern expression:

```
Veir.lowerBswap 64
```

```
llvm.intr.bswap(X)  =>  builtin.unrealized_conversion_cast(riscv.rev8(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.intr.bswap(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.rev8(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
```

Veir.bswap32

Source

Builder families: `Veir.lowerBswap`.

Pattern expression:

```
Veir.lowerBswap 32
```

```
llvm.intr.bswap(X)  =>
builtin.unrealized_conversion_cast(riscv.srli(riscv.rev8(builtin.unrealized_conversion_cast(X))))
where
  root = llvm.intr.bswap(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.rev8(new1) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.srli(new2) : T2 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T1 [created]
  P4 = properties(new4)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = { value := { toFin := ⟨32, …⟩ } }
  P4 = PUnit.unit
```

Veir.bitreverse64

Source

Builder families: `Veir.lowerBitreverse`.

Pattern expression:

```
Veir.lowerBitreverse 64
```

```
llvm.intr.bitreverse(X)  =>  builtin.unrealized_conversion_cast(riscv.rev8(riscv.or(riscv.slli(riscv.and(new14,
new13)), riscv.and(new14, riscv.srli(new13)))))
where
  root = llvm.intr.bitreverse(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.li() : T2 [created]
  P2 = properties(new2)
  new3 = riscv.and(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.slli(new3) : T2 [created]
  P4 = properties(new4)
  new5 = riscv.srli(new1) : T2 [created]
  P5 = properties(new5)
  new6 = riscv.and(new2, new5) : T2 [created]
  P6 = properties(new6)
  new7 = riscv.or(new4, new6) : T2 [created]
  P7 = properties(new7)
  new8 = riscv.li() : T2 [created]
  P8 = properties(new8)
  new9 = riscv.and(new8, new7) : T2 [created]
  P9 = properties(new9)
  new10 = riscv.slli(new9) : T2 [created]
  P10 = properties(new10)
  new11 = riscv.srli(new7) : T2 [created]
  P11 = properties(new11)
  new12 = riscv.and(new8, new11) : T2 [created]
  P12 = properties(new12)
  new13 = riscv.or(new10, new12) : T2 [created]
  P13 = properties(new13)
  new14 = riscv.li() : T2 [created]
  P14 = properties(new14)
  new15 = riscv.and(new14, new13) : T2 [created]
  P15 = properties(new15)
  new16 = riscv.slli(new15) : T2 [created]
  P16 = properties(new16)
  new17 = riscv.srli(new13) : T2 [created]
  P17 = properties(new17)
  new18 = riscv.and(new14, new17) : T2 [created]
```

```
  P18 = properties(new18)
  new19 = riscv.or(new16, new18) : T2 [created]
  P19 = properties(new19)
  new20 = riscv.rev8(new19) : T2 [created]
  P20 = properties(new20)
  new21 = builtin.unrealized_conversion_cast(new20) : T1 [created]
  P21 = properties(new21)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = { value := { toFin := ⟨6148914691236517205, …⟩ } }
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
  P8 = { value := { toFin := ⟨3689348814741910323, …⟩ } }
  P9 = PUnit.unit
  P10 = { value := { toFin := ⟨2, …⟩ } }
  P11 = { value := { toFin := ⟨2, …⟩ } }
  P12 = PUnit.unit
  P13 = PUnit.unit
  P14 = { value := { toFin := ⟨1085102592571150095, …⟩ } }
  P15 = PUnit.unit
  P16 = { value := { toFin := ⟨4, …⟩ } }
  P17 = { value := { toFin := ⟨4, …⟩ } }
  P18 = PUnit.unit
  P19 = PUnit.unit
  P20 = PUnit.unit
  P21 = PUnit.unit
```

Veir.bitreverse32

Source

Builder families: `Veir.lowerBitreverse`.

Pattern expression:

```
Veir.lowerBitreverse 32
```

```
llvm.intr.bitreverse(X)  =>
builtin.unrealized_conversion_cast(riscv.srli(riscv.rev8(riscv.or(riscv.slli(riscv.and(new14, new13)),
riscv.and(new14, riscv.srli(new13))))))
where
  root = llvm.intr.bitreverse(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.li() : T2 [created]
  P2 = properties(new2)
  new3 = riscv.and(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.slli(new3) : T2 [created]
  P4 = properties(new4)
  new5 = riscv.srli(new1) : T2 [created]
  P5 = properties(new5)
  new6 = riscv.and(new2, new5) : T2 [created]
  P6 = properties(new6)
  new7 = riscv.or(new4, new6) : T2 [created]
  P7 = properties(new7)
  new8 = riscv.li() : T2 [created]
  P8 = properties(new8)
  new9 = riscv.and(new8, new7) : T2 [created]
  P9 = properties(new9)
  new10 = riscv.slli(new9) : T2 [created]
  P10 = properties(new10)
  new11 = riscv.srli(new7) : T2 [created]
  P11 = properties(new11)
  new12 = riscv.and(new8, new11) : T2 [created]
  P12 = properties(new12)
  new13 = riscv.or(new10, new12) : T2 [created]
  P13 = properties(new13)
  new14 = riscv.li() : T2 [created]
```

```
  P14 = properties(new14)
  new15 = riscv.and(new14, new13) : T2 [created]
  P15 = properties(new15)
  new16 = riscv.slli(new15) : T2 [created]
  P16 = properties(new16)
  new17 = riscv.srli(new13) : T2 [created]
  P17 = properties(new17)
  new18 = riscv.and(new14, new17) : T2 [created]
  P18 = properties(new18)
  new19 = riscv.or(new16, new18) : T2 [created]
  P19 = properties(new19)
  new20 = riscv.rev8(new19) : T2 [created]
  P20 = properties(new20)
  new21 = riscv.srli(new20) : T2 [created]
  P21 = properties(new21)
  new22 = builtin.unrealized_conversion_cast(new21) : T1 [created]
  P22 = properties(new22)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = { value := { toFin := ⟨1431655765, …⟩ } }
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
  P8 = { value := { toFin := ⟨858993459, …⟩ } }
  P9 = PUnit.unit
  P10 = { value := { toFin := ⟨2, …⟩ } }
  P11 = { value := { toFin := ⟨2, …⟩ } }
  P12 = PUnit.unit
  P13 = PUnit.unit
  P14 = { value := { toFin := ⟨252645135, …⟩ } }
  P15 = PUnit.unit
  P16 = { value := { toFin := ⟨4, …⟩ } }
  P17 = { value := { toFin := ⟨4, …⟩ } }
  P18 = PUnit.unit
  P19 = PUnit.unit
  P20 = PUnit.unit
  P21 = { value := { toFin := ⟨32, …⟩ } }
  P22 = PUnit.unit
```

Veir.constant_pattern

Source

```
llvm.mlir.constant()  =>  builtin.unrealized_conversion_cast(riscv.li())
where
  root = llvm.mlir.constant() : T0
  P0 = properties(root)
  new1 = riscv.li() : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T0 [created]
  P2 = properties(new2)
  require match P0.value with
  | Veir.LLVMConstantValue.integer value => true
  | x => false
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth ≤ 64)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 ← match T0.val with
  | Veir.Attribute.integerType type' =>
    match P0.value with
    | Veir.LLVMConstantValue.integer const =>
      some { value := BitVec.signExtend 64 (BitVec.ofInt type'.bitwidth const.value) }
    | x => none
  | x => none
  P2 = PUnit.unit
```

Veir.ashr64

Source

Builder families: `Veir.lowerAshr`.

Pattern expression:

```
Veir.lowerAshr 64
```

```
llvm.ashr(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sra(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.ashr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sra(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64 ∨ t.bitwidth = 32 ∨ t.bitwidth = 8)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64 ∨ t.bitwidth = 32 ∨ t.bitwidth = 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.ashr32

Source

Builder families: `Veir.lowerAshr`.

Pattern expression:

```
Veir.lowerAshr 32
```

```
llvm.ashr(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sraw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.ashr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sraw(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 32)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64 ∨ t.bitwidth = 32 ∨ t.bitwidth = 8)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64 ∨ t.bitwidth = 32 ∨ t.bitwidth = 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.ashr8

Source

Builder families: `Veir.lowerAshr`.

Pattern expression:

```
Veir.lowerAshr 8
```

```
llvm.ashr(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sra(riscv.sextb(builtin.unrealized_conversion_cast(X)),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.ashr(X, Y) : T2
  P0 = properties(root)
```

```
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sra(new3, new2) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 8)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64 ∨ t.bitwidth = 32 ∨ t.bitwidth = 8)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64 ∨ t.bitwidth = 32 ∨ t.bitwidth = 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Veir.lowerIcmp

Source

Variant 1: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.eq 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltiu(riscv.xor(riscv.sextb(builtin.unrealized_conversion_cast(Y)),
riscv.sextb(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.xor(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.sltiu(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.eq)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 2: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.eq 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltiu(riscv.xor(riscv.sextw(builtin.unrealized_conversion_cast(Y)),
riscv.sextw(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
```

```
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.xor(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.sltiu(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.eq)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 3: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.eq 64 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltiu(riscv.xor(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.xor(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sltiu(new3) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.eq)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 4: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.eq 8 true …`

```
llvm.icmp(X, C0)  =>
builtin.unrealized_conversion_cast(riscv.sltiu(riscv.sextb(builtin.unrealized_conversion_cast(X))))
where
  C0 = llvm.mlir.constant() : T1
  P0 = properties(C0)
  root = llvm.icmp(X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.sextb(new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.sltiu(new4) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  require decide ((cast … P1).predicate = Veir.Data.LLVM.IntPred.eq)
```

```
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  require Veir.isConstantZero T1 P0
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 5: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.eq 32 true …`

```
llvm.icmp(X, C0)  =>
builtin.unrealized_conversion_cast(riscv.sltiu(riscv.sextw(builtin.unrealized_conversion_cast(X))))
where
  C0 = llvm.mlir.constant() : T1
  P0 = properties(C0)
  root = llvm.icmp(X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.sextw(new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.sltiu(new4) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  require decide ((cast … P1).predicate = Veir.Data.LLVM.IntPred.eq)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  require Veir.isConstantZero T1 P0
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 6: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.eq 64 true …`

```
llvm.icmp(X, C0)  =>  builtin.unrealized_conversion_cast(riscv.sltiu(builtin.unrealized_conversion_cast(X)))
where
  C0 = llvm.mlir.constant() : T1
  P0 = properties(C0)
  root = llvm.icmp(X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sltiu(new2) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  require decide ((cast … P1).predicate = Veir.Data.LLVM.IntPred.eq)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  require Veir.isConstantZero T1 P0
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 7: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ne 8 false …`

```
llvm.icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
riscv.xor(riscv.sextb(builtin.unrealized_conversion_cast(Y)),
riscv.sextb(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.xor(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.li() : T3 [created]
  P5 = properties(new6)
  new7 = riscv.sltu(new6, new5) : T3 [created]
  P6 = properties(new7)
  new8 = builtin.unrealized_conversion_cast(new7) : T2 [created]
  P7 = properties(new8)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ne)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨0, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
```

Variant 8: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ne 32 false …`

```
llvm.icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
riscv.xor(riscv.sextw(builtin.unrealized_conversion_cast(Y)),
riscv.sextw(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.xor(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.li() : T3 [created]
  P5 = properties(new6)
  new7 = riscv.sltu(new6, new5) : T3 [created]
  P6 = properties(new7)
  new8 = builtin.unrealized_conversion_cast(new7) : T2 [created]
  P7 = properties(new8)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ne)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨0, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
```

Variant 9: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ne 64 false …`

```
llvm.icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
riscv.xor(builtin.unrealized_conversion_cast(Y), builtin.unrealized_conversion_cast(X))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.xor(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.li() : T3 [created]
  P4 = properties(new4)
  new5 = riscv.sltu(new4, new3) : T3 [created]
  P5 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P6 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ne)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨0, …⟩ } }
  P5 = PUnit.unit
  P6 = PUnit.unit
```

Variant 10: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ne 8 true …`

```
llvm.icmp(X, C0)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
riscv.sextb(builtin.unrealized_conversion_cast(X))))
where
  C0 = llvm.mlir.constant() : T1
  P0 = properties(C0)
  root = llvm.icmp(X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.sextb(new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.li() : T3 [created]
  P5 = properties(new6)
  new7 = riscv.sltu(new6, new4) : T3 [created]
  P6 = properties(new7)
  new8 = builtin.unrealized_conversion_cast(new7) : T2 [created]
  P7 = properties(new8)
  X : T0
  require decide ((cast … P1).predicate = Veir.Data.LLVM.IntPred.ne)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  require Veir.isConstantZero T1 P0
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨0, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
```

Variant 11: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ne 32 true …`

```
llvm.icmp(X, C0)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
riscv.sextw(builtin.unrealized_conversion_cast(X))))
where
  C0 = llvm.mlir.constant() : T1
```

```
  P0 = properties(C0)
  root = llvm.icmp(X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.sextw(new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.li() : T3 [created]
  P5 = properties(new6)
  new7 = riscv.sltu(new6, new4) : T3 [created]
  P6 = properties(new7)
  new8 = builtin.unrealized_conversion_cast(new7) : T2 [created]
  P7 = properties(new8)
  X : T0
  require decide ((cast … P1).predicate = Veir.Data.LLVM.IntPred.ne)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  require Veir.isConstantZero T1 P0
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨0, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
```

Variant 12: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ne 64 true …`

```
llvm.icmp(X, C0)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
builtin.unrealized_conversion_cast(X)))
where
  C0 = llvm.mlir.constant() : T1
  P0 = properties(C0)
  root = llvm.icmp(X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.li() : T3 [created]
  P4 = properties(new4)
  new5 = riscv.sltu(new4, new2) : T3 [created]
  P5 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P6 = properties(new6)
  X : T0
  require decide ((cast … P1).predicate = Veir.Data.LLVM.IntPred.ne)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  require Veir.isConstantZero T1 P0
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨0, …⟩ } }
  P5 = PUnit.unit
  P6 = PUnit.unit
```

Variant 13: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.slt 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.slt(riscv.sextb(builtin.unrealized_conversion_cast(X)),
riscv.sextb(builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
```

```
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.slt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 14: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.slt 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.slt(riscv.sextw(builtin.unrealized_conversion_cast(X)),
riscv.sextw(builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.slt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 15: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.slt 64 false …`

```
llvm.icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.slt(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.slt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
```

```
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 16: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sgt 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.slt(riscv.sextb(builtin.unrealized_conversion_cast(Y)),
riscv.sextb(builtin.unrealized_conversion_cast(X))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sgt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 17: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sgt 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.slt(riscv.sextw(builtin.unrealized_conversion_cast(Y)),
riscv.sextw(builtin.unrealized_conversion_cast(X))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sgt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 18: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sgt 64 false …`

```
llvm.icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.slt(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  root = llvm.icmp(X, Y) : T2
```

```
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sgt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 19: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ult 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltu(riscv.sextb(builtin.unrealized_conversion_cast(X)),
riscv.sextb(builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.sltu(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ult)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 20: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ult 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltu(riscv.sextw(builtin.unrealized_conversion_cast(X)),
riscv.sextw(builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.sltu(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
```

```
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ult)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 21: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ult 64 false …`

```
llvm.icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ult)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 22: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ugt 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltu(riscv.sextb(builtin.unrealized_conversion_cast(Y)),
riscv.sextb(builtin.unrealized_conversion_cast(X))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.sltu(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ugt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 23: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ugt 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltu(riscv.sextw(builtin.unrealized_conversion_cast(Y)),
riscv.sextw(builtin.unrealized_conversion_cast(X))))
```

```
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.sltu(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P5 = properties(new6)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ugt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Variant 24: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ugt 64 false …`

```
llvm.icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ugt)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 25: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sge 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(riscv.sextb(builtin.unrealized_conversion_cast(X)),
riscv.sextb(builtin.unrealized_conversion_cast(Y)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
```

```
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sge)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 26: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sge 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(riscv.sextw(builtin.unrealized_conversion_cast(X)),
riscv.sextw(builtin.unrealized_conversion_cast(Y)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sge)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 27: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sge 64 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sge)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
```

```
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 28: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sle 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(riscv.sextb(builtin.unrealized_conversion_cast(Y)),
riscv.sextb(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sle)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 29: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sle 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(riscv.sextw(builtin.unrealized_conversion_cast(Y)),
riscv.sextw(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.slt(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sle)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

```
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 30: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.sle 64 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.sle)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 31: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.uge 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(riscv.sextb(builtin.unrealized_conversion_cast(X)),
riscv.sextb(builtin.unrealized_conversion_cast(Y)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.sltu(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.uge)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 32: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.uge 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(riscv.sextw(builtin.unrealized_conversion_cast(X)),
riscv.sextw(builtin.unrealized_conversion_cast(Y)))))
where
```

```
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.sltu(new3, new4) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.uge)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 33: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.uge 64 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.uge)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 34: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ule 8 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(riscv.sextb(builtin.unrealized_conversion_cast(Y)),
riscv.sextb(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextb(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextb(new2) : T3 [created]
```

```
  P3 = properties(new4)
  new5 = riscv.sltu(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ule)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 8)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 35: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ule 32 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(riscv.sextw(builtin.unrealized_conversion_cast(Y)),
riscv.sextw(builtin.unrealized_conversion_cast(X)))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sextw(new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sextw(new2) : T3 [created]
  P3 = properties(new4)
  new5 = riscv.sltu(new4, new3) : T3 [created]
  P4 = properties(new5)
  new6 = riscv.xori(new5) : T3 [created]
  P5 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T2 [created]
  P6 = properties(new7)
  X : T0
  Y : T1
  require decide ((cast … P0).predicate = Veir.Data.LLVM.IntPred.ule)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 32)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
```

Variant 36: `Veir.lowerIcmp (Veir.OpCode.llvm Veir.Llvm.icmp) Veir.Data.LLVM.IntPred.ule 64 false …`

```
llvm.icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X))))
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
```

```
  Y : T1
  require decide ((cast ⋯ P0).predicate = Veir.Data.LLVM.IntPred.ule)
  T2 : Veir.IntegerType
  T1 : Veir.TypeAttr, where t = T1: Option.any (fun x => decide (x ∈ [64, 32, 8])) (Veir.icmpTypeWidth? t)
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.icmpTypeWidth? t = some 64)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, ⋯⟩ } }
  P5 = PUnit.unit
```

Veir.trunc_pattern

Source

Builder families: `Veir.lowerRegCast`.

```
llvm.trunc(X)  =>  builtin.unrealized_conversion_cast(builtin.unrealized_conversion_cast(X))
where
  root = llvm.trunc(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T1 [created]
  P2 = properties(new2)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.isLegalTrunc T0 T1
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Veir.bitcast_pattern

Source

Builder families: `Veir.lowerRegCast`.

```
llvm.bitcast(X)  =>  builtin.unrealized_conversion_cast(builtin.unrealized_conversion_cast(X))
where
  root = llvm.bitcast(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T1 [created]
  P2 = properties(new2)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.isLegalBitcast T0 T1
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Veir.lifetimeStart

Source

Builder families: `Veir.lowerLifetime`.

Pattern expression:

```
Veir.lowerLifetime Veir.Llvm.intr__lifetime__start
```

```
llvm: intr.lifetime.start(X)  =>  ()
where
  root = intr.lifetime.start(X) : ()
  P0 = properties(root)
  X : T0
  T0 : Veir.TypeAttr
```

Veir.lifetimeEnd

Source

Builder families: `Veir.lowerLifetime`.

Pattern expression:

```
Veir.lowerLifetime Veir.Llvm.intr__lifetime__end
```

```
llvm: intr.lifetime.end(X)  =>  ()
where
  root = intr.lifetime.end(X) : ()
  P0 = properties(root)
  X : T0
  T0 : Veir.TypeAttr
```

Veir.lowerLoad

Source

Variant 1: `Veir.lowerLoad false 1 Veir.Riscv.lb …`

```
llvm.load(X)  =>  builtin.unrealized_conversion_cast(riscv.lb(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.load(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.lb(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.TypeAttr, where t = T1: decide (Veir.memAccessWidth? t = some 1)
  T0 : Veir.TypeAttr
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
  P3 = PUnit.unit
```

Variant 2: `Veir.lowerLoad false 2 Veir.Riscv.lh …`

```
llvm.load(X)  =>  builtin.unrealized_conversion_cast(riscv.lh(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.load(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.lh(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.TypeAttr, where t = T1: decide (Veir.memAccessWidth? t = some 2)
  T0 : Veir.TypeAttr
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
  P3 = PUnit.unit
```

Variant 3: `Veir.lowerLoad false 4 Veir.Riscv.lw …`

```
llvm.load(X)  =>  builtin.unrealized_conversion_cast(riscv.lw(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.load(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.lw(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.TypeAttr, where t = T1: decide (Veir.memAccessWidth? t = some 4)
  T0 : Veir.TypeAttr
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
  P3 = PUnit.unit
```

Variant 4: `Veir.lowerLoad false 8 Veir.Riscv.ld …`

```
llvm.load(X)  =>  builtin.unrealized_conversion_cast(riscv.ld(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.load(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.ld(new1) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P3 = properties(new3)
  X : T0
  T1 : Veir.TypeAttr, where t = T1: decide (Veir.memAccessWidth? t = some 8)
  T0 : Veir.TypeAttr
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
  P3 = PUnit.unit
```

Variant 5: `Veir.lowerLoad true 1 Veir.Riscv.lb …`

```
llvm.load(llvm.getelementptr(X, C0))  =>
builtin.unrealized_conversion_cast(riscv.lb(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  a1 = llvm.getelementptr(X, C0) : T2
  P1 = properties(a1)
  root = llvm.load(a1) : T3
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.lb(new3) : T4 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T3 [created]
  P5 = properties(new5)
  X : T0
  T3 : Veir.TypeAttr, where t = T3: decide (Veir.memAccessWidth? t = some 1)
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
  P4 ← Option.map (fun offset => cast … { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
  P5 = PUnit.unit
```

Variant 6: `Veir.lowerLoad true 2 Veir.Riscv.lh …`

```
llvm.load(llvm.getelementptr(X, C0))  =>
builtin.unrealized_conversion_cast(riscv.lh(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  a1 = llvm.getelementptr(X, C0) : T2
  P1 = properties(a1)
  root = llvm.load(a1) : T3
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.lh(new3) : T4 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T3 [created]
  P5 = properties(new5)
  X : T0
  T3 : Veir.TypeAttr, where t = T3: decide (Veir.memAccessWidth? t = some 2)
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
  P4 ← Option.map (fun offset => cast … { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
  P5 = PUnit.unit
```

Variant 7: `Veir.lowerLoad true 4 Veir.Riscv.lw …`

```
llvm.load(llvm.getelementptr(X, C0))  =>
builtin.unrealized_conversion_cast(riscv.lw(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  a1 = llvm.getelementptr(X, C0) : T2
  P1 = properties(a1)
  root = llvm.load(a1) : T3
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.lw(new3) : T4 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T3 [created]
  P5 = properties(new5)
  X : T0
  T3 : Veir.TypeAttr, where t = T3: decide (Veir.memAccessWidth? t = some 4)
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
  P4 ← Option.map (fun offset => cast … { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
  P5 = PUnit.unit
```

Variant 8: `Veir.lowerLoad true 8 Veir.Riscv.ld …`

```
llvm.load(llvm.getelementptr(X, C0))  =>
builtin.unrealized_conversion_cast(riscv.ld(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  a1 = llvm.getelementptr(X, C0) : T2
  P1 = properties(a1)
  root = llvm.load(a1) : T3
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.ld(new3) : T4 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T3 [created]
  P5 = properties(new5)
  X : T0
  T3 : Veir.TypeAttr, where t = T3: decide (Veir.memAccessWidth? t = some 8)
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
  P4 ← Option.map (fun offset => cast … { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
  P5 = PUnit.unit
```

Veir.lowerStore

Source

Variant 1: `Veir.lowerStore false 1 Veir.Riscv.sb …`

```
llvm.store(X, Y)  =>  ()
where
  root = llvm.store(X, Y) : ()
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sb(new2, new1) : () [created]
  P3 = properties(new3)
  X : T0
  Y : T1
  T1 : Veir.TypeAttr
```

```
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 1)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
```

Variant 2: `Veir.lowerStore false 2 Veir.Riscv.sh …`

```
llvm.store(X, Y)  =>  ()
where
  root = llvm.store(X, Y) : ()
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sh(new2, new1) : () [created]
  P3 = properties(new3)
  X : T0
  Y : T1
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 2)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
```

Variant 3: `Veir.lowerStore false 4 Veir.Riscv.sw …`

```
llvm.store(X, Y)  =>  ()
where
  root = llvm.store(X, Y) : ()
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sw(new2, new1) : () [created]
  P3 = properties(new3)
  X : T0
  Y : T1
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 4)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
```

Variant 4: `Veir.lowerStore false 8 Veir.Riscv.sd …`

```
llvm.store(X, Y)  =>  ()
where
  root = llvm.store(X, Y) : ()
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sd(new2, new1) : () [created]
  P3 = properties(new3)
  X : T0
  Y : T1
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 8)
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 ← some (cast … { value := 0, volatile_ := P0.volatile_ })
```

Variant 5: `Veir.lowerStore true 1 Veir.Riscv.sb …`

```
llvm.store(X, llvm.getelementptr(Y, C0))  =>  ()
where
  C0 = gmir.g_constant() : T2
  P0 = properties(C0)
  a1 = llvm.getelementptr(Y, C0) : T3
```

```
  P1 = properties(a1)
  root = llvm.store(X, a1) : ()
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.sb(new4, new3) : () [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T3 : Veir.TypeAttr
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 1)
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 ← Option.map (fun offset => cast ⋯ { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
```

Variant 6: `Veir.lowerStore true 2 Veir.Riscv.sh ⋯`

```
llvm.store(X, llvm.getelementptr(Y, C0))  =>  ()
where
  C0 = gmir.g_constant() : T2
  P0 = properties(C0)
  a1 = llvm.getelementptr(Y, C0) : T3
  P1 = properties(a1)
  root = llvm.store(X, a1) : ()
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.sh(new4, new3) : () [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T3 : Veir.TypeAttr
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 2)
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 ← Option.map (fun offset => cast ⋯ { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
```

Variant 7: `Veir.lowerStore true 4 Veir.Riscv.sw ⋯`

```
llvm.store(X, llvm.getelementptr(Y, C0))  =>  ()
where
  C0 = gmir.g_constant() : T2
  P0 = properties(C0)
  a1 = llvm.getelementptr(Y, C0) : T3
  P1 = properties(a1)
  root = llvm.store(X, a1) : ()
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.sw(new4, new3) : () [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T3 : Veir.TypeAttr
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 4)
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
```

```
  P4 = PUnit.unit
  P5 ← Option.map (fun offset => cast … { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
```

Variant 8: `Veir.lowerStore true 8 Veir.Riscv.sd …`

```
llvm.store(X, llvm.getelementptr(Y, C0))  =>  ()
where
  C0 = gmir.g_constant() : T2
  P0 = properties(C0)
  a1 = llvm.getelementptr(Y, C0) : T3
  P1 = properties(a1)
  root = llvm.store(X, a1) : ()
  P2 = properties(root)
  new3 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.sd(new4, new3) : () [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T3 : Veir.TypeAttr
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr, where t = T0: decide (Veir.memAccessWidth? t = some 8)
  require (Veir.selectAddrRegImm P1 P0).isSome
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 ← Option.map (fun offset => cast … { value := BitVec.ofInt 64 offset, volatile_ := P2.volatile_ })
    (Veir.selectAddrRegImm P1 P0)
```

Veir.lowerGetelementptr

Source

Variant 1: `Veir.lowerGetelementptr 1`

```
llvm.getelementptr(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.add(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = llvm.getelementptr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.add(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide (P0.rawConstantIndices.values = #[-2147483648])
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require Option.any (fun x => decide (x = 1)) (Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 2: `Veir.lowerGetelementptr 2`

```
llvm.getelementptr(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sh1add(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  root = llvm.getelementptr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
```

```
  P2 = properties(new2)
  new3 = riscv.sh1add(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide (P0.rawConstantIndices.values = #[-2147483648])
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require Option.any (fun x => decide (x = 2)) (Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 3: `Veir.lowerGetelementptr 4`

```
llvm.getelementptr(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sh2add(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  root = llvm.getelementptr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sh2add(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide (P0.rawConstantIndices.values = #[-2147483648])
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require Option.any (fun x => decide (x = 4)) (Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 4: `Veir.lowerGetelementptr 8`

```
llvm.getelementptr(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sh3add(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  root = llvm.getelementptr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sh3add(new2, new1) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  Y : T1
  require decide (P0.rawConstantIndices.values = #[-2147483648])
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require Option.any (fun x => decide (x = 8)) (Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.getelementptrShift_pattern

Source

```
llvm.getelementptr(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.add(builtin.unrealized_conversion_cast(X),
riscv.slli(builtin.unrealized_conversion_cast(Y))))
where
  root = llvm.getelementptr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.slli(new2) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.add(new1, new3) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  require decide (P0.rawConstantIndices.values = #[-2147483648])
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require Option.any (fun scale => decide (¬scale ∈ [1, 2, 4, 8] ∧ 0 < scale ∧ scale &&& scale - 1 = 0 ∧
scale.log2 < 64))
    (Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 ← Option.map (fun scale => { value := BitVec.ofInt 64 ↑scale.log2 })
    (Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val)
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Veir.getelementptrMul_pattern

Source

```
llvm.getelementptr(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.add(builtin.unrealized_conversion_cast(X),
riscv.mul(builtin.unrealized_conversion_cast(Y), riscv.li())))
where
  root = llvm.getelementptr(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.li() : T3 [created]
  P3 = properties(new3)
  new4 = riscv.mul(new2, new3) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.add(new1, new4) : T3 [created]
  P5 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P6 = properties(new6)
  X : T0
  Y : T1
  require decide (P0.rawConstantIndices.values = #[-2147483648])
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  require Option.any (fun scale => decide (¬scale ∈ [1, 2, 4, 8] ∧ ¬(0 < scale ∧ scale &&& scale - 1 = 0 ∧
scale.log2 < 64)))
    (Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val)
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 ← Option.map (fun scale => { value := BitVec.ofInt 64 scale })
    ((Veir.DataLayout.riscv64.getTypeAllocSize P0.elem_type.val).bind fun a => some ↑a)
  P4 = PUnit.unit
  P5 = PUnit.unit
  P6 = PUnit.unit
```

Veir.selectCzeroeqz_pattern

Source

```
llvm.select(X, Y, C0)  =>
builtin.unrealized_conversion_cast(riscv.czeroeqz(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T2
  P0 = properties(C0)
  root = llvm.select(X, Y, C0) : T3
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.czeroeqz(new2, new3) : T4 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T3 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T3 : Veir.IntegerType, where t = T3: decide (t.bitwidth = 64 ∨ t.bitwidth = 32)
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require decide (BitVec.ofInt P0.value.type.bitwidth P0.value.value = 0)
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Veir.selectCzeronez_pattern

Source

```
llvm.select(X, C0, Y)  =>
builtin.unrealized_conversion_cast(riscv.czeronez(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T2
  P0 = properties(C0)
  root = llvm.select(X, C0, Y) : T3
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.czeronez(new2, new3) : T4 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T3 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T3 : Veir.IntegerType, where t = T3: decide (t.bitwidth = 64 ∨ t.bitwidth = 32)
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require decide (BitVec.ofInt P0.value.type.bitwidth P0.value.value = 0)
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Veir.selectGeneral_pattern

Source

```
llvm.select(X, Y, Z)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.czeroeqz(builtin.unrealized_conversion_cast(Y), new3),
riscv.czeronez(builtin.unrealized_conversion_cast(Z), new3)))
where
  root = llvm.select(X, Y, Z) : T3
  P0 = properties(root)
```

```
  new1 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Z) : T4 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.czeroeqz(new1, new3) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.czeronez(new2, new3) : T4 [created]
  P5 = properties(new5)
  new6 = riscv.or(new4, new5) : T4 [created]
  P6 = properties(new6)
  new7 = builtin.unrealized_conversion_cast(new6) : T3 [created]
  P7 = properties(new7)
  X : T0
  Y : T1
  Z : T2
  T3 : Veir.IntegerType, where t = T3: decide (t.bitwidth = 64 ∨ t.bitwidth = 32 ∨ t.bitwidth = 1)
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
  P6 = PUnit.unit
  P7 = PUnit.unit
```

Veir.saddSat_pattern

Source

```
llvm.intr.sadd.sat(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.czeroeqz(riscv.xor(riscv.srai(new4), riscv.slli(riscv.li())),
new9), riscv.czeronez(new4, new9)))
where
  root = llvm.intr.sadd.sat(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.li() : T3 [created]
  P3 = properties(new3)
  new4 = riscv.add(new1, new2) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.srli(new2) : T3 [created]
  P5 = properties(new5)
  new6 = riscv.slt(new4, new1) : T3 [created]
  P6 = properties(new6)
  new7 = riscv.srai(new4) : T3 [created]
  P7 = properties(new7)
  new8 = riscv.slli(new3) : T3 [created]
  P8 = properties(new8)
  new9 = riscv.xor(new5, new6) : T3 [created]
  P9 = properties(new9)
  new10 = riscv.xor(new7, new8) : T3 [created]
  P10 = properties(new10)
  new11 = riscv.czeronez(new4, new9) : T3 [created]
  P11 = properties(new11)
  new12 = riscv.czeroeqz(new10, new9) : T3 [created]
  P12 = properties(new12)
  new13 = riscv.or(new12, new11) : T3 [created]
  P13 = properties(new13)
  new14 = builtin.unrealized_conversion_cast(new13) : T2 [created]
  P14 = properties(new14)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
```

```
  P2 = PUnit.unit
  P3 = Veir.mkRISCVImm (Int.negSucc 0)
  P4 = PUnit.unit
  P5 = Veir.mkRISCVImm (Int.ofNat 63)
  P6 = PUnit.unit
  P7 = Veir.mkRISCVImm (Int.ofNat 63)
  P8 = Veir.mkRISCVImm (Int.ofNat 63)
  P9 = PUnit.unit
  P10 = PUnit.unit
  P11 = PUnit.unit
  P12 = PUnit.unit
  P13 = PUnit.unit
  P14 = PUnit.unit
```

Veir.ssubSat_pattern

Source

```
llvm.intr.ssub.sat(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.czeroeqz(riscv.xor(riscv.srai(new4), riscv.slli(riscv.li(1))),
new9), riscv.czeronez(new4, new9)))
where
  root = llvm.intr.ssub.sat(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.li() : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sub(new1, new2) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.slt(new1, new2) : T3 [created]
  P5 = properties(new5)
  new6 = riscv.srli(new4) : T3 [created]
  P6 = properties(new6)
  new7 = riscv.srai(new4) : T3 [created]
  P7 = properties(new7)
  new8 = riscv.slli(new3) : T3 [created]
  P8 = properties(new8)
  new9 = riscv.xor(new5, new6) : T3 [created]
  P9 = properties(new9)
  new10 = riscv.xor(new7, new8) : T3 [created]
  P10 = properties(new10)
  new11 = riscv.czeronez(new4, new9) : T3 [created]
  P11 = properties(new11)
  new12 = riscv.czeroeqz(new10, new9) : T3 [created]
  P12 = properties(new12)
  new13 = riscv.or(new12, new11) : T3 [created]
  P13 = properties(new13)
  new14 = builtin.unrealized_conversion_cast(new13) : T2 [created]
  P14 = properties(new14)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = Veir.mkRISCVImm (Int.negSucc 0)
  P4 = PUnit.unit
  P5 = PUnit.unit
  P6 = Veir.mkRISCVImm (Int.ofNat 63)
  P7 = Veir.mkRISCVImm (Int.ofNat 63)
  P8 = Veir.mkRISCVImm (Int.ofNat 63)
  P9 = PUnit.unit
  P10 = PUnit.unit
  P11 = PUnit.unit
  P12 = PUnit.unit
  P13 = PUnit.unit
  P14 = PUnit.unit
```

Veir.uaddSat_pattern

Source

```
llvm.intr.uadd.sat(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.add(riscv.minu(builtin.unrealized_conversion_cast(X), riscv.xori(new2)),
new2))
where
  root = llvm.intr.uadd.sat(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.xori(new2) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.minu(new1, new3) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.add(new4, new2) : T3 [created]
  P5 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T2 [created]
  P6 = properties(new6)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = Veir.mkRISCVImm (Int.negSucc 0)
  P4 = PUnit.unit
  P5 = PUnit.unit
  P6 = PUnit.unit
```

Veir.usubSat_pattern

Source

```
llvm.intr.usub.sat(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sub(riscv.maxu(builtin.unrealized_conversion_cast(X), new2), new2))
where
  root = llvm.intr.usub.sat(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.maxu(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.sub(new3, new2) : T3 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T2 [created]
  P5 = properties(new5)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
```

Veir.sshlSat_pattern

Source

```
llvm.intr.sshl.sat(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.czeroeqz(riscv.xor(riscv.srai(new1), riscv.srli(riscv.li())),
new8), riscv.czeronez(new3, new8)))
where
  root = llvm.intr.sshl.sat(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
```

```
  P2 = properties(new2)
  new3 = riscv.sll(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.li() : T3 [created]
  P4 = properties(new4)
  new5 = riscv.sra(new3, new2) : T3 [created]
  P5 = properties(new5)
  new6 = riscv.srai(new1) : T3 [created]
  P6 = properties(new6)
  new7 = riscv.srli(new4) : T3 [created]
  P7 = properties(new7)
  new8 = riscv.xor(new1, new5) : T3 [created]
  P8 = properties(new8)
  new9 = riscv.xor(new6, new7) : T3 [created]
  P9 = properties(new9)
  new10 = riscv.czeronez(new3, new8) : T3 [created]
  P10 = properties(new10)
  new11 = riscv.czeroeqz(new9, new8) : T3 [created]
  P11 = properties(new11)
  new12 = riscv.or(new11, new10) : T3 [created]
  P12 = properties(new12)
  new13 = builtin.unrealized_conversion_cast(new12) : T2 [created]
  P13 = properties(new13)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = Veir.mkRISCVImm (Int.negSucc 0)
  P5 = PUnit.unit
  P6 = Veir.mkRISCVImm (Int.ofNat 63)
  P7 = Veir.mkRISCVImm (Int.ofNat 1)
  P8 = PUnit.unit
  P9 = PUnit.unit
  P10 = PUnit.unit
  P11 = PUnit.unit
  P12 = PUnit.unit
  P13 = PUnit.unit
```

Veir.ushlSat_pattern

Source

```
llvm.intr.ushl.sat(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.or(riscv.addi(riscv.sltiu(riscv.xor(new1,
riscv.srl(new3, new2)))), new3))
where
  root = llvm.intr.ushl.sat(X, Y) : T2
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.sll(new1, new2) : T3 [created]
  P3 = properties(new3)
  new4 = riscv.srl(new3, new2) : T3 [created]
  P4 = properties(new4)
  new5 = riscv.xor(new1, new4) : T3 [created]
  P5 = properties(new5)
  new6 = riscv.sltiu(new5) : T3 [created]
  P6 = properties(new6)
  new7 = riscv.addi(new6) : T3 [created]
  P7 = properties(new7)
  new8 = riscv.or(new7, new3) : T3 [created]
  P8 = properties(new8)
  new9 = builtin.unrealized_conversion_cast(new8) : T2 [created]
  P9 = properties(new9)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
```

```
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
  P5 = PUnit.unit
  P6 = Veir.mkRISCVImm (Int.ofNat 1)
  P7 = Veir.mkRISCVImm (Int.negSucc 0)
  P8 = PUnit.unit
  P9 = PUnit.unit
```

Veir.abs_pattern

Source

```
llvm.intr.abs(X)  =>  builtin.unrealized_conversion_cast(riscv.max(new1, riscv.neg(new1)))
where
  root = llvm.intr.abs(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = riscv.neg(new1) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.max(new1, new2) : T2 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T1 [created]
  P4 = properties(new4)
  X : T0
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth = 64)
  T0 : Veir.TypeAttr
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.fshrConst64

Source

Builder families: `Veir.lowerFshConst`.

Pattern expression:

```
Veir.lowerFshConst false 64
```

```
llvm.intr.fshr(X, X, C0)  =>
builtin.unrealized_conversion_cast(riscv.rori(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  root = llvm.intr.fshr(X, X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.rori(new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.IntegerType
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 ← match T1.val with
  | Veir.Attribute.integerType t =>
    some
      {
        value :=
          BitVec.ofInt 64
            (if false = true then (↑64 - ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑64 + ↑64) % ↑64) %
↑64
            else ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑64 + ↑64) % ↑64) }
  | x => none
  P4 = PUnit.unit
```

Veir.fshrConst32

Source

Builder families: `Veir.lowerFshConst`.

Pattern expression:

```
Veir.lowerFshConst false 32
```

```
llvm.intr.fshr(X, X, C0)  =>
builtin.unrealized_conversion_cast(riscv.roriw(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  root = llvm.intr.fshr(X, X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.roriw(new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 32)
  T1 : Veir.IntegerType
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 ← match T1.val with
  | Veir.Attribute.integerType t =>
    some
      {
        value :=
          BitVec.ofInt 64
            (if false = true then (↑32 - ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑32 + ↑32) % ↑32) %
↑32
            else ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑32 + ↑32) % ↑32) }
  | x => none
  P4 = PUnit.unit
```

Veir.fshlConst64

Source

Builder families: `Veir.lowerFshConst`.

Pattern expression:

```
Veir.lowerFshConst true 64
```

```
llvm.intr.fshl(X, X, C0)  =>
builtin.unrealized_conversion_cast(riscv.rori(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  root = llvm.intr.fshl(X, X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.rori(new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 64)
  T1 : Veir.IntegerType
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 ← match T1.val with
  | Veir.Attribute.integerType t =>
    some
      {
        value :=
          BitVec.ofInt 64
            (if true = true then (↑64 - ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑64 + ↑64) % ↑64) % ↑64
            else ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑64 + ↑64) % ↑64) }
```

```
  | x => none
  P4 = PUnit.unit
```

Veir.fshlConst32

Source

Builder families: `Veir.lowerFshConst`.

Pattern expression:

```
Veir.lowerFshConst true 32
```

```
llvm.intr.fshl(X, X, C0)  =>
builtin.unrealized_conversion_cast(riscv.roriw(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T1
  P0 = properties(C0)
  root = llvm.intr.fshl(X, X, C0) : T2
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T3 [created]
  P2 = properties(new2)
  new3 = riscv.roriw(new2) : T3 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T2 [created]
  P4 = properties(new4)
  X : T0
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth = 32)
  T1 : Veir.IntegerType
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 ← match T1.val with
  | Veir.Attribute.integerType t =>
    some
      {
        value :=
          BitVec.ofInt 64
            (if true = true then (↑32 - ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑32 + ↑32) % ↑32) % ↑32
            else ((BitVec.ofInt t.bitwidth P0.value.value).toInt % ↑32 + ↑32) % ↑32) }
  | x => none
  P4 = PUnit.unit
```

Veir.fshlGeneral64

Source

Builder families: `Veir.lowerFshlGeneral`.

Pattern expression:

```
Veir.lowerFshlGeneral 64
```

```
llvm.intr.fshl(X, Y, Z)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.sll(builtin.unrealized_conversion_cast(X), new3),
riscv.srl(riscv.srli(builtin.unrealized_conversion_cast(Y)), riscv.xori(new3))))
where
  root = llvm.intr.fshl(X, Y, Z) : T3
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(Z) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.sll(new1, new3) : T4 [created]
  P5 = properties(new5)
  new6 = riscv.srli(new2) : T4 [created]
  P6 = properties(new6)
  new7 = riscv.srl(new6, new4) : T4 [created]
  P7 = properties(new7)
  new8 = riscv.or(new5, new7) : T4 [created]
  P8 = properties(new8)
  new9 = builtin.unrealized_conversion_cast(new8) : T3 [created]
  P9 = properties(new9)
```

```
  X : T0
  Y : T1
  Z : T2
  T3 : Veir.IntegerType, where t = T3: decide (t.bitwidth = 64)
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨18446744073709551615, …⟩ } }
  P5 = PUnit.unit
  P6 = { value := { toFin := ⟨1, …⟩ } }
  P7 = PUnit.unit
  P8 = PUnit.unit
  P9 = PUnit.unit
```

Veir.fshlGeneral32

Source

Builder families: `Veir.lowerFshlGeneral`.

Pattern expression:

```
Veir.lowerFshlGeneral 32
```

```
llvm.intr.fshl(X, Y, Z)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.sllw(builtin.unrealized_conversion_cast(X), new3),
riscv.srlw(riscv.srliw(builtin.unrealized_conversion_cast(Y)), riscv.xori(new3))))
where
  root = llvm.intr.fshl(X, Y, Z) : T3
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(Z) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.sllw(new1, new3) : T4 [created]
  P5 = properties(new5)
  new6 = riscv.srliw(new2) : T4 [created]
  P6 = properties(new6)
  new7 = riscv.srlw(new6, new4) : T4 [created]
  P7 = properties(new7)
  new8 = riscv.or(new5, new7) : T4 [created]
  P8 = properties(new8)
  new9 = builtin.unrealized_conversion_cast(new8) : T3 [created]
  P9 = properties(new9)
  X : T0
  Y : T1
  Z : T2
  T3 : Veir.IntegerType, where t = T3: decide (t.bitwidth = 32)
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨18446744073709551615, …⟩ } }
  P5 = PUnit.unit
  P6 = { value := { toFin := ⟨1, …⟩ } }
  P7 = PUnit.unit
  P8 = PUnit.unit
  P9 = PUnit.unit
```

Veir.fshrGeneral64

Source

Builder families: `Veir.lowerFshrGeneral`.

Pattern expression:

```
Veir.lowerFshrGeneral 64
```

```
llvm.intr.fshr(X, Y, Z)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.sll(riscv.slli(builtin.unrealized_conversion_cast(X)),
riscv.xori(new3)), riscv.srl(builtin.unrealized_conversion_cast(Y), new3)))
where
  root = llvm.intr.fshr(X, Y, Z) : T3
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(Z) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.slli(new1) : T4 [created]
  P5 = properties(new5)
  new6 = riscv.sll(new5, new4) : T4 [created]
  P6 = properties(new6)
  new7 = riscv.srl(new2, new3) : T4 [created]
  P7 = properties(new7)
  new8 = riscv.or(new6, new7) : T4 [created]
  P8 = properties(new8)
  new9 = builtin.unrealized_conversion_cast(new8) : T3 [created]
  P9 = properties(new9)
  X : T0
  Y : T1
  Z : T2
  T3 : Veir.IntegerType, where t = T3: decide (t.bitwidth = 64)
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨18446744073709551615, …⟩ } }
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
  P8 = PUnit.unit
  P9 = PUnit.unit
```

Veir.fshrGeneral32

Source

Builder families: `Veir.lowerFshrGeneral`.

Pattern expression:

```
Veir.lowerFshrGeneral 32
```

```
llvm.intr.fshr(X, Y, Z)  =>
builtin.unrealized_conversion_cast(riscv.or(riscv.sllw(riscv.slliw(builtin.unrealized_conversion_cast(X)),
riscv.xori(new3)), riscv.srlw(builtin.unrealized_conversion_cast(Y), new3)))
where
  root = llvm.intr.fshr(X, Y, Z) : T3
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T4 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T4 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(Z) : T4 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T4 [created]
  P4 = properties(new4)
  new5 = riscv.slliw(new1) : T4 [created]
  P5 = properties(new5)
  new6 = riscv.sllw(new5, new4) : T4 [created]
  P6 = properties(new6)
  new7 = riscv.srlw(new2, new3) : T4 [created]
  P7 = properties(new7)
  new8 = riscv.or(new6, new7) : T4 [created]
  P8 = properties(new8)
```

```
  new9 = builtin.unrealized_conversion_cast(new8) : T3 [created]
  P9 = properties(new9)
  X : T0
  Y : T1
  Z : T2
  T3 : Veir.IntegerType, where t = T3: decide (t.bitwidth = 32)
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T4 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨18446744073709551615, …⟩ } }
  P5 = { value := { toFin := ⟨1, …⟩ } }
  P6 = PUnit.unit
  P7 = PUnit.unit
  P8 = PUnit.unit
  P9 = PUnit.unit
```

Veir.poisonConst_pattern

Source

```
llvm.mlir.poison()  =>  builtin.unrealized_conversion_cast(riscv.li())
where
  root = llvm.mlir.poison() : T0
  P0 = properties(root)
  new1 = riscv.li() : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T0 [created]
  P2 = properties(new2)
  T0 : Veir.TypeAttr
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = { value := { toFin := ⟨0, …⟩ } }
  P2 = PUnit.unit
```

Veir.zeroConst_pattern

Source

```
llvm.mlir.zero()  =>  builtin.unrealized_conversion_cast(riscv.li())
where
  root = llvm.mlir.zero() : T0
  P0 = properties(root)
  new1 = riscv.li() : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T0 [created]
  P2 = properties(new2)
  T0 : Veir.TypeAttr, where t = T0: Option.any (fun x => decide (x ≤ 64)) (Veir.icmpTypeWidth? t)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = { value := { toFin := ⟨0, …⟩ } }
  P2 = PUnit.unit
```

Veir.freeze_pattern

Source

Builder families: `Veir.lowerRegCast`.

```
llvm.freeze(X)  =>  builtin.unrealized_conversion_cast(builtin.unrealized_conversion_cast(X))
where
  root = llvm.freeze(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T1 [created]
  P2 = properties(new2)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require match T0.val, T1.val with
  | Veir.Attribute.integerType opType, Veir.Attribute.integerType resType =>
    decide ((opType.bitwidth = 64 ∨ opType.bitwidth = 32) ∧ (resType.bitwidth = 64 ∨ resType.bitwidth = 32))
  | x, x_1 => false
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
```

```
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Veir.gmirAdd_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_add(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.add(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_add(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.add(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_add #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirSub_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_sub(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sub(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_sub(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.sub(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_sub #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirTrunc_pattern

Source

Builder families: `Veir.lowerRegCast`.

```
gmir.g_trunc(X)  =>  builtin.unrealized_conversion_cast(builtin.unrealized_conversion_cast(X))
where
  root = gmir.g_trunc(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T1 [created]
  P2 = properties(new2)
  X : T0
  T1 : Veir.TypeAttr
```

```
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_trunc #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Veir.gmirAnyext_pattern

Source

Builder families: `Veir.lowerRegCast`.

```
gmir.g_anyext(X)  =>  builtin.unrealized_conversion_cast(builtin.unrealized_conversion_cast(X))
where
  root = gmir.g_anyext(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T1 [created]
  P2 = properties(new2)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_anyext #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Veir.gmirAnd_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_and(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.and(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_and(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.and(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_and #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirOr_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_or(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.or(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_or(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.or(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
```

```
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_or #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirXor_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_xor(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.xor(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_xor(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.xor(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_xor #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirMul_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_mul(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.mul(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_mul(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.mul(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_mul #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirSdiv_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_sdiv(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.div(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_sdiv(X, Y) : T0
```

```
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.div(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_sdiv #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirUdiv_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_udiv(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.divu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_udiv(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.divu(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_udiv #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirSrem_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_srem(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.rem(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_srem(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.rem(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_srem #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirUrem_pattern

Source

Builder families: `Veir.lowerBinary`.

```
gmir.g_urem(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.remu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_urem(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P2 = properties(new2)
  new3 = riscv.remu(new1, new2) : T1 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 64)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_urem #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Veir.gmirConstant_pattern

Source

```
gmir.g_constant()  =>  builtin.unrealized_conversion_cast(riscv.li())
where
  root = gmir.g_constant() : T0
  P0 = properties(root)
  new1 = riscv.li() : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(new1) : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_constant #[T0] P0
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 ← match P0 with
  | { value := const } => some { value := BitVec.ofInt 64 const.value }
  P2 = PUnit.unit
```

Veir.gmirIcmp_pattern

Source

Variant 1: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.eq false`

```
gmir.g_icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.sltiu(riscv.xor(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X))))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.xor(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.sltiu(new3) : T2 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T1 [created]
  P5 = properties(new5)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.eq)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
```

```
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 2: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.eq true`

```
gmir.g_icmp(X, C0)  =>  builtin.unrealized_conversion_cast(riscv.sltiu(builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T0
  P0 = properties(C0)
  root = gmir.g_icmp(X, C0) : T1
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.sltiu(new2) : T2 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T1 [created]
  P5 = properties(new5)
  X : T0
  require decide (P1.predicate = Veir.Data.LLVM.IntPred.eq)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require decide (BitVec.ofInt P0.value.type.bitwidth P0.value.value = 0)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P1
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 3: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.ne false`

```
gmir.g_icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
riscv.xor(builtin.unrealized_conversion_cast(Y), builtin.unrealized_conversion_cast(X))))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.xor(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.li() : T2 [created]
  P4 = properties(new4)
  new5 = riscv.sltu(new4, new3) : T2 [created]
  P5 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T1 [created]
  P6 = properties(new6)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.ne)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨0, …⟩ } }
  P5 = PUnit.unit
  P6 = PUnit.unit
```

Variant 4: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.ne true`

```
gmir.g_icmp(X, C0)  =>  builtin.unrealized_conversion_cast(riscv.sltu(riscv.li(),
builtin.unrealized_conversion_cast(X)))
where
  C0 = gmir.g_constant() : T0
  P0 = properties(C0)
  root = gmir.g_icmp(X, C0) : T1
  P1 = properties(root)
  new2 = builtin.unrealized_conversion_cast(X) : T2 [created]
```

```
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(C0) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.li() : T2 [created]
  P4 = properties(new4)
  new5 = riscv.sltu(new4, new2) : T2 [created]
  P5 = properties(new5)
  new6 = builtin.unrealized_conversion_cast(new5) : T1 [created]
  P6 = properties(new6)
  X : T0
  require decide (P1.predicate = Veir.Data.LLVM.IntPred.ne)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require decide (BitVec.ofInt P0.value.type.bitwidth P0.value.value = 0)
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P1
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨0, …⟩ } }
  P5 = PUnit.unit
  P6 = PUnit.unit
```

Variant 5: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.slt false`

```
gmir.g_icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.slt(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new1, new2) : T2 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T1 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.slt)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 6: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.sgt false`

```
gmir.g_icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.slt(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T1 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.sgt)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 7: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.ult false`

```
gmir.g_icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new1, new2) : T2 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T1 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.ult)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 8: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.ugt false`

```
gmir.g_icmp(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.sltu(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X)))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T1 [created]
  P4 = properties(new4)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.ugt)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = PUnit.unit
```

Variant 9: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.sge false`

```
gmir.g_icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y))))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new1, new2) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T2 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T1 [created]
  P5 = properties(new5)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.sge)
  T1 : Veir.TypeAttr
```

```
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 10: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.sle false`

```
gmir.g_icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.slt(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X))))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.slt(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T2 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T1 [created]
  P5 = properties(new5)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.sle)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 11: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.uge false`

```
gmir.g_icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y))))
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new1, new2) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T2 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T1 [created]
  P5 = properties(new5)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.uge)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, …⟩ } }
  P5 = PUnit.unit
```

Variant 12: `Veir.gmirIcmp_pattern Veir.Data.LLVM.IntPred.ule false`

```
gmir.g_icmp(X, Y)  =>
builtin.unrealized_conversion_cast(riscv.xori(riscv.sltu(builtin.unrealized_conversion_cast(Y),
builtin.unrealized_conversion_cast(X))))
```

```
where
  root = gmir.g_icmp(X, Y) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T2 [created]
  P2 = properties(new2)
  new3 = riscv.sltu(new2, new1) : T2 [created]
  P3 = properties(new3)
  new4 = riscv.xori(new3) : T2 [created]
  P4 = properties(new4)
  new5 = builtin.unrealized_conversion_cast(new4) : T1 [created]
  P5 = properties(new5)
  X : T0
  Y : T0
  require decide (P0.predicate = Veir.Data.LLVM.IntPred.ule)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.riscv64LegalizerInfo.isLegal Veir.GMIR.g_icmp #[T1, T0] P0
  T2 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
  P3 = PUnit.unit
  P4 = { value := { toFin := ⟨1, ⋯⟩ } }
  P5 = PUnit.unit
```

LLVM to GMIR

Veir.Passes.LLVMToGMIR

Source module

Veir.g_add

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.add Veir.GMIR.g_add Veir.g_add._proof_1
```

```
llvm.add(X, Y)  =>  gmir.g_add(X, Y)
where
  root = llvm.add(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_add(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_add._proof_1 P0)
```

Veir.g_sub

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.sub Veir.GMIR.g_sub Veir.g_sub._proof_1
```

```
llvm.sub(X, Y)  =>  gmir.g_sub(X, Y)
where
  root = llvm.sub(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_sub(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
```

```
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_sub._proof_1 P0)
```

Veir.g_icmp

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.icmp Veir.GMIR.g_icmp Veir.g_icmp._proof_1
```

```
llvm.icmp(X, Y)  =>  gmir.g_icmp(X, Y)
where
  root = llvm.icmp(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_icmp(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_icmp._proof_1 P0)
```

Veir.g_and

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.and Veir.GMIR.g_and Veir.g_and._proof_1
```

```
llvm.and(X, Y)  =>  gmir.g_and(X, Y)
where
  root = llvm.and(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_and(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_and._proof_1 P0)
```

Veir.g_or

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.or Veir.GMIR.g_or Veir.g_or._proof_1
```

```
llvm.or(X, Y)  =>  gmir.g_or(X, Y)
where
  root = llvm.or(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_or(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_or._proof_1 P0)
```

Veir.g_xor

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.xor Veir.GMIR.g_xor Veir.g_xor._proof_1
```

```
llvm.xor(X, Y)  =>  gmir.g_xor(X, Y)
where
  root = llvm.xor(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_xor(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_xor._proof_1 P0)
```

Veir.g_mul

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.mul Veir.GMIR.g_mul Veir.g_mul._proof_1
```

```
llvm.mul(X, Y)  =>  gmir.g_mul(X, Y)
where
  root = llvm.mul(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_mul(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_mul._proof_1 P0)
```

Veir.g_sdiv

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.sdiv Veir.GMIR.g_sdiv Veir.g_sdiv._proof_1
```

```
llvm.sdiv(X, Y)  =>  gmir.g_sdiv(X, Y)
where
  root = llvm.sdiv(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_sdiv(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_sdiv._proof_1 P0)
```

Veir.g_udiv

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.udiv Veir.GMIR.g_udiv Veir.g_udiv._proof_1
```

```
llvm.udiv(X, Y)  =>  gmir.g_udiv(X, Y)
where
  root = llvm.udiv(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_udiv(X, Y) : T2 [created]
  P1 = properties(new1)
```

```
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_udiv._proof_1 P0)
```

Veir.g_srem

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.srem Veir.GMIR.g_srem Veir.g_srem._proof_1
```

```
llvm.srem(X, Y)  =>  gmir.g_srem(X, Y)
where
  root = llvm.srem(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_srem(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_srem._proof_1 P0)
```

Veir.g_urem

Source

Builder families: `Veir.lowerBinop`.

Pattern expression:

```
Veir.lowerBinop Veir.Llvm.urem Veir.GMIR.g_urem Veir.g_urem._proof_1
```

```
llvm.urem(X, Y)  =>  gmir.g_urem(X, Y)
where
  root = llvm.urem(X, Y) : T2
  P0 = properties(root)
  new1 = gmir.g_urem(X, Y) : T2 [created]
  P1 = properties(new1)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  P1 ← some (cast Veir.g_urem._proof_1 P0)
```

Veir.g_sext

Source

Builder families: `Veir.lowerIntegerCast`.

Pattern expression:

```
Veir.lowerIntegerCast Veir.Llvm.sext Veir.GMIR.g_sext Veir.g_sext._proof_1
```

```
llvm.sext(X)  =>  gmir.g_sext(X)
where
  root = llvm.sext(X) : T1
  P0 = properties(root)
  new1 = gmir.g_sext(X) : T1 [created]
  P1 = properties(new1)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  P1 ← some (cast Veir.g_sext._proof_1 P0)
```

Veir.g_zext

Source

Builder families: `Veir.lowerIntegerCast`.

Pattern expression:

```
Veir.lowerIntegerCast Veir.Llvm.zext Veir.GMIR.g_zext Veir.g_zext._proof_1
```

```
llvm.zext(X)  =>  gmir.g_zext(X)
where
  root = llvm.zext(X) : T1
  P0 = properties(root)
  new1 = gmir.g_zext(X) : T1 [created]
  P1 = properties(new1)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  P1 ← some (cast Veir.g_zext._proof_1 P0)
```

Veir.g_trunc

Source

Builder families: `Veir.lowerIntegerCast`.

Pattern expression:

```
Veir.lowerIntegerCast Veir.Llvm.trunc Veir.GMIR.g_trunc Veir.g_trunc._proof_1
```

```
llvm.trunc(X)  =>  gmir.g_trunc(X)
where
  root = llvm.trunc(X) : T1
  P0 = properties(root)
  new1 = gmir.g_trunc(X) : T1 [created]
  P1 = properties(new1)
  X : T0
  T1 : Veir.IntegerType
  T0 : Veir.IntegerType
  P1 ← some (cast Veir.g_trunc._proof_1 P0)
```

Veir.lowerIntegerConstant

Source

```
llvm.mlir.constant()  =>  C1
where
  root = llvm.mlir.constant() : T0
  P0 = properties(root)
  C1 = gmir.g_constant() : T0 [created]
  P1 = properties(C1)
  require match P0.value with
  | Veir.LLVMConstantValue.integer value => true
  | x => false
  T0 : Veir.IntegerType
  P1 ← match P0.value with
  | Veir.LLVMConstantValue.integer value => some { value := value }
  | x => none
```

Veir.g_truncByte

Source

Builder families: `Veir.lowerUnaryByteOp`.

Pattern expression:

```
Veir.lowerUnaryByteOp Veir.Llvm.trunc Veir.GMIR.g_trunc Veir.g_trunc._proof_1
```

```
llvm.trunc(X)  =>  builtin.unrealized_conversion_cast(gmir.g_trunc(builtin.unrealized_conversion_cast(X)))
where
  root = llvm.trunc(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T2 [created]
  P1 = properties(new1)
  new2 = gmir.g_trunc(new1) : T3 [created]
  P2 = properties(new2)
  new3 = builtin.unrealized_conversion_cast(new2) : T1 [created]
  P1 = properties(new3)
  X : T0
```

```
  T1 : Veir.LLVM.ByteType
  T0 : Veir.LLVM.ByteType
  T2 ← do
    let type ← T0.cast? Veir.LLVM.ByteType
    pure (Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless type.bitwidth))
  T3 ← do
    let type ← T1.cast? Veir.LLVM.ByteType
    pure (Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless type.bitwidth))
  P1 = PUnit.unit
  P2 ← some (cast Veir.g_trunc._proof_1 P0)
```

Veir.lowerByteIntBitcast

Source

```
llvm.bitcast(X)  =>  builtin.unrealized_conversion_cast(X)
where
  root = llvm.bitcast(X) : T1
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  X : T0
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  require Veir.isByteIntBitcast T0 T1
  P1 = PUnit.unit
```

Legalization

Veir.Passes.Legalization.LegalizerHelper

Source module

Veir.widenTypeGroup

Source

Variant 1: `Veir.widenTypeGroup Veir.GMIR.g_add (Veir.TypeGroup.type 0) Veir.GMIR.g_anyext width Veir.clearNswNuwProperties; width : Nat`

```
gmir: g_add(X, Y)  =>  g_trunc(g_add(g_anyext(X), g_anyext(Y)))
where
  root = g_add(X, Y) : T2
  P0 = properties(root)
  new1 = g_anyext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_anyext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_add(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (Veir.clearNswNuwProperties P0)
  P3 = { nsw := false, nuw := false }
```

Variant 2: `Veir.widenTypeGroup Veir.GMIR.g_sub (Veir.TypeGroup.type 0) Veir.GMIR.g_anyext width Veir.clearNswNuwProperties; width : Nat`

```
gmir: g_sub(X, Y)  =>  g_trunc(g_sub(g_anyext(X), g_anyext(Y)))
where
  root = g_sub(X, Y) : T2
  P0 = properties(root)
  new1 = g_anyext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_anyext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_sub(new1, new2) : T3 [created]
```

```
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (Veir.clearNswNuwProperties P0)
  P3 = { nsw := false, nuw := false }
```

Variant 3: `Veir.widenTypeGroup Veir.GMIR.g_icmp (Veir.TypeGroup.type 0) Veir.GMIR.g_anyext width; width : Nat`

```
gmir: g_icmp(X, Y)  =>  g_trunc(new1)
where
  root = g_icmp(X, Y) : T2
  P0 = properties(root)
  new1 = g_icmp(X, Y) : T3 [created]
  P2 = properties(new1)
  new2 = g_trunc(new1) : T2 [created]
  P3 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.TypeAttr
  T0 : Veir.TypeAttr
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 4: `Veir.widenTypeGroup Veir.GMIR.g_icmp (Veir.TypeGroup.type 1) Veir.GMIR.g_sext width; width : Nat`

```
gmir: g_icmp(X, Y)  =>  g_icmp(g_sext(X), g_sext(Y))
where
  root = g_icmp(X, Y) : T2
  P0 = properties(root)
  new1 = g_sext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_sext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_icmp(new1, new2) : T2 [created]
  P2 = properties(new3)
  X : T0
  Y : T1
  T2 : Veir.TypeAttr
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 5: `Veir.widenTypeGroup Veir.GMIR.g_and (Veir.TypeGroup.type 0) Veir.GMIR.g_anyext width; width : Nat`

```
gmir: g_and(X, Y)  =>  g_trunc(g_and(g_anyext(X), g_anyext(Y)))
where
  root = g_and(X, Y) : T2
  P0 = properties(root)
  new1 = g_anyext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_anyext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_and(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
```

```
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 6: `Veir.widenTypeGroup Veir.GMIR.g_or (Veir.TypeGroup.type 0) Veir.GMIR.g_anyext width; width : Nat`

```
gmir: g_or(X, Y)  =>  g_trunc(g_or(g_anyext(X), g_anyext(Y)))
where
  root = g_or(X, Y) : T2
  P0 = properties(root)
  new1 = g_anyext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_anyext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_or(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 7: `Veir.widenTypeGroup Veir.GMIR.g_xor (Veir.TypeGroup.type 0) Veir.GMIR.g_anyext width; width : Nat`

```
gmir: g_xor(X, Y)  =>  g_trunc(g_xor(g_anyext(X), g_anyext(Y)))
where
  root = g_xor(X, Y) : T2
  P0 = properties(root)
  new1 = g_anyext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_anyext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_xor(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 8: `Veir.widenTypeGroup Veir.GMIR.g_mul (Veir.TypeGroup.type 0) Veir.GMIR.g_anyext width Veir.clearNswNuwProperties; width : Nat`

```
gmir: g_mul(X, Y)  =>  g_trunc(g_mul(g_anyext(X), g_anyext(Y)))
where
  root = g_mul(X, Y) : T2
  P0 = properties(root)
  new1 = g_anyext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_anyext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_mul(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (Veir.clearNswNuwProperties P0)
  P3 = { nsw := false, nuw := false }
```

Variant 9: `Veir.widenTypeGroup Veir.GMIR.g_sdiv (Veir.TypeGroup.type 0) Veir.GMIR.g_sext width; width : Nat`

```
gmir: g_sdiv(X, Y)  =>  g_trunc(g_sdiv(g_sext(X), g_sext(Y)))
where
  root = g_sdiv(X, Y) : T2
  P0 = properties(root)
  new1 = g_sext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_sext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_sdiv(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 10: `Veir.widenTypeGroup Veir.GMIR.g_srem (Veir.TypeGroup.type 0) Veir.GMIR.g_sext width; width : Nat`

```
gmir: g_srem(X, Y)  =>  g_trunc(g_srem(g_sext(X), g_sext(Y)))
where
  root = g_srem(X, Y) : T2
  P0 = properties(root)
  new1 = g_sext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_sext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_srem(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = PUnit.unit
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 11: `Veir.widenTypeGroup Veir.GMIR.g_udiv (Veir.TypeGroup.type 0) Veir.GMIR.g_zext width; width : Nat`

```
gmir: g_udiv(X, Y)  =>  g_trunc(g_udiv(g_zext(X), g_zext(Y)))
where
  root = g_udiv(X, Y) : T2
  P0 = properties(root)
  new1 = g_zext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_zext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_udiv(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = { nneg := false }
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Variant 12: `Veir.widenTypeGroup Veir.GMIR.g_urem (Veir.TypeGroup.type 0) Veir.GMIR.g_zext width; width : Nat`

```
gmir: g_urem(X, Y)  =>  g_trunc(g_urem(g_zext(X), g_zext(Y)))
where
```

```
  root = g_urem(X, Y) : T2
  P0 = properties(root)
  new1 = g_zext(X) : T3 [created]
  P1 = properties(new1)
  new2 = g_zext(Y) : T3 [created]
  P1 = properties(new2)
  new3 = g_urem(new1, new2) : T3 [created]
  P2 = properties(new3)
  new4 = g_trunc(new3) : T2 [created]
  P3 = properties(new4)
  X : T0
  Y : T1
  T2 : Veir.IntegerType, where t = T2: decide (t.bitwidth < width)
  T1 : Veir.IntegerType, where t = T1: decide (t.bitwidth < width)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T3 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 = { nneg := false }
  P2 ← some (id P0)
  P3 = { nsw := false, nuw := false }
```

Veir.widenConstant

Source

Parameters: `width : Nat`

```
gmir: g_constant()  =>  g_trunc(C1)
where
  root = g_constant() : T0
  P0 = properties(root)
  C1 = g_constant() : T1 [created]
  P1 = properties(C1)
  new2 = g_trunc(C1) : T0 [created]
  P2 = properties(new2)
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth < width)
  T1 = Veir.TypeAttr.of Veir.IntegerType (Veir.IntegerType.signless width)
  P1 ← match P0 with
  | { value := { value := value, type := { bitwidth := w, signedness := signedness } } } =>
    some
      {
        value :=
          Veir.IntegerAttr.ofInt (if w % 8 = 0 then value else ↑(BitVec.ofInt w value).toNat)
            (Veir.IntegerType.signless width) }
  P2 = { nsw := false, nuw := false }
```

Veir.Passes.Legalization.RISCV64LegalizerInfo

Source module

Veir.customLegalizeWord

Source

Variant 1: `Veir.customLegalizeWord Veir.GMIR.g_add Veir.Riscv.addw ()`

```
gmir.g_add(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.addw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_add(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P1 = properties(new2)
  new3 = riscv.addw(new1, new2) : T1 [created]
  P2 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P1 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Variant 2: `Veir.customLegalizeWord Veir.GMIR.g_sub Veir.Riscv.subw ()`

```
gmir.g_sub(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.subw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_sub(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P1 = properties(new2)
  new3 = riscv.subw(new1, new2) : T1 [created]
  P2 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P1 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Variant 3: `Veir.customLegalizeWord Veir.GMIR.g_mul Veir.Riscv.mulw ()`

```
gmir.g_mul(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.mulw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_mul(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P1 = properties(new2)
  new3 = riscv.mulw(new1, new2) : T1 [created]
  P2 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P1 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Variant 4: `Veir.customLegalizeWord Veir.GMIR.g_sdiv Veir.Riscv.divw ()`

```
gmir.g_sdiv(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.divw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_sdiv(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P1 = properties(new2)
  new3 = riscv.divw(new1, new2) : T1 [created]
  P2 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P1 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Variant 5: `Veir.customLegalizeWord Veir.GMIR.g_udiv Veir.Riscv.divuw ()`

```
gmir.g_udiv(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.divuw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_udiv(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P1 = properties(new2)
  new3 = riscv.divuw(new1, new2) : T1 [created]
```

```
  P2 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P1 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

Variant 6: `Veir.customLegalizeWord Veir.GMIR.g_urem Veir.Riscv.remuw ()`

```
gmir.g_urem(X, Y)  =>  builtin.unrealized_conversion_cast(riscv.remuw(builtin.unrealized_conversion_cast(X),
builtin.unrealized_conversion_cast(Y)))
where
  root = gmir.g_urem(X, Y) : T0
  P0 = properties(root)
  new1 = builtin.unrealized_conversion_cast(X) : T1 [created]
  P1 = properties(new1)
  new2 = builtin.unrealized_conversion_cast(Y) : T1 [created]
  P1 = properties(new2)
  new3 = riscv.remuw(new1, new2) : T1 [created]
  P2 = properties(new3)
  new4 = builtin.unrealized_conversion_cast(new3) : T0 [created]
  P1 = properties(new4)
  X : T0
  Y : T0
  T0 : Veir.IntegerType, where t = T0: decide (t.bitwidth = 32)
  T1 = Veir.TypeAttr.of Veir.RegisterType { }
  P1 = PUnit.unit
  P2 = PUnit.unit
```

RISC-V combines

Veir.Passes.RISCVCombines.Combine

Source module

Veir.RISCV.right_identity_zero_add_pattern

Source

```
riscv: add(X, li())  =>  X
where
  a0 = li() : T0
  P0 = properties(a0)
  root = add(X, a0) : T0
  P1 = properties(root)
  X : T0
  require P0.value == 0#64
  T0 : Veir.RegisterType, where t = T0: t.index.isNone
```

Veir.RISCV.zextw_zextb_pattern

Source

```
riscv: zextw(a0)  =>  a0
where
  a0 = zextb(X) : T1
  P0 = properties(a0)
  root = zextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.zextb_lbu_pattern

Source

```
riscv: zextb(a0)  =>  a0
where
  a0 = lbu(X) : T1
  P0 = properties(a0)
```

```
  root = zextb(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_lw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.lw 1 (fun x => true) Veir.RISCV.sextw_lw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = lw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_lh

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.lh 1 (fun x => true) Veir.RISCV.sextw_lh._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = lh(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_lb

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.lb 1 (fun x => true) Veir.RISCV.sextw_lb._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = lb(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_lhu

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.lhu 1 (fun x => true) Veir.RISCV.sextw_lhu._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
```

```
  a0 = lhu(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_lbu

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.lbu 1 (fun x => true) Veir.RISCV.sextw_lbu._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = lbu(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_addw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.addw 2 (fun x => true) Veir.RISCV.sextw_addw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = addw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_addiw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.addiw 1 (fun x => true) Veir.RISCV.sextw_addiw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = addiw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_subw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.subw 2 (fun x => true) Veir.RISCV.sextw_subw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = subw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_mulw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.mulw 2 (fun x => true) Veir.RISCV.sextw_mulw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = mulw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_divw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.divw 2 (fun x => true) Veir.RISCV.sextw_divw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = divw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_divuw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.divuw 2 (fun x => true) Veir.RISCV.sextw_divuw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = divuw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
```

```
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_remw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.remw 2 (fun x => true) Veir.RISCV.sextw_remw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = remw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_remuw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.remuw 2 (fun x => true) Veir.RISCV.sextw_remuw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = remuw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_sllw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.sllw 2 (fun x => true) Veir.RISCV.sextw_sllw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = sllw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_slliw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.slliw 1 (fun x => true) Veir.RISCV.sextw_slliw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = slliw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_srlw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.srlw 2 (fun x => true) Veir.RISCV.sextw_srlw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = srlw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_srliw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.srliw 1 (fun x => true) Veir.RISCV.sextw_srliw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = srliw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_sraw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.sraw 2 (fun x => true) Veir.RISCV.sextw_sraw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = sraw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_sraiw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.sraiw 1 (fun x => true) Veir.RISCV.sextw_sraiw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = sraiw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_rolw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.rolw 2 (fun x => true) Veir.RISCV.sextw_rolw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = rolw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_rorw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.rorw 2 (fun x => true) Veir.RISCV.sextw_rorw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = rorw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_roriw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.roriw 1 (fun x => true) Veir.RISCV.sextw_roriw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = roriw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
```

```
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_packw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.packw 2 (fun x => true) Veir.RISCV.sextw_packw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = packw(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_lui

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.lui 0 (fun x => true) Veir.RISCV.sextw_lui._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = lui() : T0
  P0 = properties(a0)
  root = sextw(a0) : T0
  P1 = properties(root)
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_cpop

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.cpop 1 (fun x => true) Veir.RISCV.sextw_cpop._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = cpop(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_cpopw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.cpopw 1 (fun x => true) Veir.RISCV.sextw_cpopw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = cpopw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
```

```
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_clz

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.clz 1 (fun x => true) Veir.RISCV.sextw_clz._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = clz(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_clzw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.clzw 1 (fun x => true) Veir.RISCV.sextw_clzw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = clzw(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_ctz

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.ctz 1 (fun x => true) Veir.RISCV.sextw_ctz._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = ctz(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_ctzw

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.ctzw 1 (fun x => true) Veir.RISCV.sextw_ctzw._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = ctzw(X) : T1
```

```
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_slt

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.slt 2 (fun x => true) Veir.RISCV.sextw_slt._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = slt(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_sltu

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.sltu 2 (fun x => true) Veir.RISCV.sextw_sltu._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = sltu(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_slti

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.slti 1 (fun x => true) Veir.RISCV.sextw_slti._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = slti(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_sltiu

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.sltiu 1 (fun x => true) Veir.RISCV.sextw_sltiu._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = sltiu(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_bext

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.bext 2 (fun x => true) Veir.RISCV.sextw_bext._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = bext(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_bexti

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.bexti 1 (fun x => true) Veir.RISCV.sextw_bexti._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = bexti(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_sextb

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.sextb 1 (fun x => true) Veir.RISCV.sextw_sextb._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = sextb(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_sexth

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.sexth 1 (fun x => true) Veir.RISCV.sextw_sexth._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = sexth(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_zexth

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.zexth 1 (fun x => true) Veir.RISCV.sextw_zexth._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = zexth(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_packh

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.packh 2 (fun x => true) Veir.RISCV.sextw_packh._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = packh(X, Y) : T2
  P0 = properties(a0)
  root = sextw(a0) : T2
  P1 = properties(root)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_srai

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.srai 1
  (fun p => decide (32#6 ≤ Veir.RISCVImmediateProperties.immField p 6)) Veir.RISCV.sextw_srai._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = srai(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  require decide (32#6 ≤ Veir.RISCVImmediateProperties.immField P0 6)
```

```
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_srli

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.srli 1
  (fun p => decide (32#6 < Veir.RISCVImmediateProperties.immField p 6)) Veir.RISCV.sextw_srli._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = srli(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  require decide (32#6 < Veir.RISCVImmediateProperties.immField P0 6)
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_andi

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.andi 1
  (fun p => !(Veir.RISCVImmediateProperties.immField p 12).msb) Veir.RISCV.sextw_andi._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = andi(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  require !(Veir.RISCVImmediateProperties.immField P0 12).msb
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.sextw_ori

Source

Builder families: `Veir.RISCV.sextw_signExtendingOpW_pattern`.

Pattern expression:

```
Veir.RISCV.sextw_signExtendingOpW_pattern Veir.Riscv.ori 1 (fun p => (Veir.RISCVImmediateProperties.immField p
12).msb)
  Veir.RISCV.sextw_ori._proof_1
```

```
riscv: sextw(a0)  =>  a0
where
  a0 = ori(X) : T1
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  X : T0
  require (Veir.RISCVImmediateProperties.immField P0 12).msb
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.zextw_slliw_lbu_pattern

Source

Parameters: `shamt : BitVec 64`

```
riscv: zextw(slliw(a0))  =>  slli(a0)
where
```

```
  a0 = lbu(X) : T1
  P0 = properties(a0)
  a1 = slliw(a0) : T2
  P1 = properties(a1)
  root = zextw(a1) : T3
  P2 = properties(root)
  new3 = slli(a0) : T3 [created]
  P3 = properties(new3)
  X : T0
  T3 : Veir.RegisterType
  require P1.value == shamt
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
  P3 = { value := shamt }
```

Veir.RISCV.packh_low_bytes_pattern

Source

Variant 1: `commuted = false`

```
riscv: or(a0, slli(a1))  =>  packh(a0, a1)
where
  a0 = lbu(X) : T1
  P0 = properties(a0)
  a1 = lbu(Y) : T3
  P1 = properties(a1)
  a2 = slli(a1) : T4
  P2 = properties(a2)
  root = or(a0, a2) : T5
  P3 = properties(root)
  new4 = packh(a0, a1) : T5 [created]
  P4 = properties(new4)
  X : T0
  Y : T2
  T5 : Veir.RegisterType
  require P2.value == 8#64
  T4 : Veir.RegisterType
  T3 : Veir.RegisterType
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
  P4 = PUnit.unit
```

Variant 2: `commuted = true`

```
riscv: or(slli(a1), a0)  =>  packh(a0, a1)
where
  a0 = lbu(X) : T1
  P0 = properties(a0)
  a1 = lbu(Y) : T3
  P1 = properties(a1)
  a2 = slli(a1) : T4
  P2 = properties(a2)
  root = or(a2, a0) : T5
  P3 = properties(root)
  new4 = packh(a0, a1) : T5 [created]
  P4 = properties(new4)
  X : T0
  Y : T2
  T5 : Veir.RegisterType
  require P2.value == 8#64
  T4 : Veir.RegisterType
  T3 : Veir.RegisterType
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
  P4 = PUnit.unit
```

Veir.RISCV.packh_high_bytes_pattern

Source

Variant 1: `commuted = false`

```
riscv: or(slli(a0), slli(a1))  =>  slli(packh(a0, a1))
where
  a0 = lbu(X) : T1
  P0 = properties(a0)
  a1 = lbu(Y) : T3
  P1 = properties(a1)
  a2 = slli(a0) : T4
  P2 = properties(a2)
  a3 = slli(a1) : T5
  P3 = properties(a3)
  root = or(a2, a3) : T6
  P4 = properties(root)
  new5 = packh(a0, a1) : T6 [created]
  P5 = properties(new5)
  new6 = slli(new5) : T6 [created]
  P6 = properties(new6)
  X : T0
  Y : T2
  T6 : Veir.RegisterType
  require P3.value == 24#64
  T5 : Veir.RegisterType
  require P2.value == 16#64
  T4 : Veir.RegisterType
  T3 : Veir.RegisterType
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
  P5 = PUnit.unit
  P6 = { value := { toFin := ⟨16, …⟩ } }
```

Variant 2: `commuted = true`

```
riscv: or(slli(a1), slli(a0))  =>  slli(packh(a0, a1))
where
  a0 = lbu(X) : T1
  P0 = properties(a0)
  a1 = lbu(Y) : T3
  P1 = properties(a1)
  a2 = slli(a0) : T4
  P2 = properties(a2)
  a3 = slli(a1) : T5
  P3 = properties(a3)
  root = or(a3, a2) : T6
  P4 = properties(root)
  new5 = packh(a0, a1) : T6 [created]
  P5 = properties(new5)
  new6 = slli(new5) : T6 [created]
  P6 = properties(new6)
  X : T0
  Y : T2
  T6 : Veir.RegisterType
  require P3.value == 24#64
  T5 : Veir.RegisterType
  require P2.value == 16#64
  T4 : Veir.RegisterType
  T3 : Veir.RegisterType
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
  P5 = PUnit.unit
  P6 = { value := { toFin := ⟨16, …⟩ } }
```

Veir.RISCV.drop_zextw_addw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: addw(zextw(X), Y)  =>  addw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = addw(a0, Y) : T2
  P1 = properties(root)
  new2 = addw(X, Y) : T2 [created]
```

```
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: addw(Y, zextw(X))  =>  addw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = addw(Y, a0) : T2
  P1 = properties(root)
  new2 = addw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_addiw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: addiw(zextw(X))  =>  addiw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = addiw(a0) : T1
  P1 = properties(root)
  new2 = addiw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_roriw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: roriw(zextw(X))  =>  roriw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = roriw(a0) : T1
  P1 = properties(root)
  new2 = roriw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_srliw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: srliw(zextw(X))  =>  srliw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = srliw(a0) : T1
  P1 = properties(root)
  new2 = srliw(X) : T1 [created]
  P1 = properties(new2)
```

```
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_subw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: subw(zextw(X), Y)  =>  subw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = subw(a0, Y) : T2
  P1 = properties(root)
  new2 = subw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: subw(Y, zextw(X))  =>  subw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = subw(Y, a0) : T2
  P1 = properties(root)
  new2 = subw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_mulw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: mulw(zextw(X), Y)  =>  mulw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = mulw(a0, Y) : T2
  P1 = properties(root)
  new2 = mulw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: mulw(Y, zextw(X))  =>  mulw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = mulw(Y, a0) : T2
  P1 = properties(root)
  new2 = mulw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
```

```
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_divw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: divw(zextw(X), Y)  =>  divw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = divw(a0, Y) : T2
  P1 = properties(root)
  new2 = divw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: divw(Y, zextw(X))  =>  divw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = divw(Y, a0) : T2
  P1 = properties(root)
  new2 = divw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_divuw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: divuw(zextw(X), Y)  =>  divuw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = divuw(a0, Y) : T2
  P1 = properties(root)
  new2 = divuw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: divuw(Y, zextw(X))  =>  divuw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = divuw(Y, a0) : T2
  P1 = properties(root)
  new2 = divuw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_remw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: remw(zextw(X), Y)  =>  remw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = remw(a0, Y) : T2
  P1 = properties(root)
  new2 = remw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: remw(Y, zextw(X))  =>  remw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = remw(Y, a0) : T2
  P1 = properties(root)
  new2 = remw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_remuw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: remuw(zextw(X), Y)  =>  remuw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = remuw(a0, Y) : T2
  P1 = properties(root)
  new2 = remuw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: remuw(Y, zextw(X))  =>  remuw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = remuw(Y, a0) : T2
  P1 = properties(root)
  new2 = remuw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_sllw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: sllw(zextw(X), Y)  =>  sllw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = sllw(a0, Y) : T2
  P1 = properties(root)
  new2 = sllw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: sllw(Y, zextw(X))  =>  sllw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = sllw(Y, a0) : T2
  P1 = properties(root)
  new2 = sllw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_srlw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: srlw(zextw(X), Y)  =>  srlw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = srlw(a0, Y) : T2
  P1 = properties(root)
  new2 = srlw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: srlw(Y, zextw(X))  =>  srlw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = srlw(Y, a0) : T2
  P1 = properties(root)
  new2 = srlw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_sraw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: sraw(zextw(X), Y)  =>  sraw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = sraw(a0, Y) : T2
  P1 = properties(root)
  new2 = sraw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: sraw(Y, zextw(X))  =>  sraw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = sraw(Y, a0) : T2
  P1 = properties(root)
  new2 = sraw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_rolw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: rolw(zextw(X), Y)  =>  rolw(X, Y)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = rolw(a0, Y) : T2
  P1 = properties(root)
  new2 = rolw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: rolw(Y, zextw(X))  =>  rolw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = rolw(Y, a0) : T2
  P1 = properties(root)
  new2 = rolw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_rorw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: rorw(zextw(X), Y)  =>  rorw(X, Y)
where
```

```
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = rorw(a0, Y) : T2
  P1 = properties(root)
  new2 = rorw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: rorw(Y, zextw(X))  =>  rorw(Y, X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = rorw(Y, a0) : T2
  P1 = properties(root)
  new2 = rorw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_sraiw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: sraiw(zextw(X))  =>  sraiw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = sraiw(a0) : T1
  P1 = properties(root)
  new2 = sraiw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_slliuw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: slliuw(zextw(X))  =>  slliuw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = slliuw(a0) : T1
  P1 = properties(root)
  new2 = slliuw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_clzw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: clzw(zextw(X))  =>  clzw(X)
where
  a0 = zextw(X) : T0
```

```
  P0 = properties(a0)
  root = clzw(a0) : T1
  P1 = properties(root)
  new2 = clzw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_ctzw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: ctzw(zextw(X))  =>  ctzw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = ctzw(a0) : T1
  P1 = properties(root)
  new2 = ctzw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_cpopw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: cpopw(zextw(X))  =>  cpopw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = cpopw(a0) : T1
  P1 = properties(root)
  new2 = cpopw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_zextw_sextw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: sextw(zextw(X))  =>  sextw(X)
where
  a0 = zextw(X) : T0
  P0 = properties(a0)
  root = sextw(a0) : T1
  P1 = properties(root)
  new2 = sextw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_addw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: addw(sextw(X), Y)  =>  addw(X, Y)
where
  a0 = sextw(X) : T0
```

```
  P0 = properties(a0)
  root = addw(a0, Y) : T2
  P1 = properties(root)
  new2 = addw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: addw(Y, sextw(X))  =>  addw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = addw(Y, a0) : T2
  P1 = properties(root)
  new2 = addw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_addiw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: addiw(sextw(X))  =>  addiw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = addiw(a0) : T1
  P1 = properties(root)
  new2 = addiw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_roriw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: roriw(sextw(X))  =>  roriw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = roriw(a0) : T1
  P1 = properties(root)
  new2 = roriw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_srliw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: srliw(sextw(X))  =>  srliw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
```

```
  root = srliw(a0) : T1
  P1 = properties(root)
  new2 = srliw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_subw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: subw(sextw(X), Y)  =>  subw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = subw(a0, Y) : T2
  P1 = properties(root)
  new2 = subw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: subw(Y, sextw(X))  =>  subw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = subw(Y, a0) : T2
  P1 = properties(root)
  new2 = subw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_mulw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: mulw(sextw(X), Y)  =>  mulw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = mulw(a0, Y) : T2
  P1 = properties(root)
  new2 = mulw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: mulw(Y, sextw(X))  =>  mulw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = mulw(Y, a0) : T2
  P1 = properties(root)
  new2 = mulw(Y, X) : T2 [created]
  P1 = properties(new2)
```

```
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_divw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: divw(sextw(X), Y)  =>  divw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = divw(a0, Y) : T2
  P1 = properties(root)
  new2 = divw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: divw(Y, sextw(X))  =>  divw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = divw(Y, a0) : T2
  P1 = properties(root)
  new2 = divw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_divuw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: divuw(sextw(X), Y)  =>  divuw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = divuw(a0, Y) : T2
  P1 = properties(root)
  new2 = divuw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: divuw(Y, sextw(X))  =>  divuw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = divuw(Y, a0) : T2
  P1 = properties(root)
  new2 = divuw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
```

```
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_remw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: remw(sextw(X), Y)  =>  remw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = remw(a0, Y) : T2
  P1 = properties(root)
  new2 = remw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: remw(Y, sextw(X))  =>  remw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = remw(Y, a0) : T2
  P1 = properties(root)
  new2 = remw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_remuw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: remuw(sextw(X), Y)  =>  remuw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = remuw(a0, Y) : T2
  P1 = properties(root)
  new2 = remuw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: remuw(Y, sextw(X))  =>  remuw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = remuw(Y, a0) : T2
  P1 = properties(root)
  new2 = remuw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
```

```
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_sllw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: sllw(sextw(X), Y)  =>  sllw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = sllw(a0, Y) : T2
  P1 = properties(root)
  new2 = sllw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: sllw(Y, sextw(X))  =>  sllw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = sllw(Y, a0) : T2
  P1 = properties(root)
  new2 = sllw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_srlw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: srlw(sextw(X), Y)  =>  srlw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = srlw(a0, Y) : T2
  P1 = properties(root)
  new2 = srlw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: srlw(Y, sextw(X))  =>  srlw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = srlw(Y, a0) : T2
  P1 = properties(root)
  new2 = srlw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_sraw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: sraw(sextw(X), Y)  =>  sraw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = sraw(a0, Y) : T2
  P1 = properties(root)
  new2 = sraw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: sraw(Y, sextw(X))  =>  sraw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = sraw(Y, a0) : T2
  P1 = properties(root)
  new2 = sraw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_rolw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: rolw(sextw(X), Y)  =>  rolw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = rolw(a0, Y) : T2
  P1 = properties(root)
  new2 = rolw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: rolw(Y, sextw(X))  =>  rolw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = rolw(Y, a0) : T2
  P1 = properties(root)
  new2 = rolw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_rorw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Variant 1: `index = 0`

```
riscv: rorw(sextw(X), Y)  =>  rorw(X, Y)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = rorw(a0, Y) : T2
  P1 = properties(root)
  new2 = rorw(X, Y) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Variant 2: `index = 1`

```
riscv: rorw(Y, sextw(X))  =>  rorw(Y, X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = rorw(Y, a0) : T2
  P1 = properties(root)
  new2 = rorw(Y, X) : T2 [created]
  P1 = properties(new2)
  X : T0
  Y : T1
  T2 : Veir.RegisterType
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_sraiw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: sraiw(sextw(X))  =>  sraiw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = sraiw(a0) : T1
  P1 = properties(root)
  new2 = sraiw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_slliuw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: slliuw(sextw(X))  =>  slliuw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = slliuw(a0) : T1
  P1 = properties(root)
  new2 = slliuw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_clzw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: clzw(sextw(X))  =>  clzw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = clzw(a0) : T1
  P1 = properties(root)
  new2 = clzw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_ctzw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: ctzw(sextw(X))  =>  ctzw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = ctzw(a0) : T1
  P1 = properties(root)
  new2 = ctzw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_cpopw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: cpopw(sextw(X))  =>  cpopw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = cpopw(a0) : T1
  P1 = properties(root)
  new2 = cpopw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```

Veir.RISCV.drop_sextw_zextw

Source

Builder families: `Veir.RISCV.drop_ext_low_word_pattern`.

Parameters: `index = 0`

```
riscv: zextw(sextw(X))  =>  zextw(X)
where
  a0 = sextw(X) : T0
  P0 = properties(a0)
  root = zextw(a0) : T1
  P1 = properties(root)
  new2 = zextw(X) : T1 [created]
  P1 = properties(new2)
  X : T0
  T1 : Veir.RegisterType
  T0 : Veir.RegisterType
```